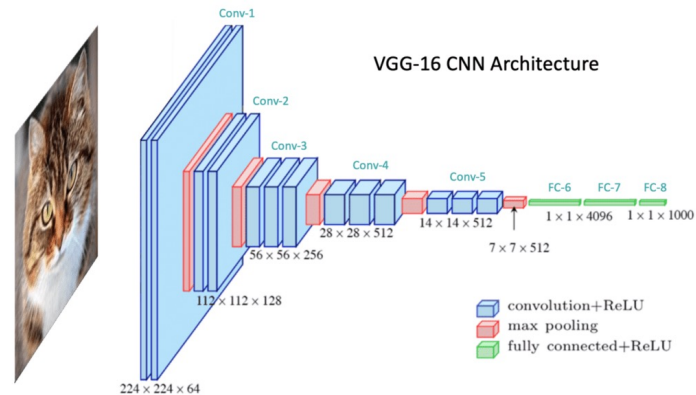
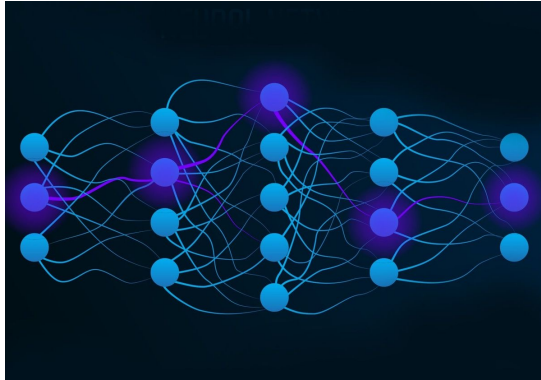




Introduction to Neural Networks

Deep Learning

Announcements



- Next class: Wednesday 8am @ Dansby Classroom
- Thursday: No class

What happens when we take a linear function of linear functions?

$$g = 5a + 7b - 3c$$

$$h = -3a + b + 2.5c$$

Suppose $z = 3g + 2h$

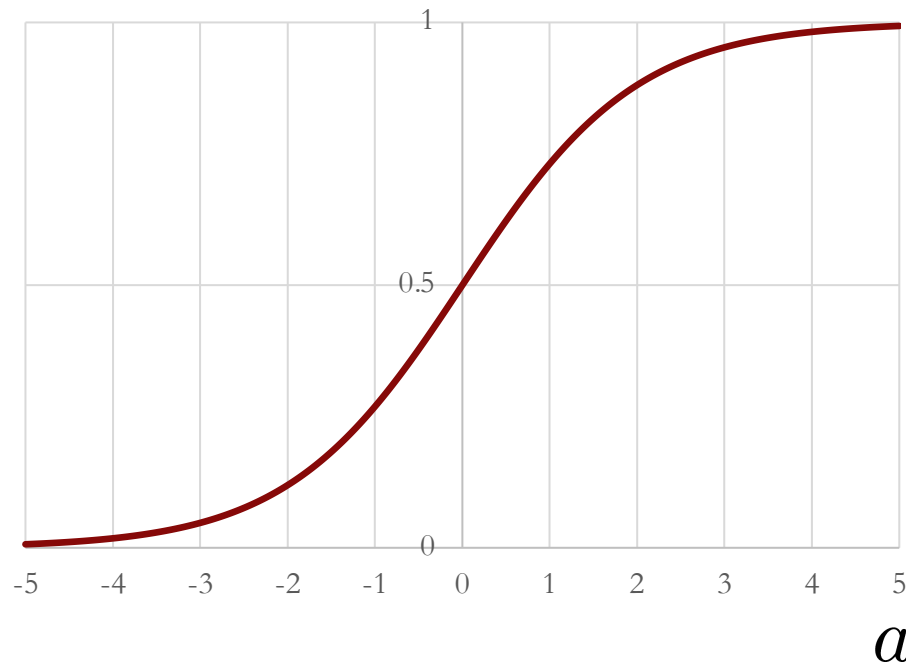
Then ... $z = 9a + 23b - 4c$

“A linear function of linear functions is (just) another linear function”

Sigmoid function

Sigmoid function:

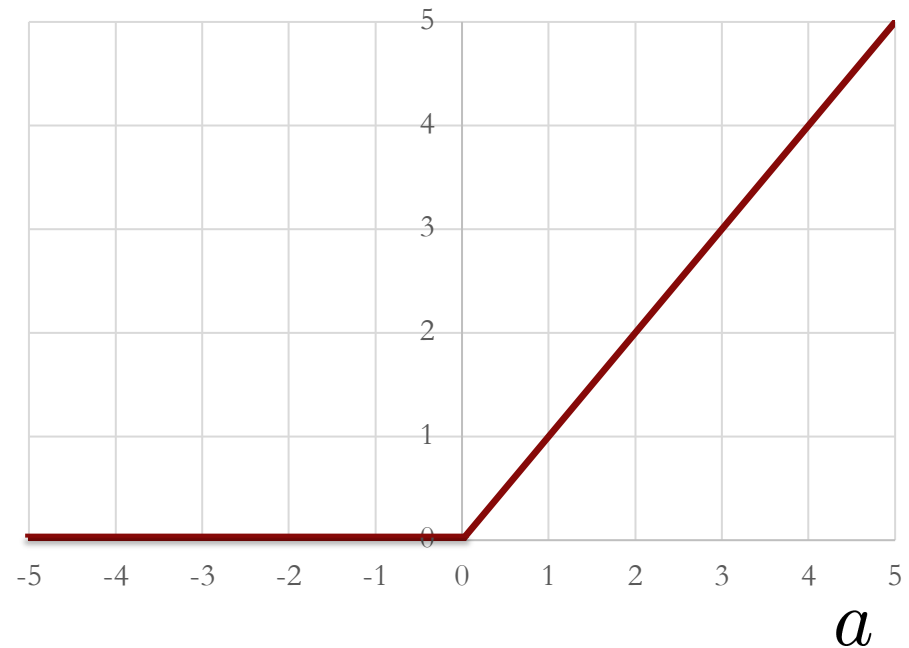
$$\sigma(a) = \frac{1}{1 + e^{-a}}$$



$\max(0, a)$

$$f(a) = \max(0, a)$$

- ReLU



Linear Regression

$$\begin{aligned}\text{Actual Solar radiation} = & 0.2262 \\ & + 0.6757 \times \text{Rad Forecast} \\ & + 0.006541 \times \text{Temp} \\ & - 0.002779 \times \text{Humidity} \\ & + 0.003281 \times \text{Wind U} \\ & - 0.005324 \times \text{Wind V} \\ & - 0.0001515 \times \text{Cloud Cover}\end{aligned}$$

Linear Regression: sumproduct

$$\begin{aligned}\text{Actual Solar radiation} = & 0.2262 \\ & + 0.6757 \times \text{Rad Forecast} \\ & + 0.006541 \times \text{Temp} \\ & - 0.002779 \times \text{Humidity} \\ & + 0.003281 \times \text{Wind U} \\ & - 0.005324 \times \text{Wind V} \\ & - 0.0001515 \times \text{Cloud Cover}\end{aligned}$$

Technology Using Deep Learning

What do all these use-cases have in common?



Medical Imaging



Self-Driving Cars



Voice Assistants

Customer reviews

★★★★★ 4.6 out of 5

10,134 global ratings

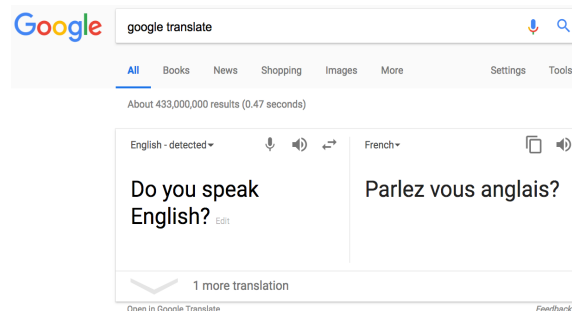
Customers say

Customers like the sound quality, quality, and ease of installation of the sound and recording equipment. They mention that it does the job quite well as a pop filter and is good value for money. Customers are also satisfied with the sound clarity, quality and ease to installation. However, some customers are mixed on stability, fit, and flexibility.

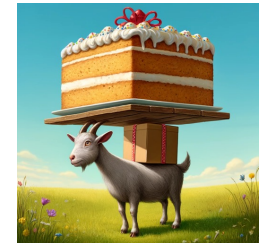
AI-generated from the text of customer reviews

Summarization

>



Translation



Generative AI

Structured vs. Unstructured Data

- **Structured Data** – data which can be naturally stored in an excel spreadsheet or SQL database.

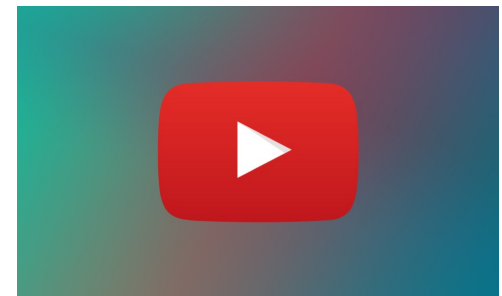
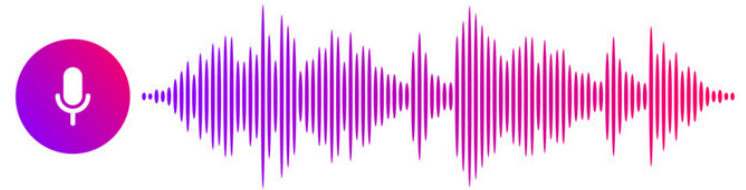
Independent Variables					Target
Age	BMI	Glucose	Blood Pressure	Insulin	Diabetes
50	33.6	148	72	220	1
31	26.6	85	66	93	0
32	23.3	183	64	301	1
21	28.1	89	66	94	0
33	43.1	137	40	168	1
30	25.6	116	74	102	0
26	31	78	50	88	1
29	35.3	115	70	320	0
53	30.5	197	70	543	1

Structured vs. Unstructured Data

- **Unstructured Data** – data which is not so readily represented as entries in a table/matrix/spreadsheet



“It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief,”



Raw Form of Unstructured Data

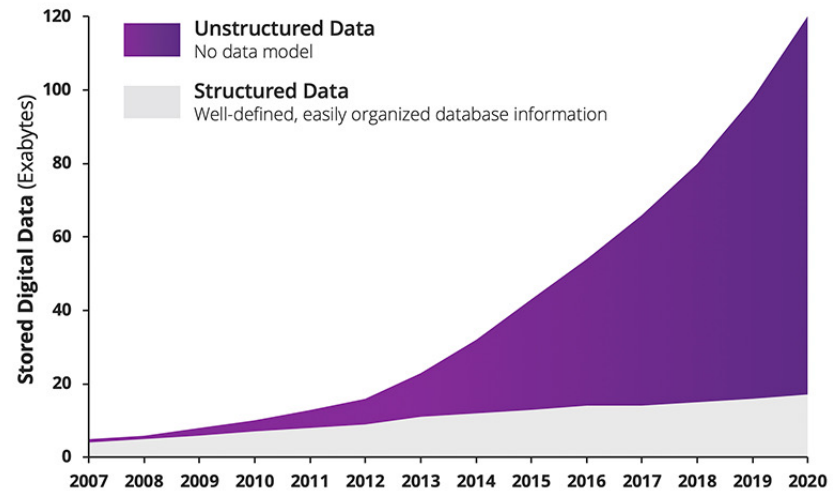
- Unstructured data may also come in an excel file, but the individual features are not meaningful.



	Pixel Number				
Species	1	2	3	...	16,384
American Robin	ff9f3f	ed9f4f	fd9f3c		fdfefd
Bluejay	f3f4f1	e3d5f1	e3d5f1		2fff2f
Northern Cardinal	0fff2f	0fff2f	0fff2f		0fff2f

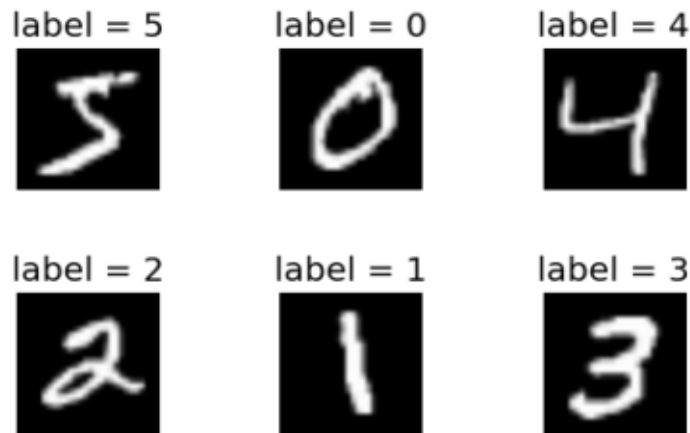
Most Data is Unstructured

- Unstructured data vastly outnumbers structured data.
 - 720,000 hours of videos are uploaded to YouTube each day.
 - 100 billion text messages daily through WhatsApp, Messenger...etc.
 - 5.3 billion photos are taken every day on smartphones or cameras.



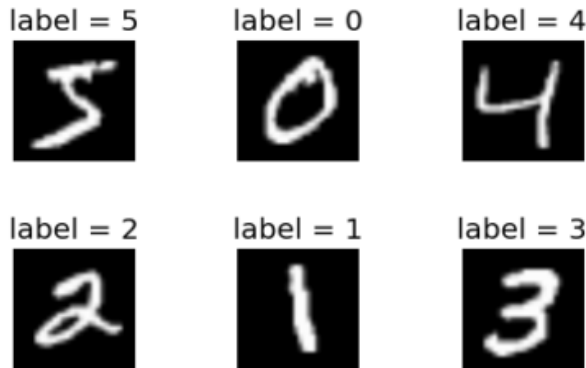
How can we make predictions from unstructured data?

Running Example: Handwriting



- Modified National Institute of Standard and Technology Dataset (MNIST)
 - 70,000 grayscale images of handwritten digits (0, 1, 2, 3, ..., 9)
 - 28 pixels by 28 pixels each (784 pixels per image)
 - Widely used for benchmarking in computer vision (i.e., image processing)
- Why is handwriting recognition important?

How might we classify images?



Handwriting → Digits



Birds → Species

Let's think through how we might do this with traditional predictive models (e.g., Logistic Regression, CART)

Attempt 1: Traditional Predictive Models on Raw Data

- Turn images into tables with columns as pixel intensities:
 - **BW Image:** Pixel 1 intensity, Pixel 2 intensity...etc.
 - **Color Image:** Pixel 1 **RGB**, Pixel 2 **RGB**, ...etc.
- Run a logistic regression model.
 - Independent Variable = Pixel Intensities
 - Dependent Variable = Species / Digit



Species	Pixel Number				
	1	2	3	...	16,384
American Robin	ff9f3f	ed9f4f	fd9f3c		fdfefd
Bluejay	f3f4f1	e3d5f1	e3d5f1		2fff2f
Northern Cardinal	0fff2f	0fff2f	0fff2f		0fff2f

Attempt 1 Performs Poorly

It is very challenging for a model to predict the digit or bird species directly from the pixel intensities.

- Complex relationships between pixels.
 - Pixel 42 on its own provides no information.
 - Pixels 42, 237, 1208...etc. together provide little information.
 - A large patch of pixels together provides more information.
- Complex properties which define each label class.
 - So many different handwriting styles for each number (e.g., do you loop your 2?)
 - Identifying bird species is even hard for a human

Predicting species/digits from pixels feels like jumping into the deep end.

Attempt 2: Defining Features

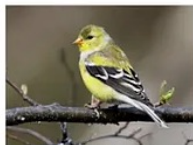


Image Number	Correct Label	Number of Loops	Has Vertical Straight Line?	Has Horizontal Straight Line	...
1	8	2	No	No	...
2	2	1	No	Yes	...
...	...	...	...	...	...
10,000	9	1	Yes	No	...

- Leverage domain knowledge to define features of interest.
 - Train traditional models to extract these features, a much more manageable task than predicting the digit outright.

Defining Features for Birds

“Unstructured” data



Manual feature extraction

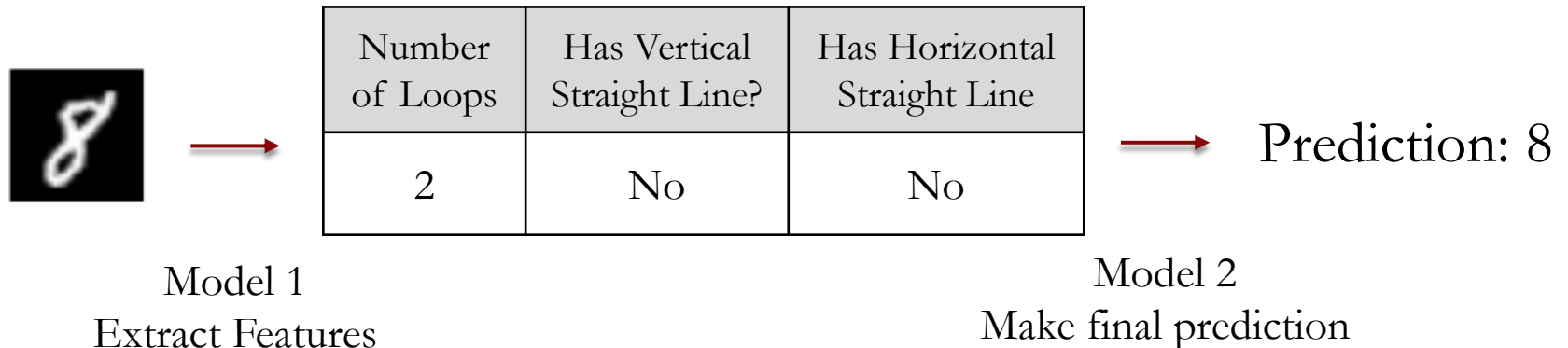


Datasets in “structured”
form like tables

	Beak length	Wing span	Primary color	...
Bird 1	3 cm	43 cm	black	...
Bird 2	0.5 cm	19 cm	yellow	...
Bird 3	5 cm	48 cm	black	...
Bird 4	4 cm	45 cm	black	...
...	...	...	...	...

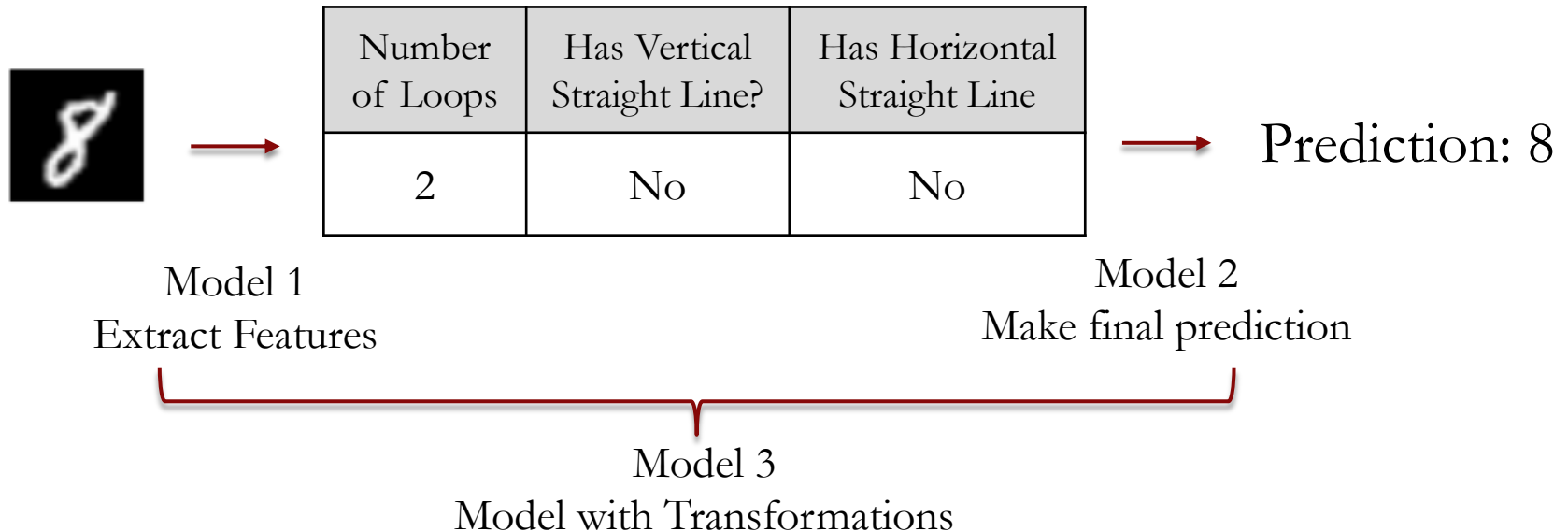
An example of a tabular dataset for reference

Two-Stage Pipeline



- This two-stage pipeline involves transforming the raw data into useful features (a representation), then applying a final model.
- Disadvantages to this Approach:
 - Requires human knowledge to define the features
 - Requires human labeling to train models 1 and 2.

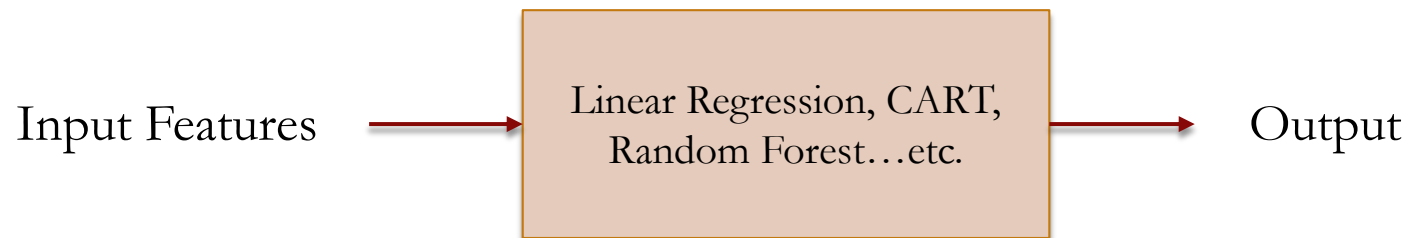
What is Different About Deep Learning?



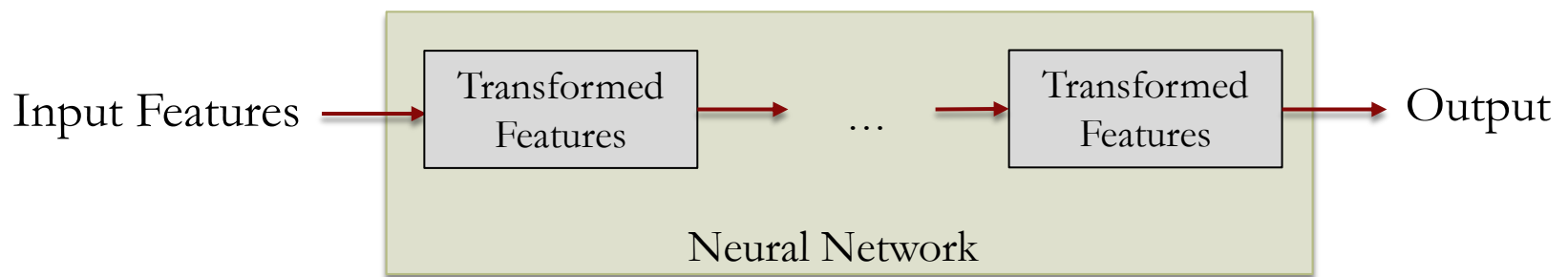
- A deep learning model is a type of predictive model that transforms input data through a sequence of interconnected layers.
 - Each layer learns patterns in the data in a hierarchical fashion
 - Designed for prediction tasks involving unstructured data.

Traditional vs. Deep Learning Models

Traditional Predictive Model

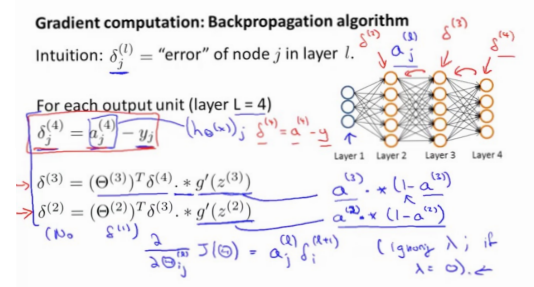
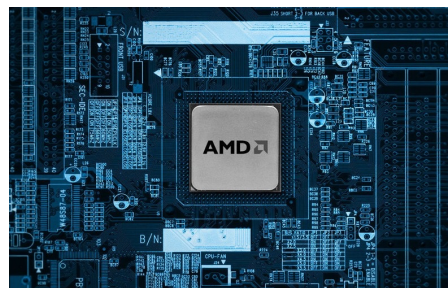


Deep Learning Model



Why Now?

- Neural networks were developed in the 1950's, but why did deep learning only take off after the 2010's?
 1. **Big Data.** Availability of enormous datasets to train the models (Wikipedia, Twitter...etc.)
 2. **Hardware.** Graphics Processing Units (GPUs)
 3. **Algorithms.** Improved algorithms for training neural networks



Deep Learning Lecture Overview

- Introduction to (Feed-Forward) Neural Networks
 - **Predictions.** How does a feed-forward neural networks make predictions?
 - **Setup and Training.** How does one learn the weights of a NN?
 - Basic image and text classification
- Modern Deep Learning Architectures
 - Convolutional Neural Networks
 - Word and Sentence Embeddings
 - Transfer Learning

Feed Forward Neural Networks

Re-View of Linear and Logistic Regression

The “original” neural network

Predict Prices of Used Toyota Corollas

	Price	Age	KM
1	13500	23	46986
2	13750	23	72937
3	13950	24	41711
4	14950	26	48000
5	13750	30	38500
6	12950	32	61000
7	16900	27	94612
8	18600	30	75889
9	21500	27	19700
10	12950	23	71138
...	...	...	...
1435	7250	70	16916
1436	6950	76	1

- Linear regression model to predict the price of a used Toyota Corolla
- Dependent variable: Price (in Euros) of a used Toyota Corolla
- Independent variables:
 - Age of vehicle (in months)
 - Kilometers driven (“mileage”)

Simple linear regression example

- Linear regression model coefficients:

```
=====
Regression Results
=====
```

```
R-squared: 0.7900
```

```
No. Observations: 1436
=====
```

```
Coefficients:
-----
```

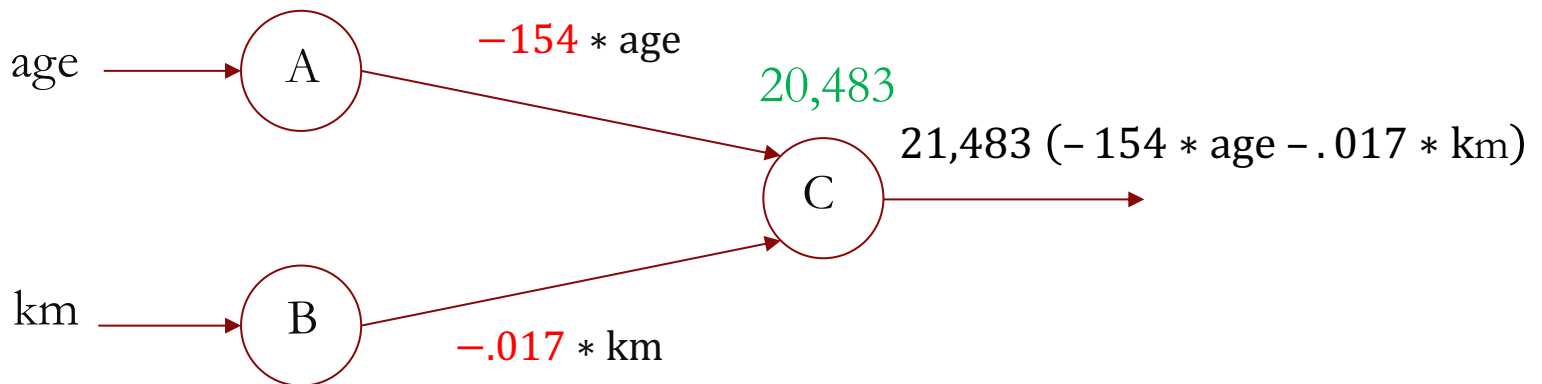
	Coef.	Std.Err.	t	P> t	[0.025	0.975]	Signif
Intercept	20483.1135	140.0473	146.2585	0.0	20208.3938	20757.8333	***
Age	-154.1492	2.7363	-56.3342	0.0	-159.5168	-148.7815	***
KM	-0.0165	0.0014	-12.1302	0.0	-0.0191	-0.0138	***

```
=====
Significance codes: '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1
=====
```

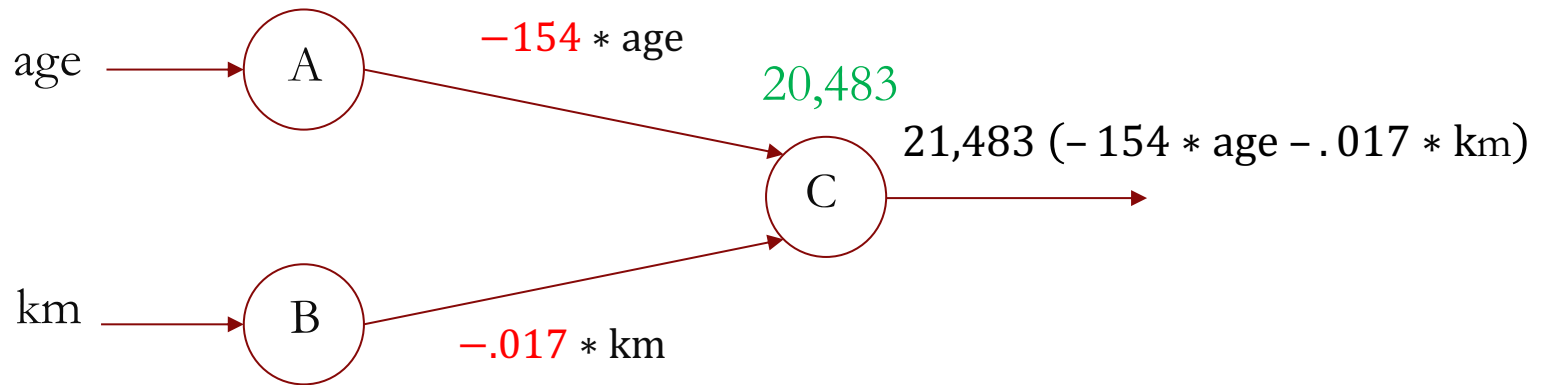
$$\text{Price} = 20,483 - 154 * \text{age} - .017 * \text{kilometers}$$

Linear Regression as a Network

- Here is a network portrayal of
$$\text{Price} = 20,483 - 154 * \text{age} - .017 * \text{kilometers}$$



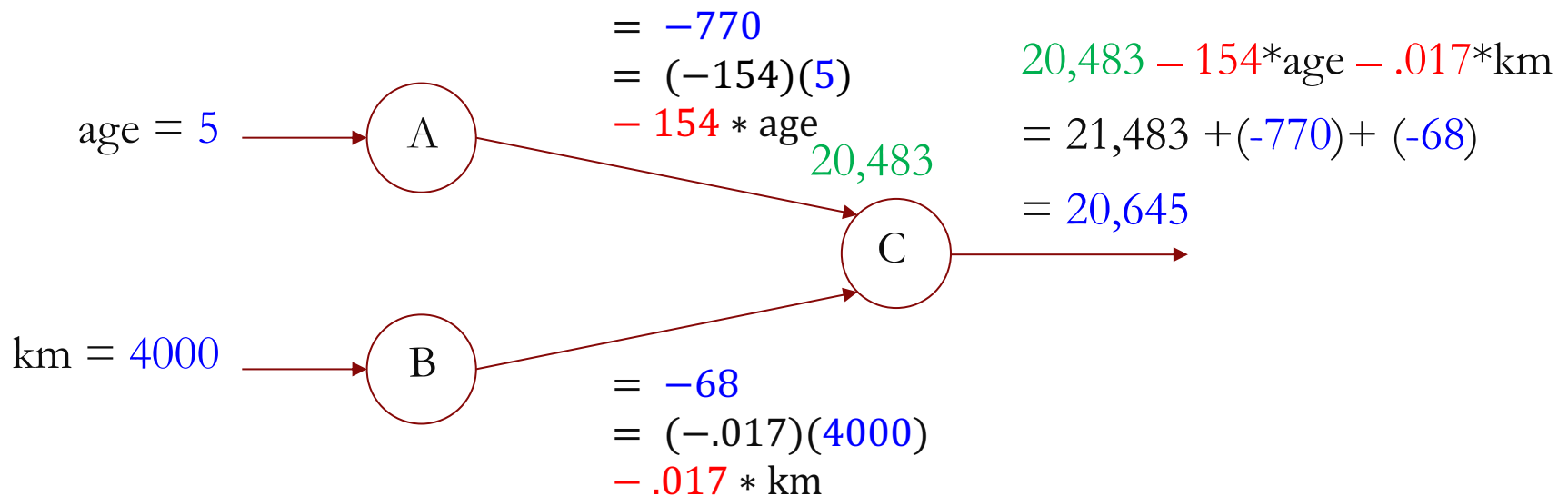
Weights and Biases



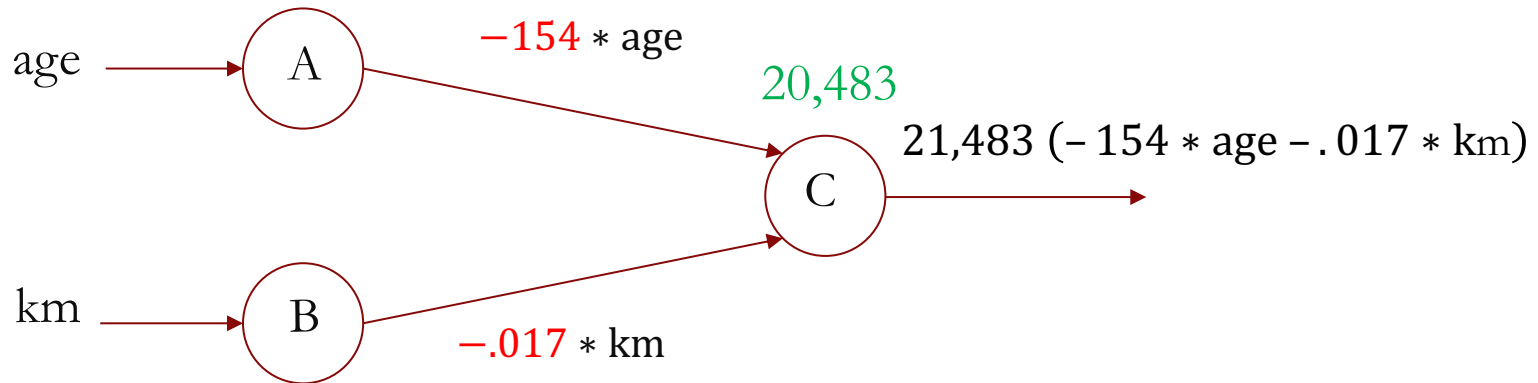
- Neural networks have their own terminology (sigh) ...
 - Multipliers on the values from each node are the **weights**
 - Additive shift is called the **bias**

Prediction with network description

- What is the predicted price of a 5-month old Toyota Corolla (relative to August 2004) that has been driven 4000 kilometers?



“The model”



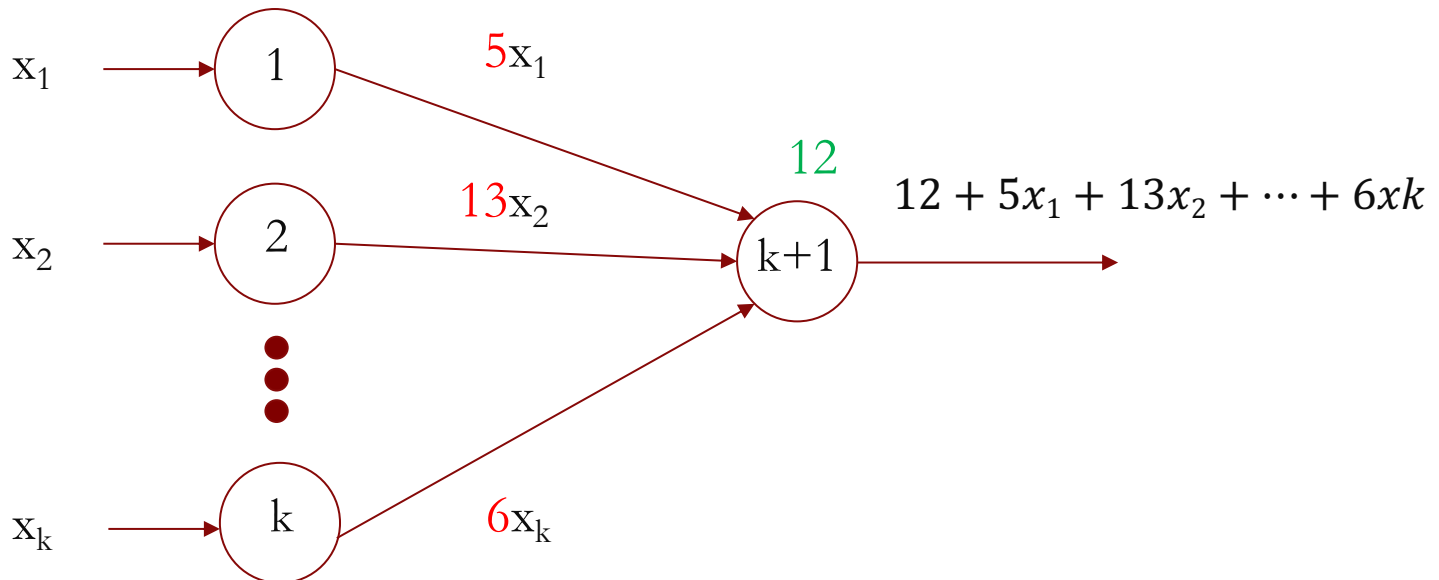
- “The model” is

$$\text{Price} = 20,483 - 154 * \text{age} - .017 * \text{kilometers}$$

- **Weights** ($-154, -.017$) and **bias** ($20,483$) computed to minimize sum square error

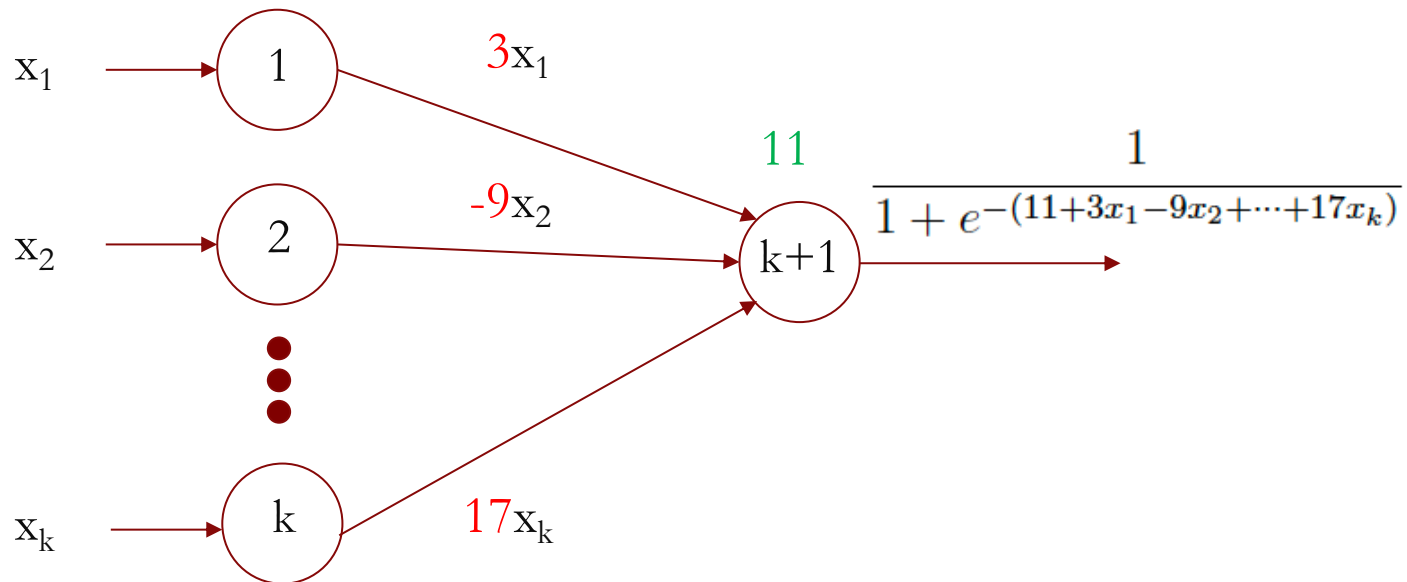
Network for General Linear Regression

- Model equation: $y = 12 + 5x_1 + 13x_2 + \cdots + 6x_k$



Network for Logistic Regression

- Model equation:
$$P(Y = 1) = \frac{1}{1 + e^{-(11+3x_1-9x_2+\dots+17x_k)}}$$



Describing a Neural Network

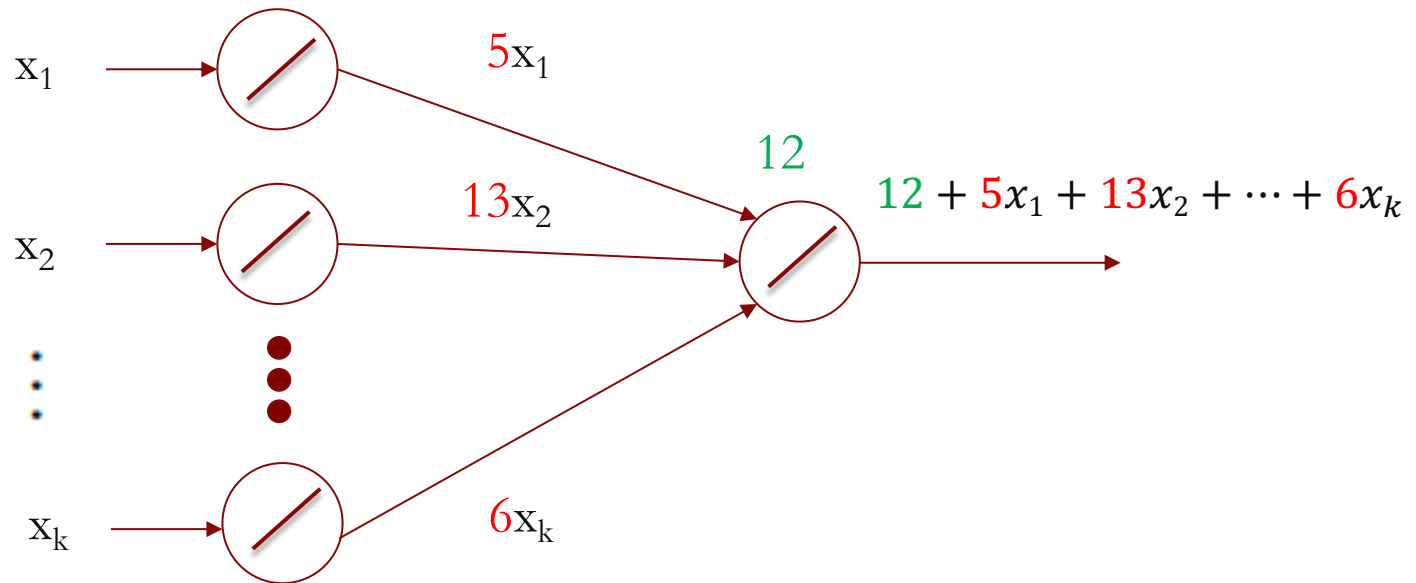


- Activation function (of a node)
- Layers and Network “shape” or “structure”

Activation functions

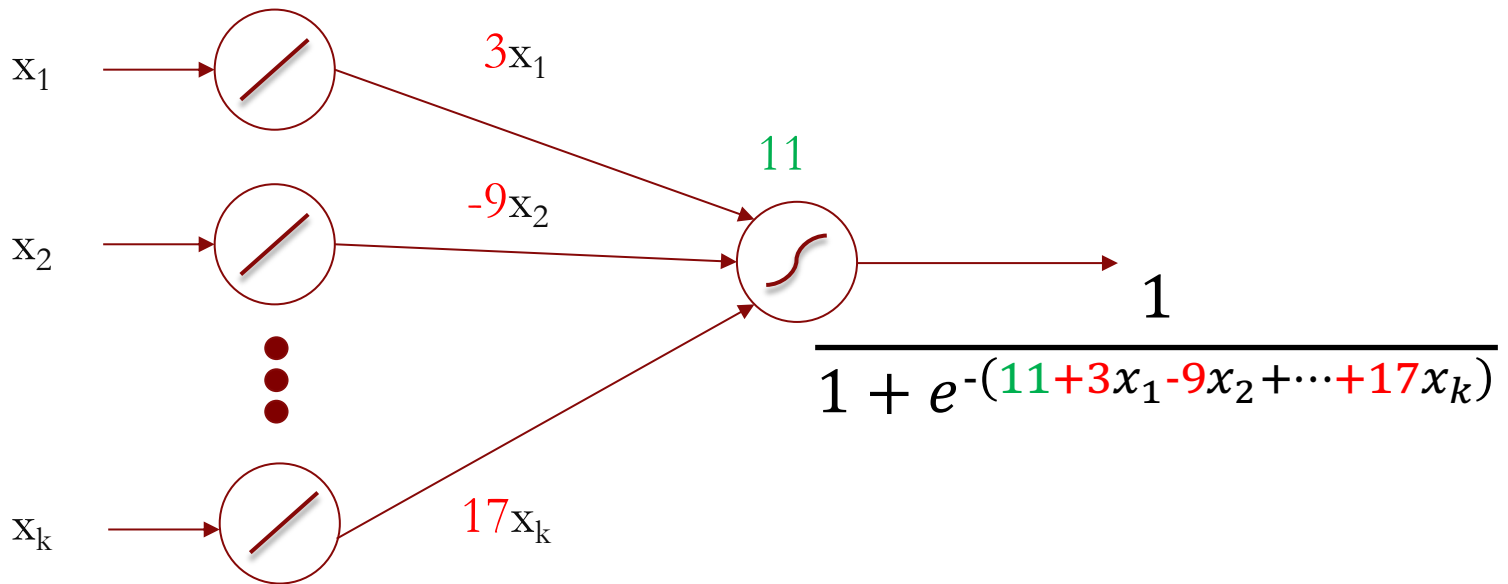
The activation function of a node is the arithmetic that is done at that node in the network

Activation functions for a linear regression model



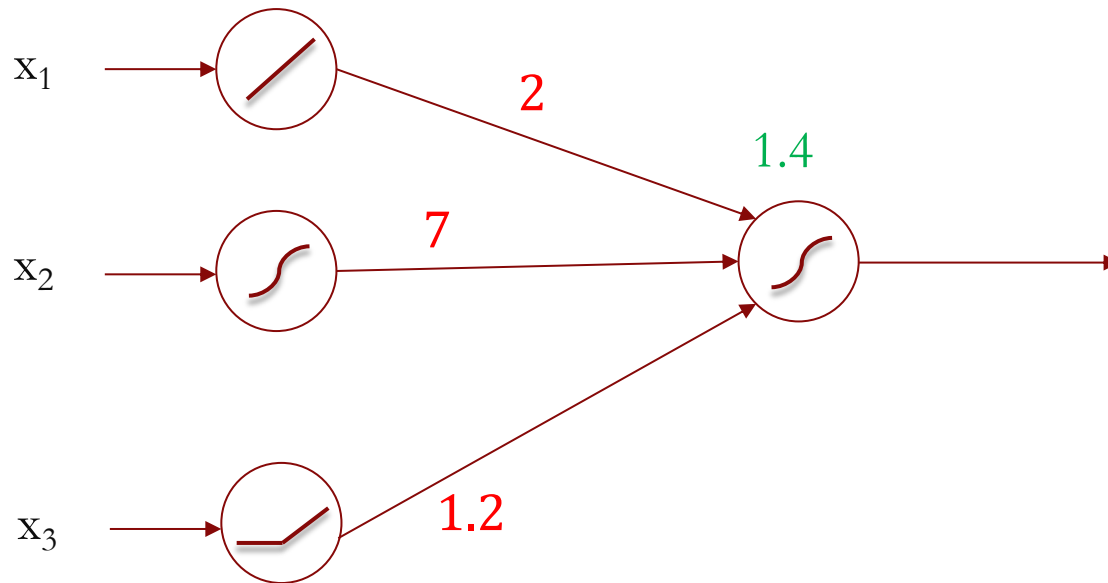
- Every activation function is a linear function of the input value, for instance $f = 12 + 5x_1 + 13x_2 + \dots + 6x_k$

Activation functions for a logistic regression model



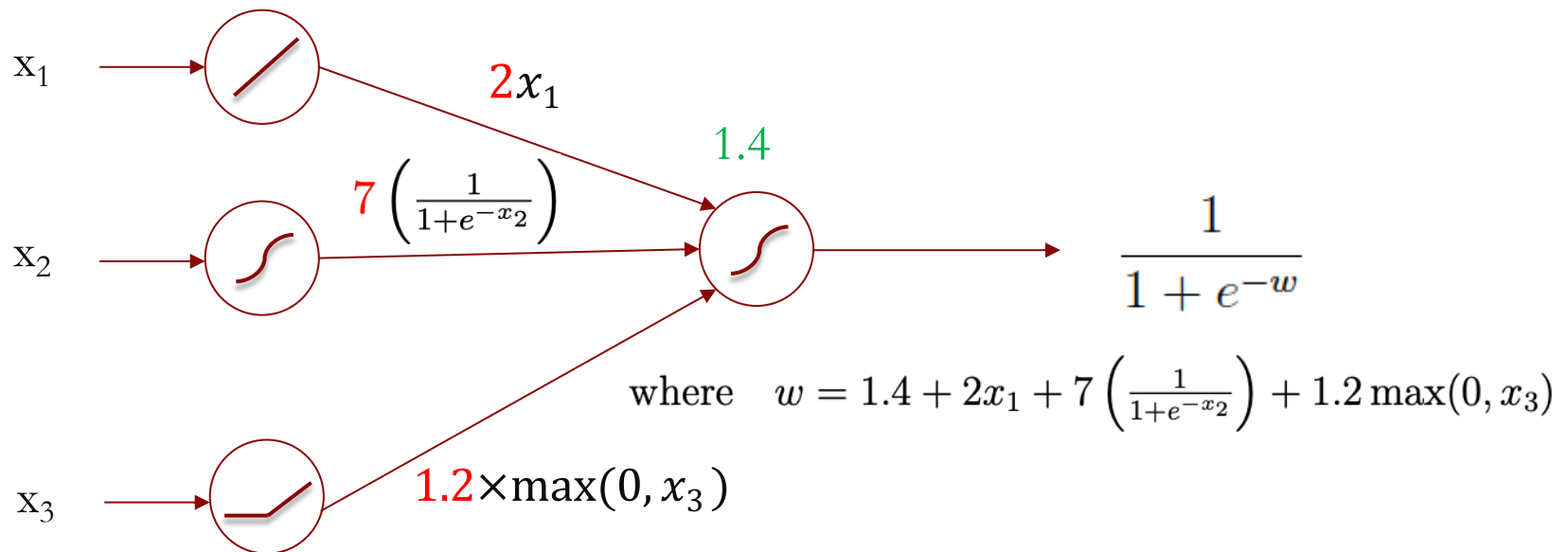
- Here the activation functions of the input nodes are linear, and the activation function of the output node is the sigmoid function

Activations functions for a more general model



Hmm, this is starting to look complicated. What does the final function look like?

General Model, continued



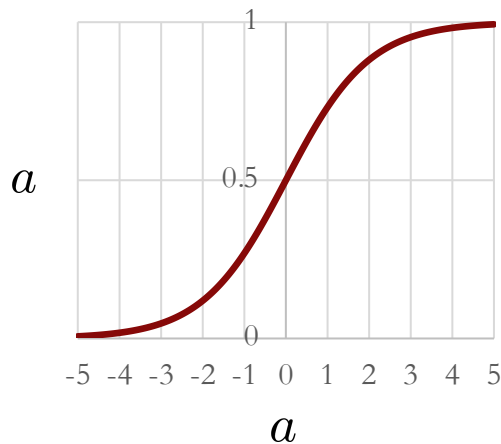
where $w = 1.4 + 2x_1 + 7 \left(\frac{1}{1+e^{-x_2}} \right) + 1.2 \max(0, x_3)$

$$f(x_1, x_2, x_3) = \frac{1}{1 + e^{-\left[1.4 + 2x_1 + 7 \left(\frac{1}{1+e^{-x_2}} \right) + 1.2 \max(0, x_3) \right]}}$$

Common Activation Functions

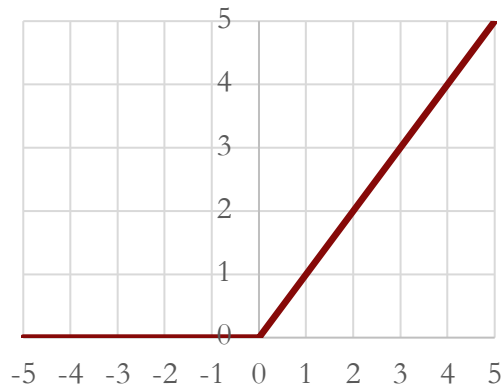
Rectified Linear Unit

Sigmoid



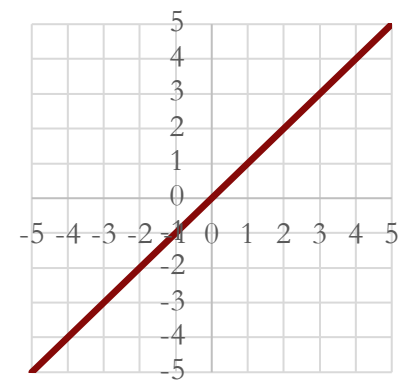
$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

ReLU



$$g(a) = \max(0, a)$$

Linear



$$f(a) = a$$

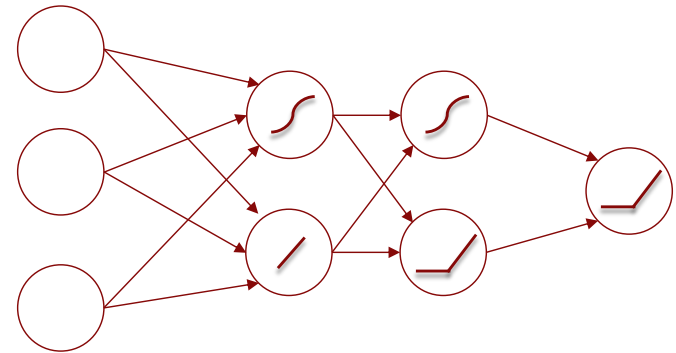
Remarks on Activation Functions

- The activation function is a crucial part of the neural network methodology because it is where more complex functional forms (nonlinear ones in particular) are composed
- Without the activation function, neural networks would just reduce to an unnecessarily complicated representation of standard linear regression
- Historically, sigmoid and hyperbolic tangent activation functions were prevalent for hidden nodes
- Surprisingly, the ReLU activation function has become **the primary choice** in recent years for all nodes except the input nodes and the output node
- ReLU also results in better behavior in optimization routines (but this is not so well-understood)

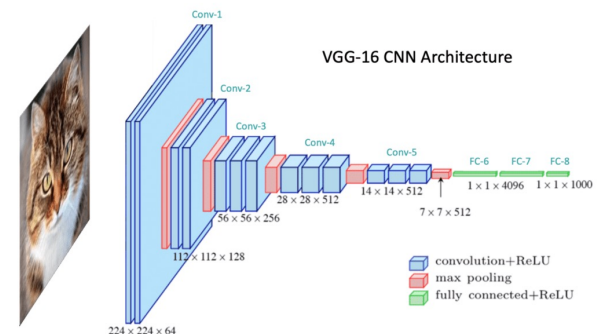
Layers and Network Structure

Deep Learning vs Neural Network

- The term neural network is broadly used to describe pretty much any deep learning model.
- The original neural network is now called a feed forward neural network
- There are many other types of neural networks (later in class)



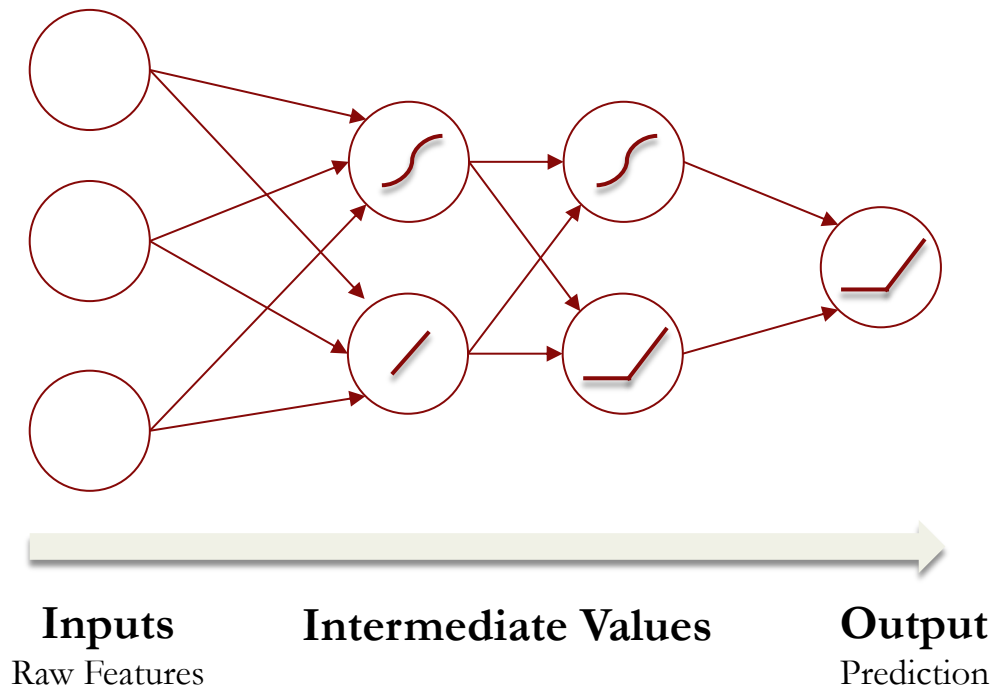
Feed-Forward Neural Network



Convolutional Neural Network

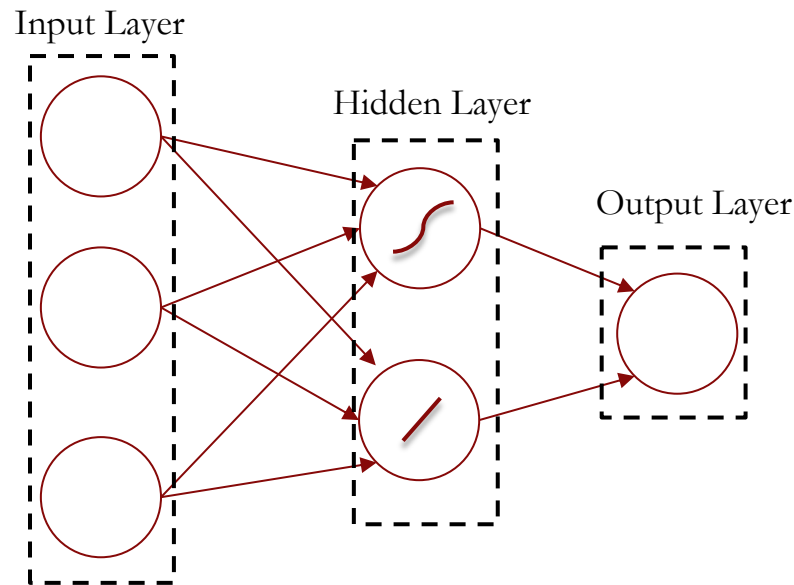
Feed Forward Neural Network

- **Intuition.** Many linear/logistic regressions applied sequentially

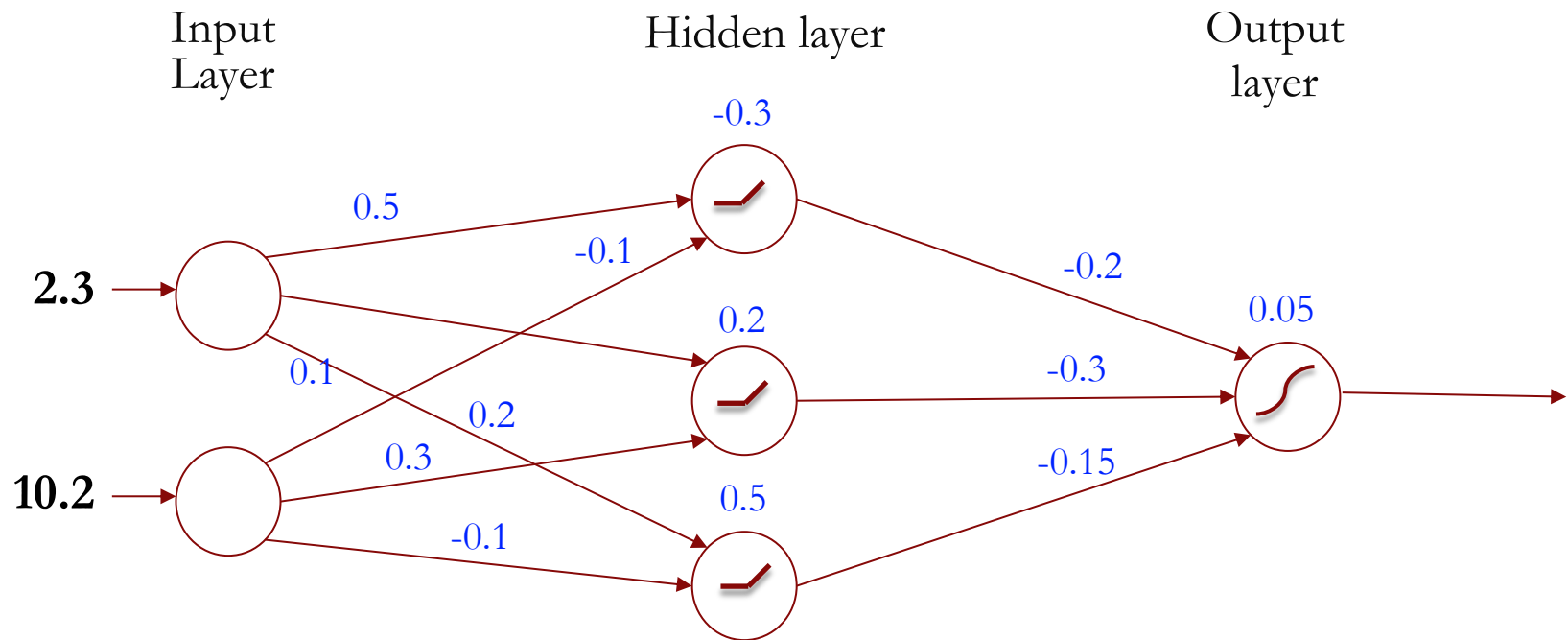


Notational Shorthand and Terminology

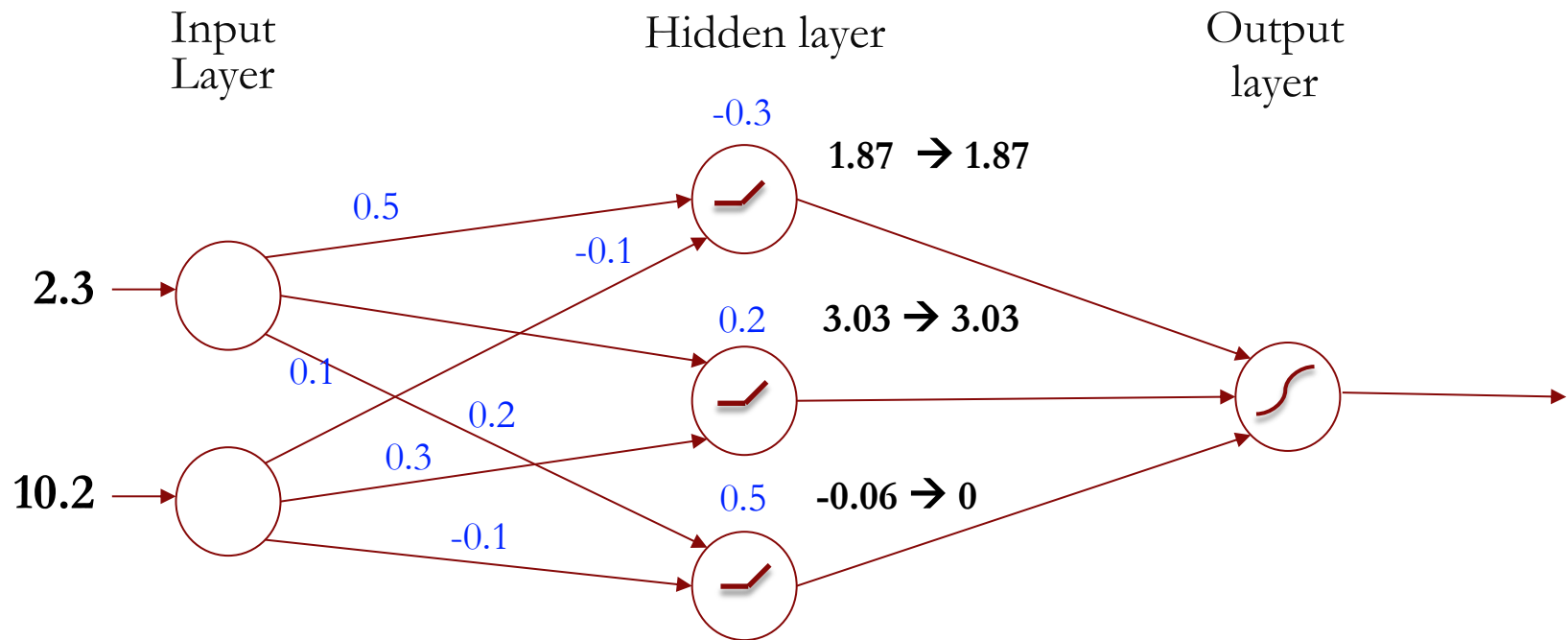
- Let's simplify the notation and define some terminology.
 - **Layers:** the stages of the input as it is transformed (input, hidden, output)
 - **Neuron** (circles): nodes in each layer where computation is performed.



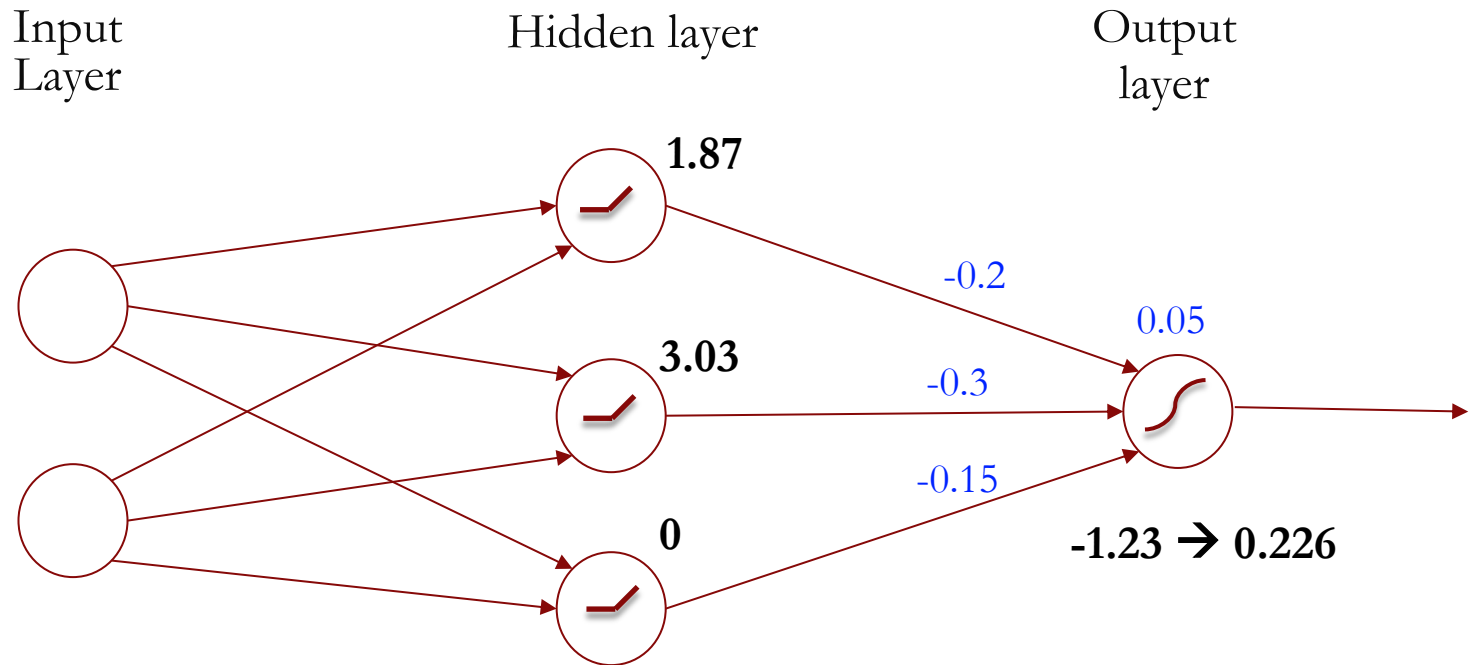
Making Predictions Example



Making Predictions Example

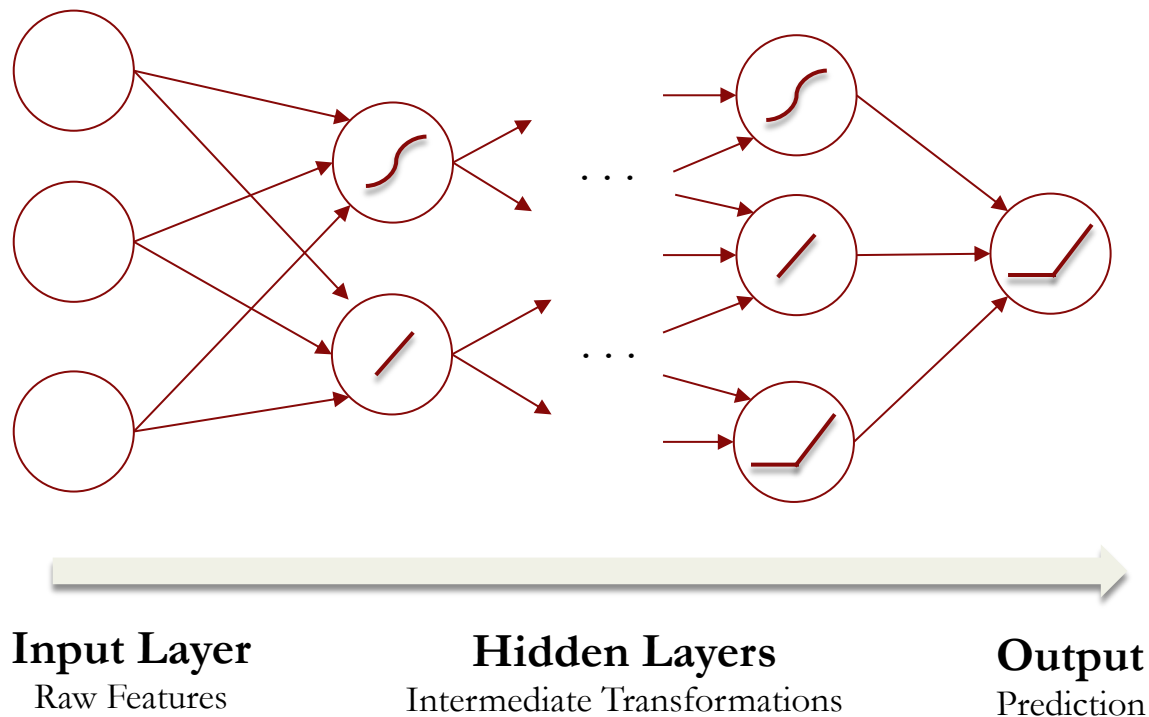


Making Predictions Example

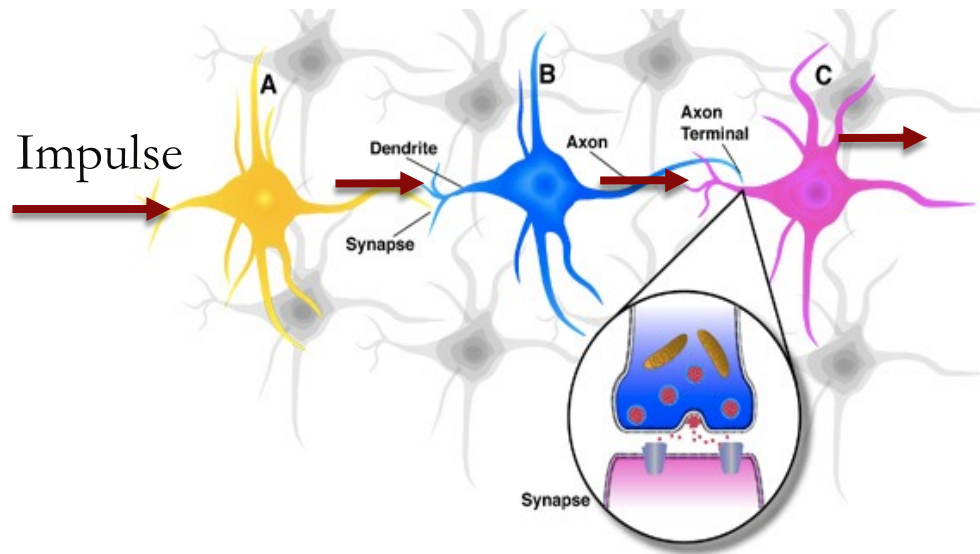


Many Hidden Layers

- Typically, a neural network will have numerous hidden layers.

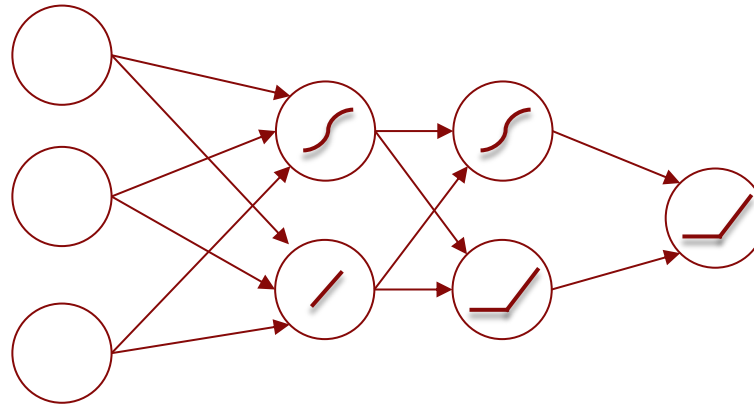


Why the Name “Neural Network”?



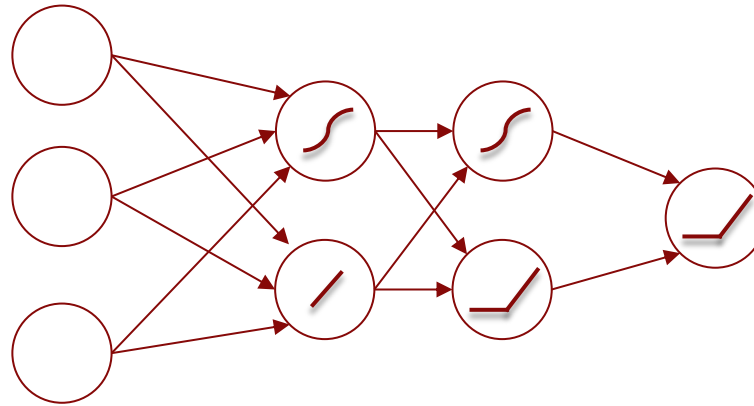
- In the brain, the dendrites of one neuron connect to the axons of other neurons at synapses, where highly activated synapses convey the information to other neurons
- This is a physical neural network. Please do not read too much into the analogy...

Summary



- A **feed forward neural network** takes an input and passes it through multiple ____, each of which consists of one or more ____.
- Each ____ performs a weighted sum based on a set of ____ and a ____.
- This weighted sum is then passed through an ____ function that introduces non-linearity.

Summary



- A **feed forward neural network** takes an input and passes it through multiple layers, each of which consists of one or more neurons.
- Each neuron performs a weighted sum based on a set of weights and a bias.
- This value is then passed through an activation function that introduces non-linearity.

Setting Up a Neural Network

Setting up a Neural Network

Suppose I have a dataset and a prediction task that I would like to apply a neural network to. What do I need to do?

Step 1. Specify the architecture

1. How many hidden layers?
2. How many neurons in each hidden layer?
3. What activation functions for each neuron?

Step 2. Train the model: find weights and biases

1. Randomly initialize weights and biases
2. Define a loss function depending on your task
3. Use gradient descent to optimize the loss function. Decide when to stop optimizing.

Aside: Multi-Class Classification

- In this course so far, we've only looked at binary classification (1/0, parole/no parole, diabetes/no diabetes).
 - Models provide a **probability that the target is 1**.
 - Uses the sigmoid function
- Multi-class classification involves classifying into more than 2 classes (digits, bird species).
 - Models provide a probability for each class (which sum up to 1).
 - Uses the **softmax** function (see next page)

Predicted Probability for Each Digit									
0	1	2	3	4	5	6	7	8	9
0.11	0.02	0.03	0.02	0.08	0.55	0.1	0.06	0.03	0

Softmax Activation Function

Binary classification: sigmoid function

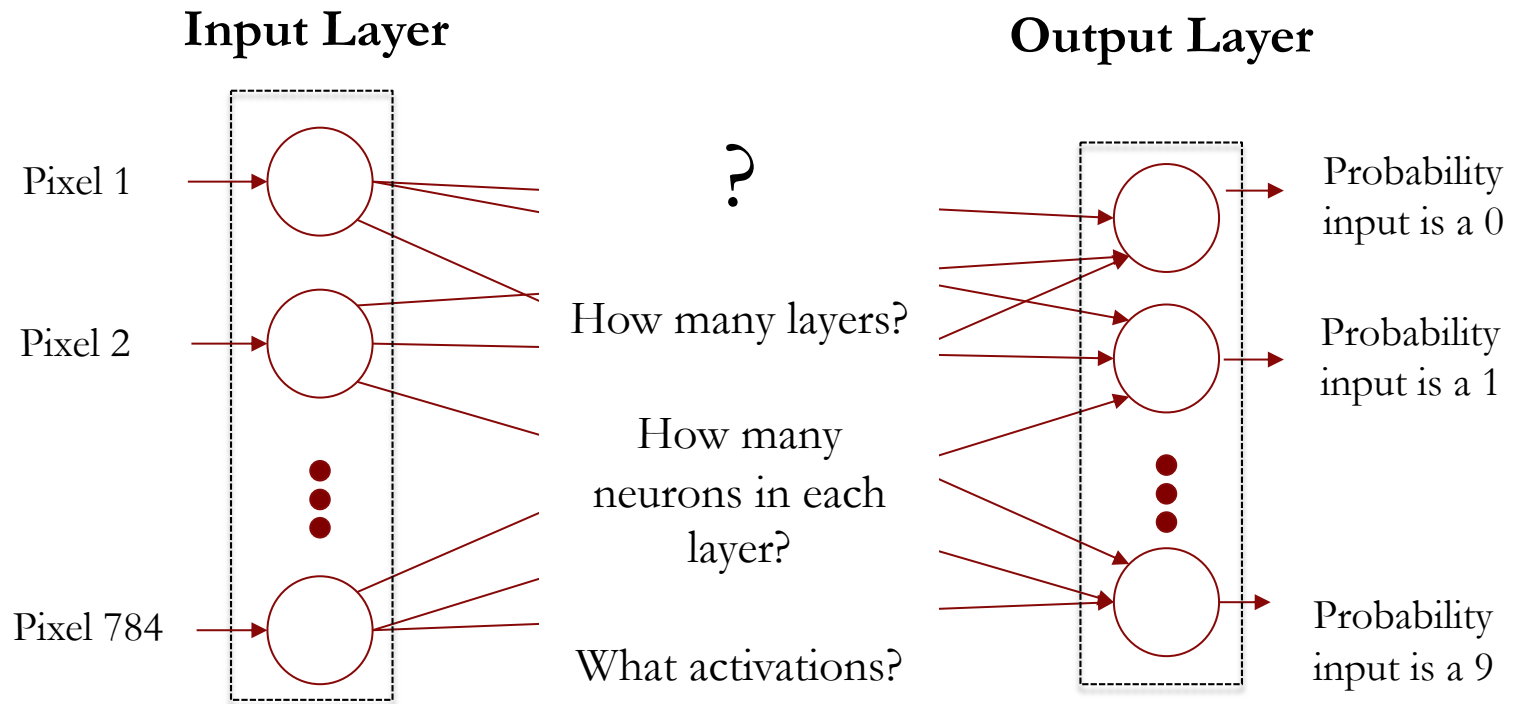
$$\frac{1}{1 + e^{-a}} = \frac{e^a}{e^a + 1} = \frac{e^a}{e^a + e^0}$$

Generalize to multiple types: ensure that the sum of the category probabilities equals one, the **softmax** activation function is used at the output node to create nonnegative numbers that sum to 1.

$$\text{softmax}(a_1, a_2, \dots, a_k) = \left(\frac{e^{a_1}}{\sum_{i=1}^k e^{a_i}}, \frac{e^{a_2}}{\sum_{i=1}^k e^{a_i}}, \dots, \frac{e^{a_k}}{\sum_{i=1}^k e^{a_i}} \right)$$

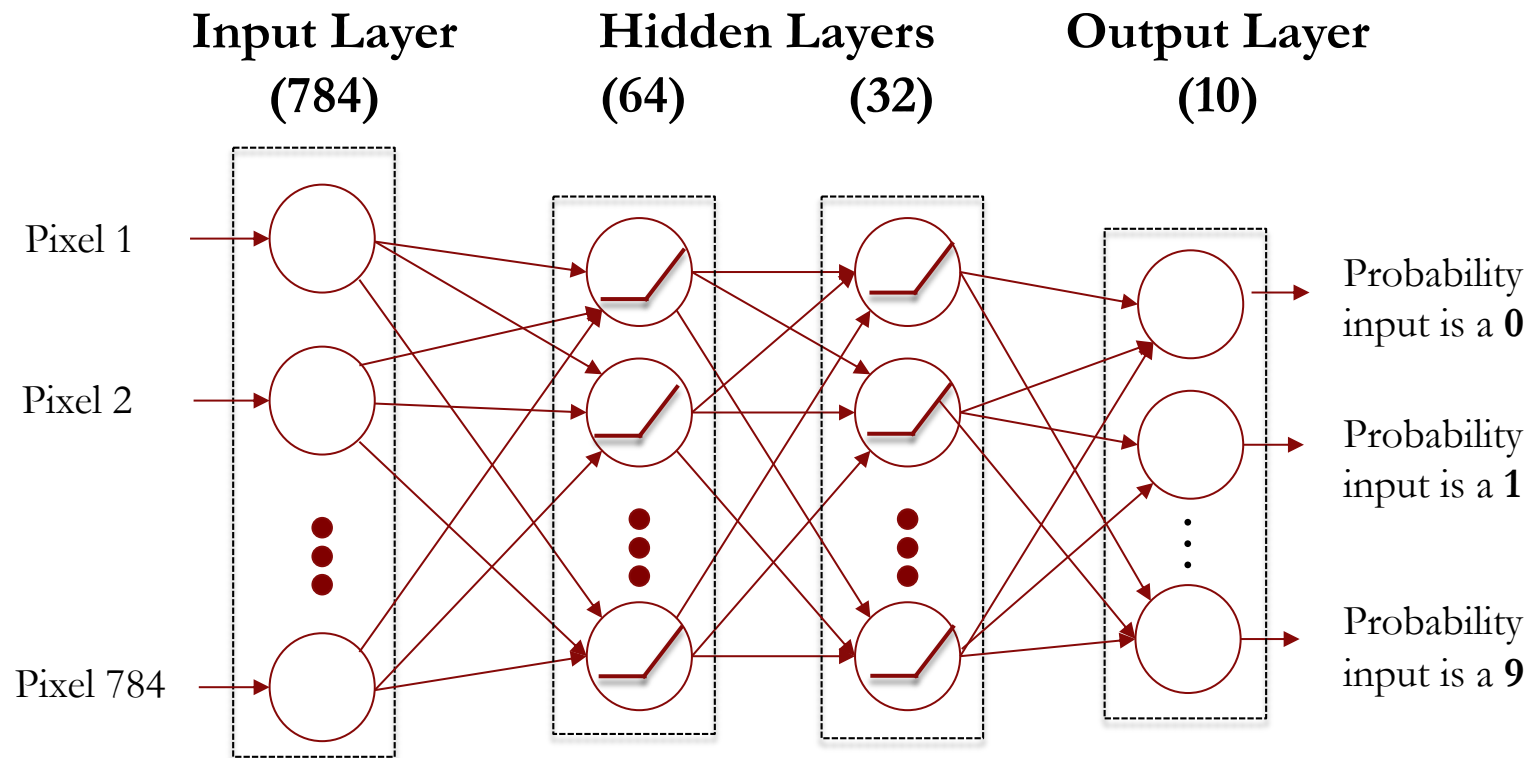
Specify the Architecture

- The input and outputs are fixed, but everything in the middle can be customized.



Example: 2 Hidden Layers

- Add two hidden layers, one with 64 neurons and another with 32 neurons. Choose ReLU for all neurons.



This is Easy to Do in Python

- TensorFlow from Google makes it easy to code up neural networks.

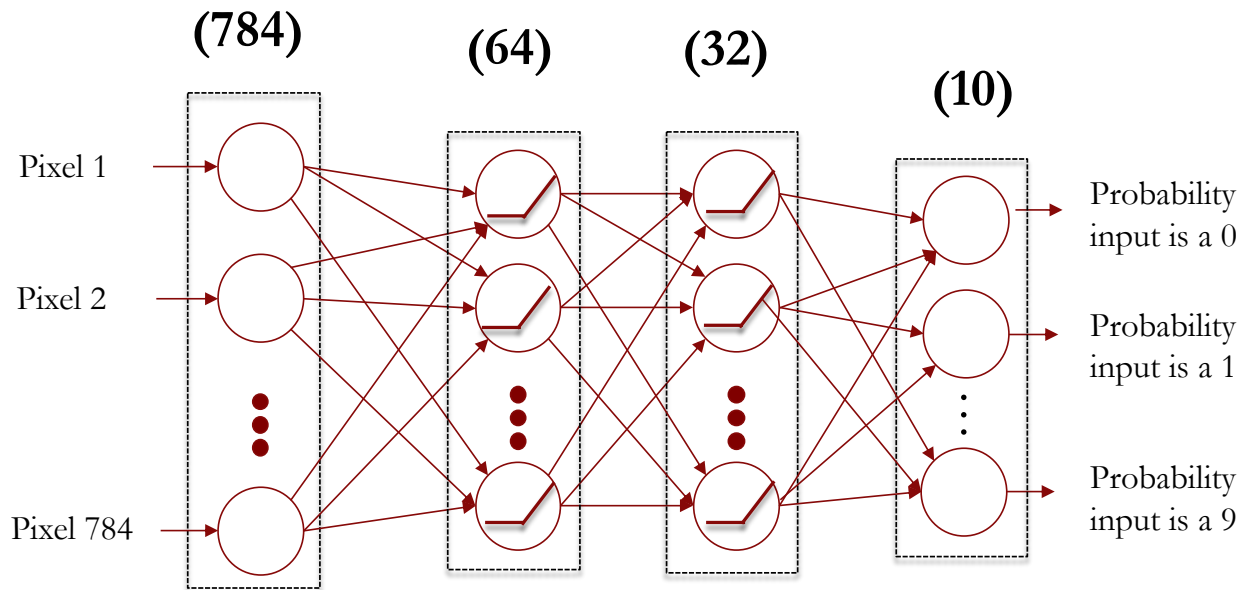


```
from keras import Sequential
from keras.layers import Input, Dense

model = Sequential([
    Input(shape=(784,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(10, activation='softmax'),
])
```

Number of Weights

- How many weights and intercepts are in this model?
 - Input to Layer 1: $784 * 64 + 64 = 50,640$
 - Layer 1 to Layer 2: $64 * 32 + 32 = 2080$
 - Layer 2 to Output: $32 * 10 + 10 = 330$
- 53,050 Weights!**



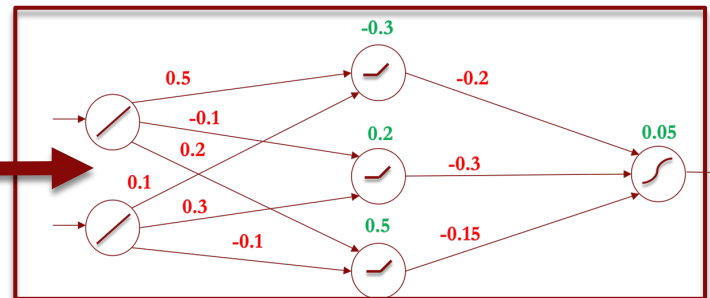
Finding the Best Weights

Data:

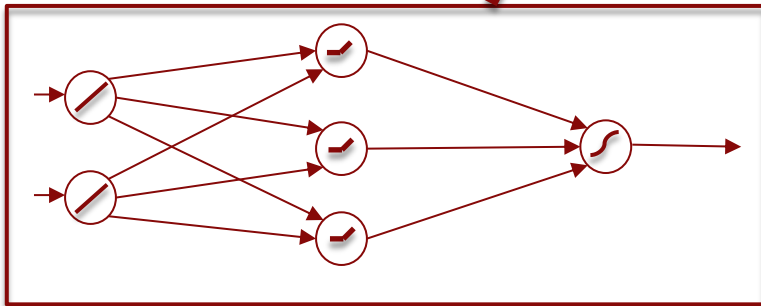
Interview	GPA	Experience
1	0	3.27
2	0	3.37
3	0	3.57
4	0	3.91
5	0	3.20
6	1	3.90
7	1	3.94
8	0	3.66
9	0	3.63
10	0	3.06
11	0	3.21
12	0	3.18
13	0	3.69
14	0	3.38
15	1	3.77
16	0	3.50
...	...	...
25	1	3.31
26	0	3.78
27	0	3.87
28	0	4.00
29	1	3.87

“Compute
the best fit”

Here is the best fit
I could find:



Model Structure:



Define a Loss Function

- What differentiates a good model (i.e., good weights) from a bad model (i.e., bad weights)?
- A **loss function** measures the discrepancy (or error) between your model's predictions and the target values.
 - We want models to minimize the loss on the training data

Regression: Squared Error loss **Classification: Cross-Entropy loss**

$$\min \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\min \frac{1}{n} \sum_{i=1}^n -\log(p_{y_i})$$

Maximum Likelihood

p_i : probability that the i -th prediction is correct

$$\begin{aligned} & \min -\frac{1}{n} (\ln p_1 + \ln p_2 + \dots + \ln p_n) \\ &= \max \frac{1}{n} (\ln p_1 + \ln p_2 + \dots + \ln p_n) \\ &= \max \frac{1}{n} \ln (p_1 p_2 \dots p_n) \end{aligned}$$

Equivalent to

$$\max \ln (p_1 p_2 \dots p_n)$$

Equivalent to

$$\max p_1 p_2 \dots p_n$$

$p_1 p_2 \dots p_n$: likelihood that all predictions are correct

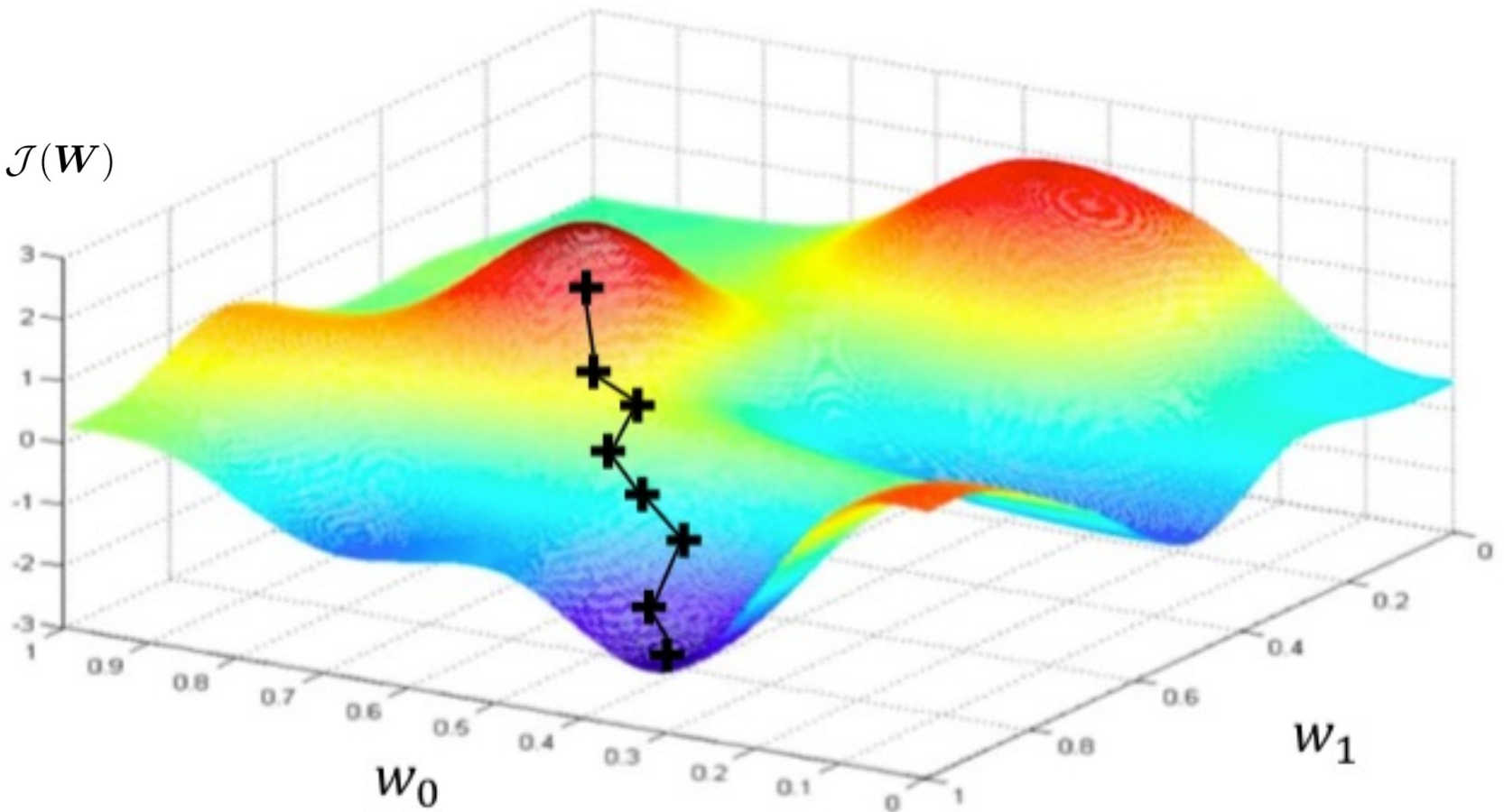
Optimizing Weights

- Start with a model with randomly initialized weights.
 - The model is making random guesses. Output is garbage.
- Make predictions on the training dataset and measure the loss.
- Change the weights slightly to improve the loss on the training data
 - This is done through gradient descent and backpropagation.
- Repeat until satisfied.



Gradient Descent, illustrated

loss $\mathcal{J}(W)$



Setting up a Neural Network

Suppose I have a dataset and a prediction task that I would like to apply a neural network to. What do I need to do?

Step 1. Specify the architecture

1. How many hidden layers?
2. How many neurons in each hidden layer?
3. What activation functions for each neuron?

Step 2. Train the model: find weights and biases

1. Randomly initialize weights and biases
2. Define a loss function depending on your task
3. Use gradient descent to optimize the loss function. Decide when to stop optimizing.

Image and Text Classification

Let's Apply Neural Networks

MNIST Dataset



5



2

Movie Review Sentiment Analysis



"A delightful surprise! The humor was spot-on, and the emotional depth caught me off guard. I'll be rewatching this one for sure."



Positive

"What a waste of time. The acting was wooden, the script was laughable, and the story went nowhere. I wouldn't recommend this to anyone."



Negative

Grayscale Images

We'll talk about color images in the next lecture.



	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]	[,11]	[,12]	[,13]	[,14]	[,15]	[,16]	[,17]	[,18]	[,19]	[,20]	[,21]	[,22]	[,23]	[,24]	[,25]	[,26]	[,27]	[,28]
[1,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[2,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[3,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[4,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[5,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[6,]	0	0	0	0	0	0	0	0	0	0	0	0	3	18	18	18	126	136	175	26	166	255	247	127	0	0	0	0
[7,]	0	0	0	0	0	0	0	0	30	36	94	154	170	253	253	253	253	253	225	172	253	242	195	64	0	0	0	0
[8,]	0	0	0	0	0	0	0	49	238	253	253	253	253	253	253	253	253	251	93	82	82	56	39	0	0	0	0	0
[9,]	0	0	0	0	0	0	0	18	219	253	253	253	253	253	198	182	247	241	0	0	0	0	0	0	0	0	0	0
[10,]	0	0	0	0	0	0	0	0	80	156	107	253	253	205	11	0	43	154	0	0	0	0	0	0	0	0	0	0
[11,]	0	0	0	0	0	0	0	0	0	14	1	154	253	90	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[12,]	0	0	0	0	0	0	0	0	0	0	0	139	253	190	2	0	0	0	0	0	0	0	0	0	0	0	0	0
[13,]	0	0	0	0	0	0	0	0	0	0	0	11	190	253	70	0	0	0	0	0	0	0	0	0	0	0	0	0
[14,]	0	0	0	0	0	0	0	0	0	0	0	0	35	241	225	160	108	1	0	0	0	0	0	0	0	0	0	0
[15,]	0	0	0	0	0	0	0	0	0	0	0	0	0	81	240	253	253	119	25	0	0	0	0	0	0	0	0	0
[16,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	45	186	253	253	150	27	0	0	0	0	0	0	0	0
[17,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	16	93	252	253	187	0	0	0	0	0	0	0	0
[18,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	249	253	249	64	0	0	0	0	0	0	0
[19,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	46	130	183	253	253	207	2	0	0	0	0	0	0	0
[20,]	0	0	0	0	0	0	0	0	0	0	0	0	39	148	229	253	253	253	250	182	0	0	0	0	0	0	0	0
[21,]	0	0	0	0	0	0	0	0	0	0	24	114	221	253	253	253	253	201	78	0	0	0	0	0	0	0	0	0
[22,]	0	0	0	0	0	0	0	23	66	213	253	253	253	253	198	81	2	0	0	0	0	0	0	0	0	0	0	0
[23,]	0	0	0	0	0	18	171	219	253	253	253	253	195	80	9	0	0	0	0	0	0	0	0	0	0	0	0	0
[24,]	0	0	0	0	55	172	226	253	253	253	253	244	133	11	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[25,]	0	0	0	0	136	253	253	253	212	135	132	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[26,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[27,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[28,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0



Pixels 1 to 784: [0, 0, 0,, 3,, 240, ..., 0,0,0]

Results: CART and Random Forest

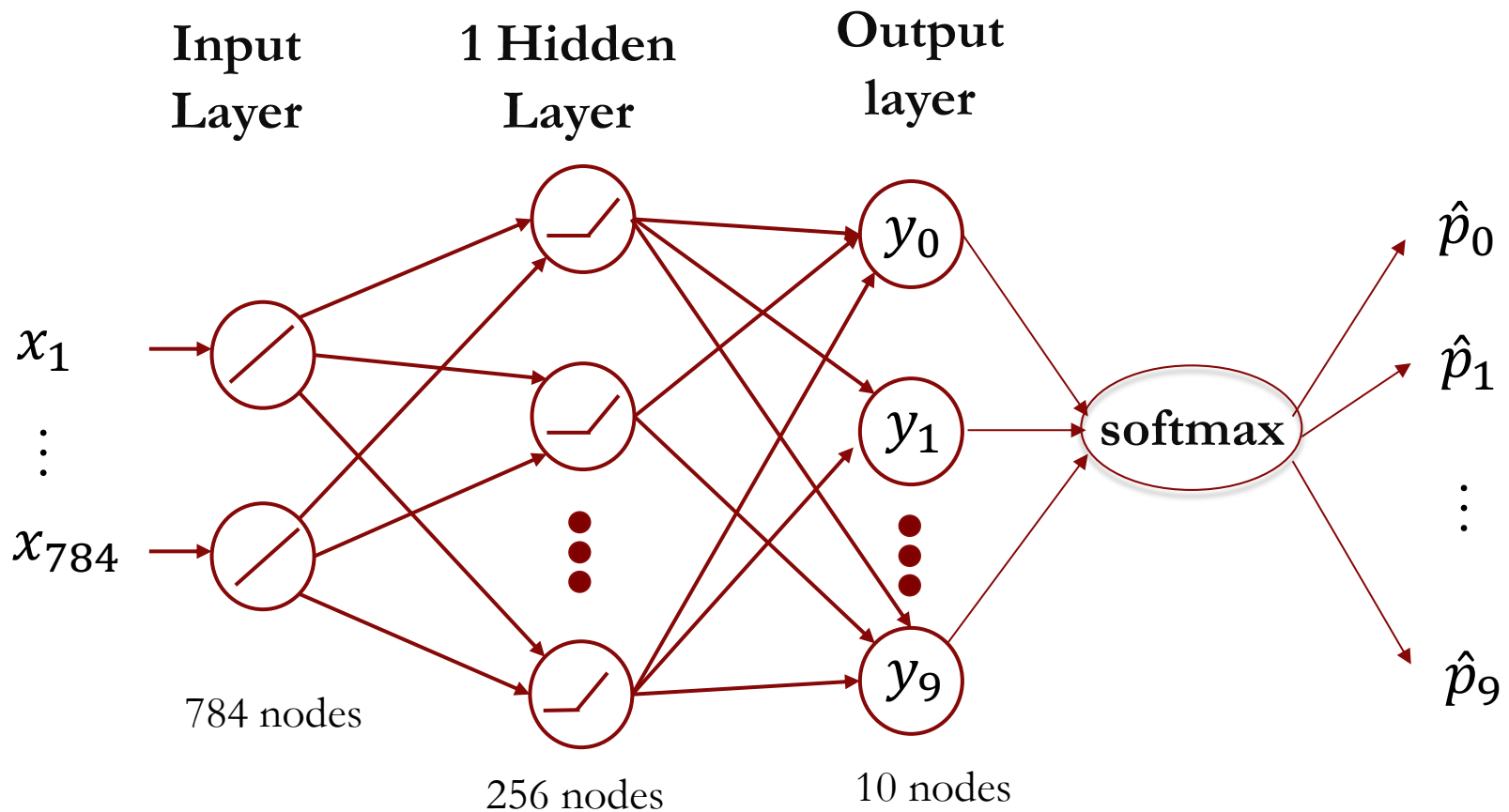
With each image converted to a 784-element vector of numbers and a categorical dependent variable that takes on 10 values, we can easily construct CART and Random Forest models

Prediction Model	Error Rate (%)	Accuracy (%)
CART	12.1	87.9
Random Forest	3.1	96.9

Both CART and Random Forest do respectably well. **Why do you think they work well here?**

Let's see how a Neural Network does . . .

A Neural Network for Digit Recognition



203,264 **weights** and 266 **bias** coefficients to determine

Results: one-hidden-layer neural net

Prediction Model	Error Rate (%)	Accuracy (%)
Logistic Regression	—	—
CART	12.1	87.9
Random Forest	3.1	96.9
Neural Network (with one hidden layer)	2.1	97.9

- A simple **Neural Network** with one hidden layer decreases Random Forest's error rate from 3.1% to 2.1%
- Let us add a second hidden layer to try to decrease the error rate even further . . .

Confusion Matrices

CART

CART Confusion Matrix on MNIST

True Label \ Predicted Label	0	1	2	3	4	5	6	7	8	9
0	914	1	7	4	6	9	16	5	8	10
1	0	1084	9	8	2	9	5	3	14	1
2	13	11	887	29	15	6	9	24	30	8
3	7	8	34	861	8	40	3	6	17	26
4	8	4	11	6	858	5	18	10	20	42
5	15	8	5	39	6	740	25	6	32	16
6	21	5	11	9	23	15	846	3	20	5
7	2	7	21	24	12	5	3	925	9	20
8	8	9	33	34	21	32	14	12	785	26
9	14	5	10	22	45	10	6	20	23	854

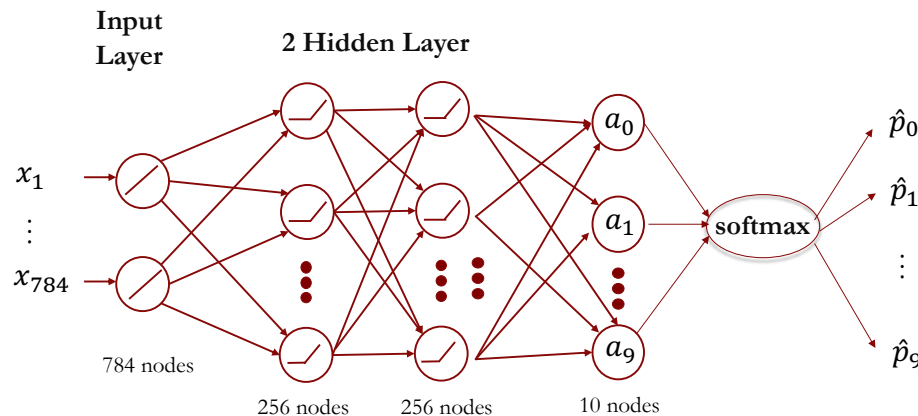
Neural Network

Confusion Matrix

True Label \ Predicted Label	0	1	2	3	4	5	6	7	8	9
0	973	1	1	2	0	0	1	0	2	0
1	0	1121	3	3	0	0	2	1	5	0
2	4	2	1007	6	1	0	2	3	6	1
3	2	0	1	991	0	4	0	4	3	5
4	3	0	5	0	948	0	3	1	0	22
5	2	1	0	5	1	878	2	0	2	1
6	8	2	2	1	2	5	938	0	0	0
7	2	3	9	4	0	0	0	1002	2	6
8	6	0	5	9	2	3	1	4	940	4
9	3	2	0	5	2	4	1	3	0	989

Neural Network with two hidden layers

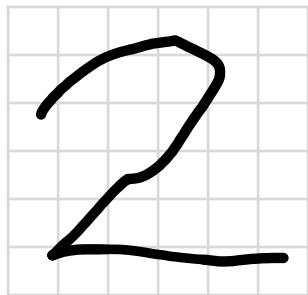
Prediction Model	Error Rate (%)	Accuracy (%)
CART	12.1	87.9
Random Forest	3.1	96.9
Neural Network with one hidden layer	2.1	97.9
Neural Network with two hidden layers	2.2	97.8



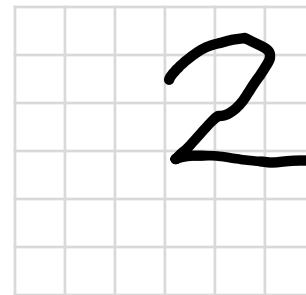
Adding a second hidden layer did not help at all

How Can We Do Better?

- Downsides of Feed-Forward NN
 - 2 Layers => ~268,000 weights, difficult to optimize
 - Pixels that are adjacent to each other in the image are not treated as such.
 - Sensitive to zooming in/out, shifting the digit around.



Vs.



- **New idea:** Convolutional Neural Networks
 - Neural networks specifically designed for image processing

IMDB Review Sentiment Analysis

- 50,000 reviews in total (25,000 positive, 25,000 negative)
- Each review is a free-form English text string.
- The target of interest is whether the review expresses a **positive** or **negative** sentiment towards the movie.



"A delightful surprise! The humor was spot-on, and the emotional depth caught me off guard. I'll be rewatching this one for sure."

→ Positive

Cleaning + Bag of Words Vectorization

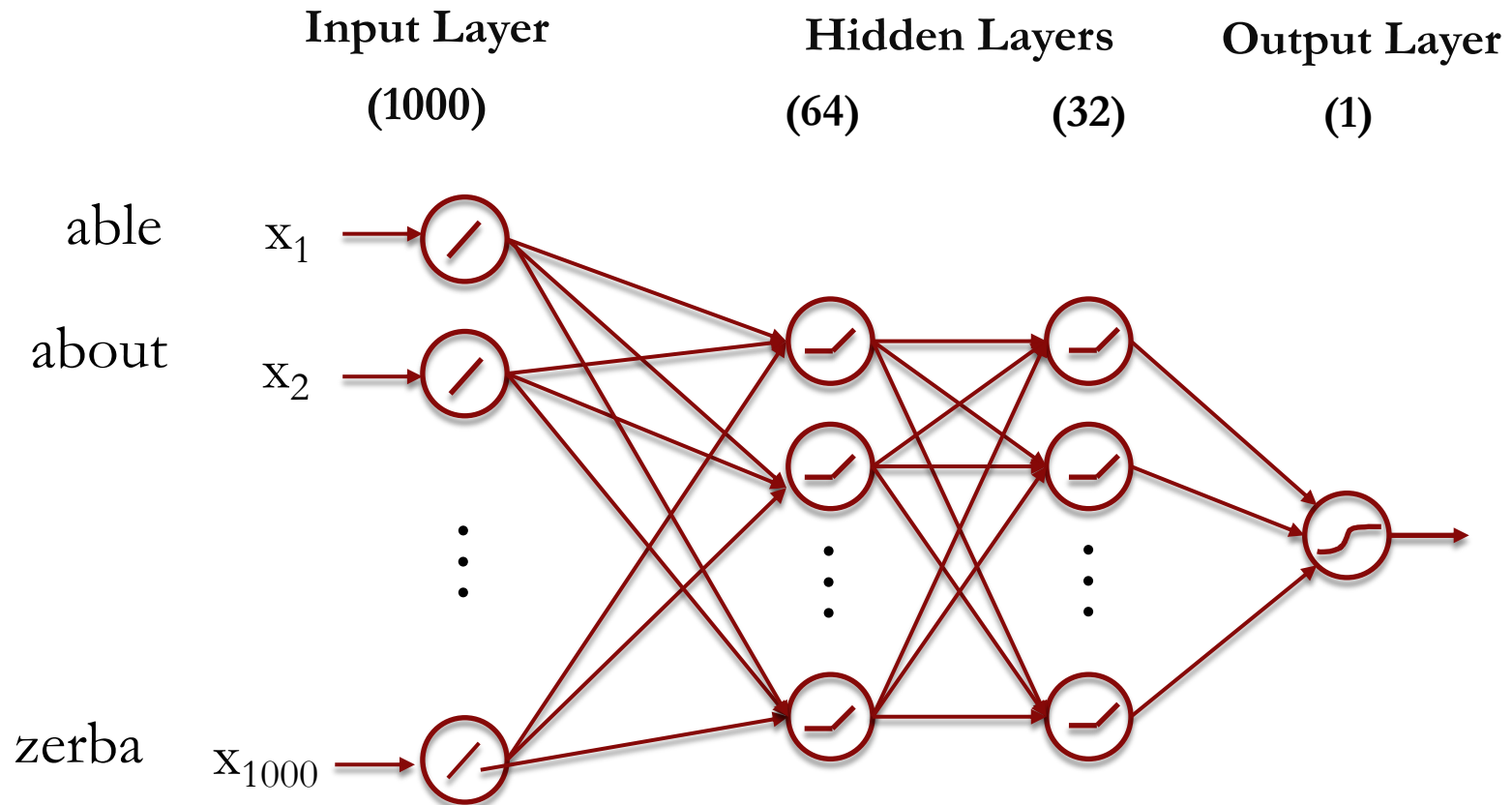
able	about	above	absolutely	across	act	acted	acting	...	year	years	yes	yet	york	you	young	your	yourself	zombie	is_positive
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	2	0	0	4	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0
0	1	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	1	0	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	2	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	2	1	0	0	1	2	0	1	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

Each row is a movie review (5000 in total)

Each column is a word (only the top 1000 words)

The values are how many times the word appears in the review.

Our Neural Network



Results

Prediction Model	Error Rate (%)	Accuracy (%)
Logistic Regression / CART	28.4%	71.6%
Random Forest	18.2%	81.8%
Neural Network with one hidden layer	16.6%	83.4%
Neural Network with two hidden layers	14.1%	85.9%

- Neural network with 2 hidden layers does the best here.
- Can we do even better?
- **New idea:** word and sentence embeddings.
(Using numbers to capture the “meaning” of words)

Trade-Offs

Aspect	Neural Networks	Traditional Models (e.g., Logistic Regression)
Performance on Unstructured Data	Excels at learning representations, extracting useful features	Struggles to detect complex patterns
Interpretability	Often considered "black box" models	Typically easier to interpret, coefficients indicate feature importance
Data Requirements	Generally requires large datasets	Can work well with smaller datasets
Computational Resources	Typically require more computational power including GPU's	Less computationally intensive
Training Time	Training can take days/weeks and much power	Generally faster to train

Convolutional Neural Networks

Convolution Operation

Let us do a “sum product” for two matrices:

1	3	2
0	3	-1
-1	-1	1

 *

2	2	1
7	1	6
4	2	5

$$= 1*2 + 3*2 + 2*1 + 0*7 + 3*1 - 1*6 - 1*4 - 1*2 + 1*5 = 6$$

This is called a **convolution operation**.

Convolution Operation (one more time)

Let us do another “sum product” for two matrices:

4	-1	1
2	1	0
1	-2	3

 *

1	3	2
0	3	-1
-1	-1	2

$$= 4*1 + (-1)*3 + 1*2 + 2*0 + 1*3 + 0*(-1) + 1*(-1) + (-2)*(-1) + 3*2 = \mathbf{13}$$

This is called a **convolution operation**.

Grayscale Images as a List of Numbers

We'll talk about color images later.

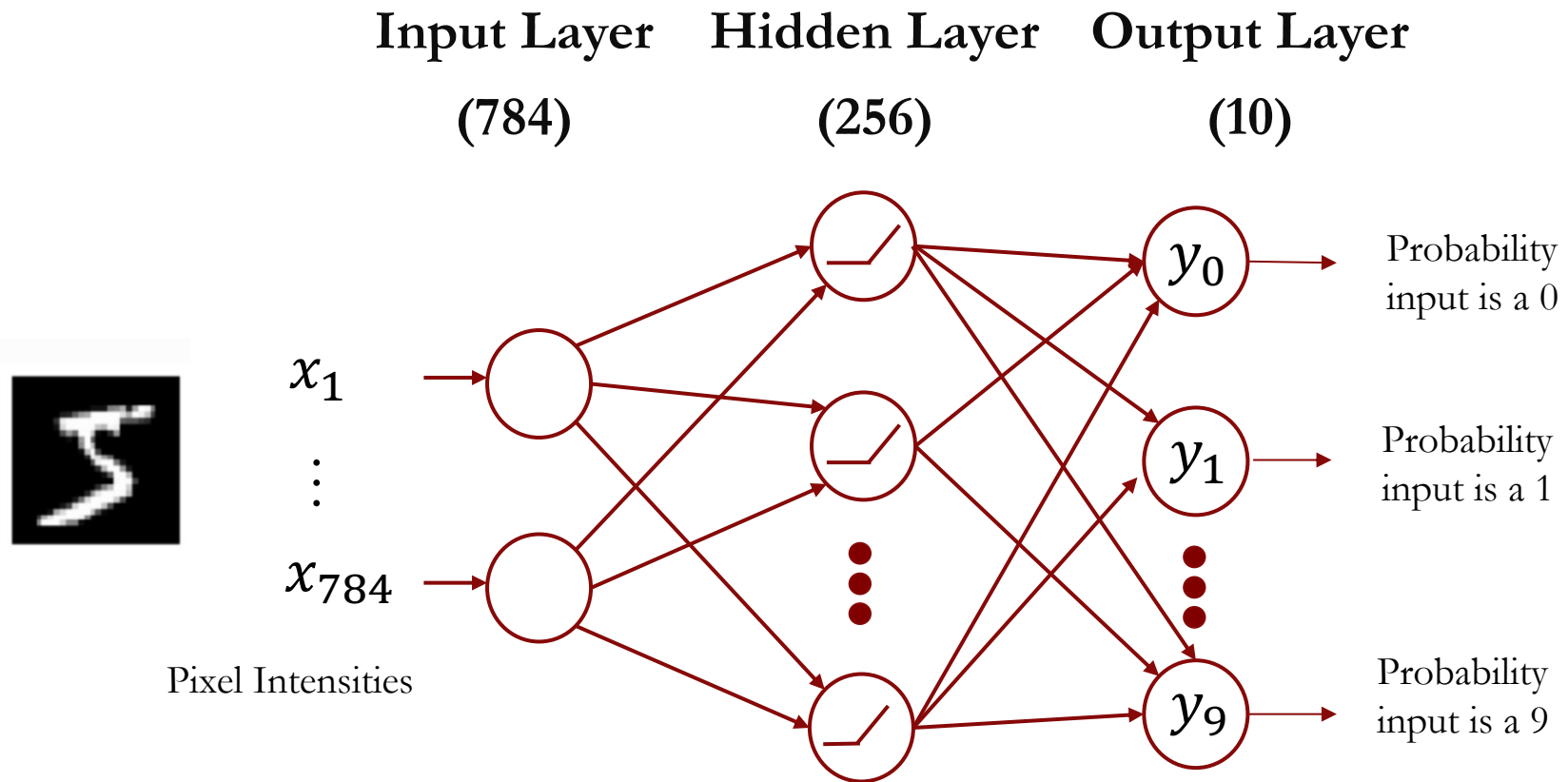


	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]	[,11]	[,12]	[,13]	[,14]	[,15]	[,16]	[,17]	[,18]	[,19]	[,20]	[,21]	[,22]	[,23]	[,24]	[,25]	[,26]	[,27]	[,28]
[1,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[2,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[3,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[4,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[5,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[6,]	0	0	0	0	0	0	0	0	0	0	0	0	3	18	18	18	126	136	175	26	166	255	247	127	0	0	0	0
[7,]	0	0	0	0	0	0	0	0	30	36	94	154	170	253	253	253	253	253	225	172	253	242	195	64	0	0	0	0
[8,]	0	0	0	0	0	0	0	49	238	253	253	253	253	253	253	253	251	93	82	82	56	39	0	0	0	0	0	0
[9,]	0	0	0	0	0	0	0	18	219	253	253	253	253	253	198	182	247	241	0	0	0	0	0	0	0	0	0	0
[10,]	0	0	0	0	0	0	0	0	80	156	107	253	253	205	11	0	43	154	0	0	0	0	0	0	0	0	0	0
[11,]	0	0	0	0	0	0	0	0	0	14	1	154	253	90	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[12,]	0	0	0	0	0	0	0	0	0	0	0	139	253	190	2	0	0	0	0	0	0	0	0	0	0	0	0	0
[13,]	0	0	0	0	0	0	0	0	0	0	0	11	190	253	70	0	0	0	0	0	0	0	0	0	0	0	0	0
[14,]	0	0	0	0	0	0	0	0	0	0	0	0	35	241	225	160	108	1	0	0	0	0	0	0	0	0	0	0
[15,]	0	0	0	0	0	0	0	0	0	0	0	0	0	81	240	253	253	119	25	0	0	0	0	0	0	0	0	0
[16,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	45	186	253	253	150	27	0	0	0	0	0	0	0	0
[17,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	16	93	252	253	187	0	0	0	0	0	0	0	0
[18,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	249	253	249	64	0	0	0	0	0	0	0
[19,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	46	130	183	253	253	207	2	0	0	0	0	0	0	0
[20,]	0	0	0	0	0	0	0	0	0	0	0	0	39	148	229	253	253	253	250	182	0	0	0	0	0	0	0	0
[21,]	0	0	0	0	0	0	0	0	0	24	114	221	253	253	253	253	253	201	78	0	0	0	0	0	0	0	0	0
[22,]	0	0	0	0	0	0	0	23	66	213	253	253	253	253	198	81	2	0	0	0	0	0	0	0	0	0	0	0
[23,]	0	0	0	0	0	0	18	171	219	253	253	253	253	195	80	9	0	0	0	0	0	0	0	0	0	0	0	0
[24,]	0	0	0	0	55	172	226	253	253	253	253	244	133	11	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[25,]	0	0	0	0	136	253	253	253	212	135	132	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[26,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[27,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[28,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0



Pixels 1 to 784: [0, 0, 0,, 3,, 240, ..., 0,0,0]

Feed Forward NN for Digit Recognition



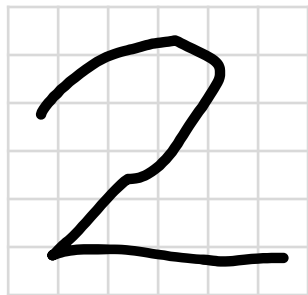
Performance Was Solid

Prediction Model	Error Rate (%)	Accuracy (%)
Logistic Regression	—	—
CART	12.1	87.9
Random Forest	3.1	96.9
Neural Network (with one hidden layer)	2.1	97.9

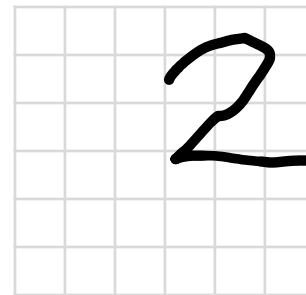
- Solid performance, but how can we do better?

How Can We Do Better?

- Downsides of Feed-Forward NN
 - 1 Hidden Layer => ~200,000 weights, difficult to optimize
 - Spatial information is lost: pixels have some proximity to each other that the neural network does not use.
 - Sensitive to zooming in/out, shifting the digit around.



Vs.



- Idea: Convolutional Neural Networks
 - Neural networks specifically designed for image processing
 - Think about this as “feature engineering”

Spatial Information is Lost



	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]	[,11]	[,12]	[,13]	[,14]	[,15]	[,16]	[,17]	[,18]	[,19]	[,20]	[,21]	[,22]	[,23]	[,24]	[,25]	[,26]	[,27]	[,28]
[1,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[2,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[3,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[4,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[5,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[6,]	0	0	0	0	0	0	0	0	0	0	0	0	3	18	18	18	126	136	175	26	166	255	247	127	0	0	0	
[7,]	0	0	0	0	0	0	0	0	30	36	94	154	170	253	253	253	253	253	225	172	253	242	195	64	0	0	0	
[8,]	0	0	0	0	0	0	0	49	238	253	253	253	253	253	253	253	251	93	82	82	56	39	0	0	0	0	0	
[9,]	0	0	0	0	0	0	0	18	219	253	253	253	253	198	182	247	241	0	0	0	0	0	0	0	0	0	0	
[10,]	0	0	0	0	0	0	0	0	80	156	107	253	253	205	11	0	43	154	0	0	0	0	0	0	0	0	0	
[11,]	0	0	0	0	0	0	0	0	0	14	1	154	253	90	0	0	0	0	0	0	0	0	0	0	0	0	0	
[12,]	0	0	0	0	0	0	0	0	0	0	0	139	253	190	2	0	0	0	0	0	0	0	0	0	0	0	0	
[13,]	0	0	0	0	0	0	0	0	0	0	0	11	190	253	70	0	0	0	0	0	0	0	0	0	0	0	0	
[14,]	0	0	0	0	0	0	0	0	0	0	0	35	241	225	160	108	1	0	0	0	0	0	0	0	0	0	0	
[15,]	0	0	0	0	0	0	0	0	0	0	0	0	81	240	253	253	119	25	0	0	0	0	0	0	0	0	0	
[16,]	0	0	0	0	0	0	0	0	0	0	0	0	0	45	186	253	253	150	27	0	0	0	0	0	0	0	0	
[17,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	16	93	252	253	187	0	0	0	0	0	0	0	0	
[18,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	249	253	249	64	0	0	0	0	0	0	0	
[19,]	0	0	0	0	0	0	0	0	0	0	0	0	0	46	130	183	253	253	207	2	0	0	0	0	0	0	0	
[20,]	0	0	0	0	0	0	0	0	0	0	0	0	39	148	229	253	253	253	250	182	0	0	0	0	0	0	0	
[21,]	0	0	0	0	0	0	0	0	0	0	24	114	221	253	253	253	253	201	78	0	0	0	0	0	0	0	0	
[22,]	0	0	0	0	0	0	0	0	23	66	213	253	253	253	198	81	2	0	0	0	0	0	0	0	0	0	0	
[23,]	0	0	0	0	0	0	18	171	219	253	253	253	253	195	80	9	0	0	0	0	0	0	0	0	0	0	0	
[24,]	0	0	0	0	55	172	226	253	253	253	253	244	133	11	0	0	0	0	0	0	0	0	0	0	0	0	0	
[25,]	0	0	0	0	136	253	253	253	212	135	132	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[26,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[27,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
[28,]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

These pixels are close together

These pixels are
close together



$[0, 0, 0, \dots, 3, \dots, 240, \dots, 0, 0, 0]$



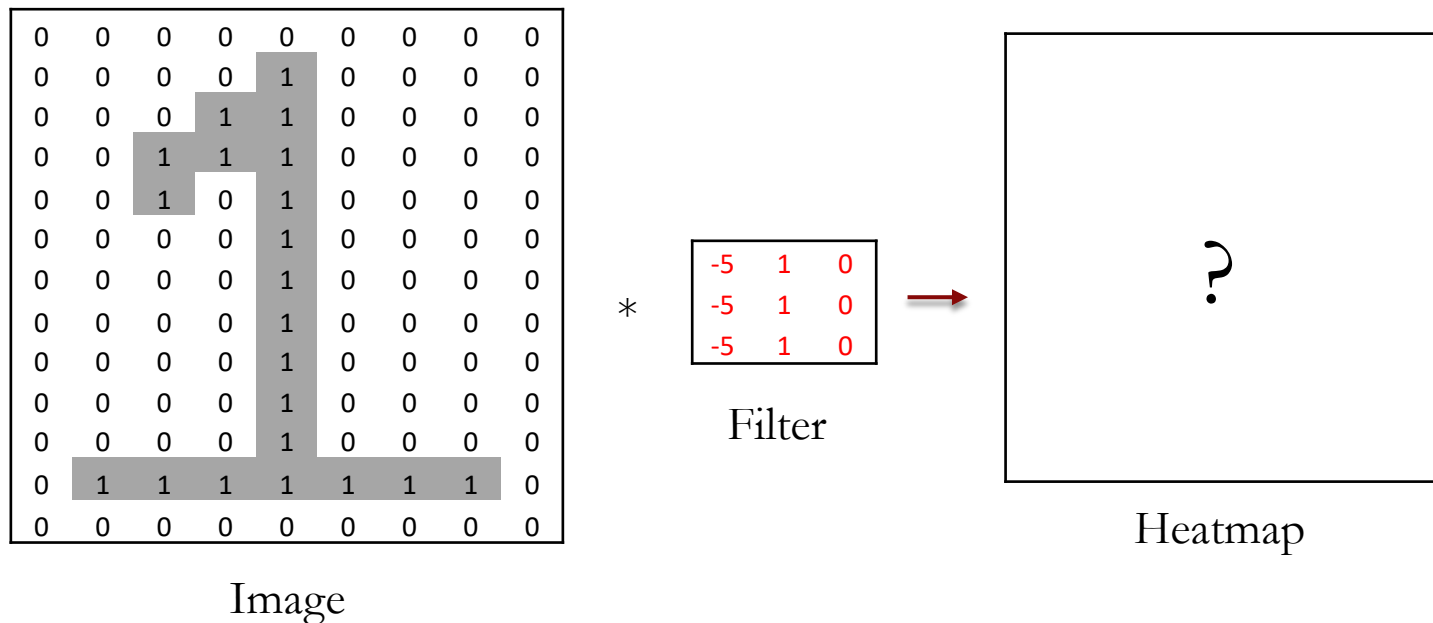
No longer all close together

Convolutional Neural Networks

- **Filters, Convolution and Pooling**
 - **Filters:** A small pattern that one hopes to “find” in the image
 - **Convolution:** An operation that finds all instances of the filter in the image.
 - **Pooling:** Condense information
- **More Complex CNN Architectures**
 - Multiple convolution layers and pooling layers that progressively extract higher level features from the image.

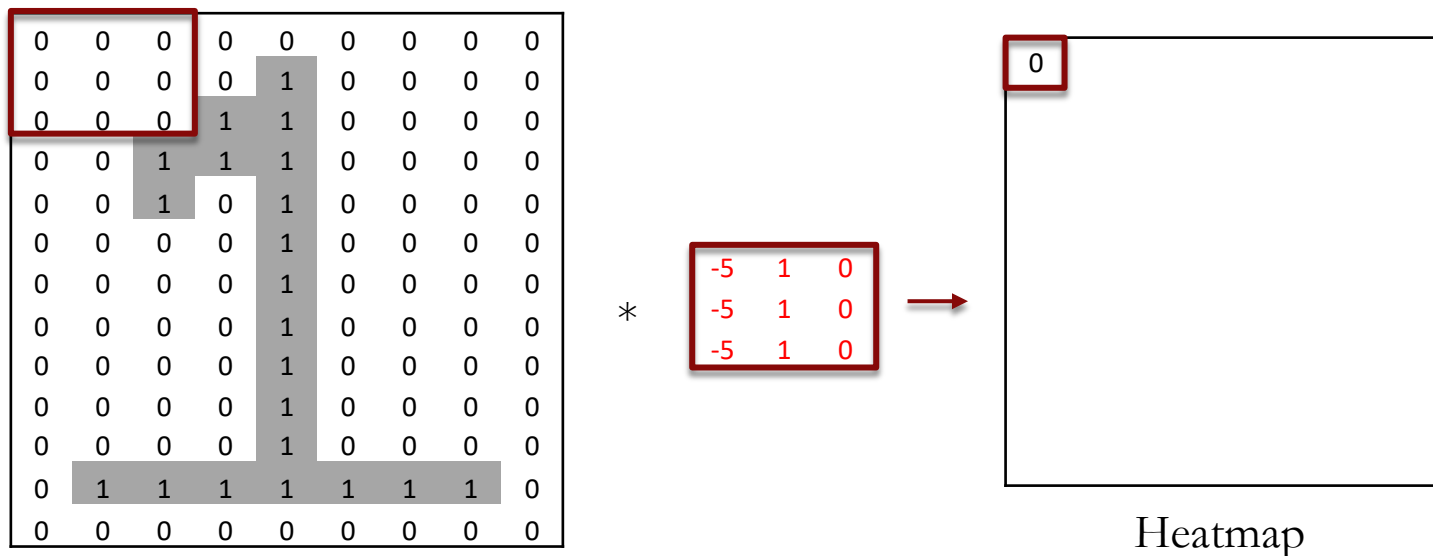
Filters and the Convolution Operation

- The **convolution operator** takes as input an image and a small **filter** (typically a small square matrix of numbers).
- It generates another image which intuitively captures where the small filters *appears* in the image. A sort of heatmap.



Performing a Convolution

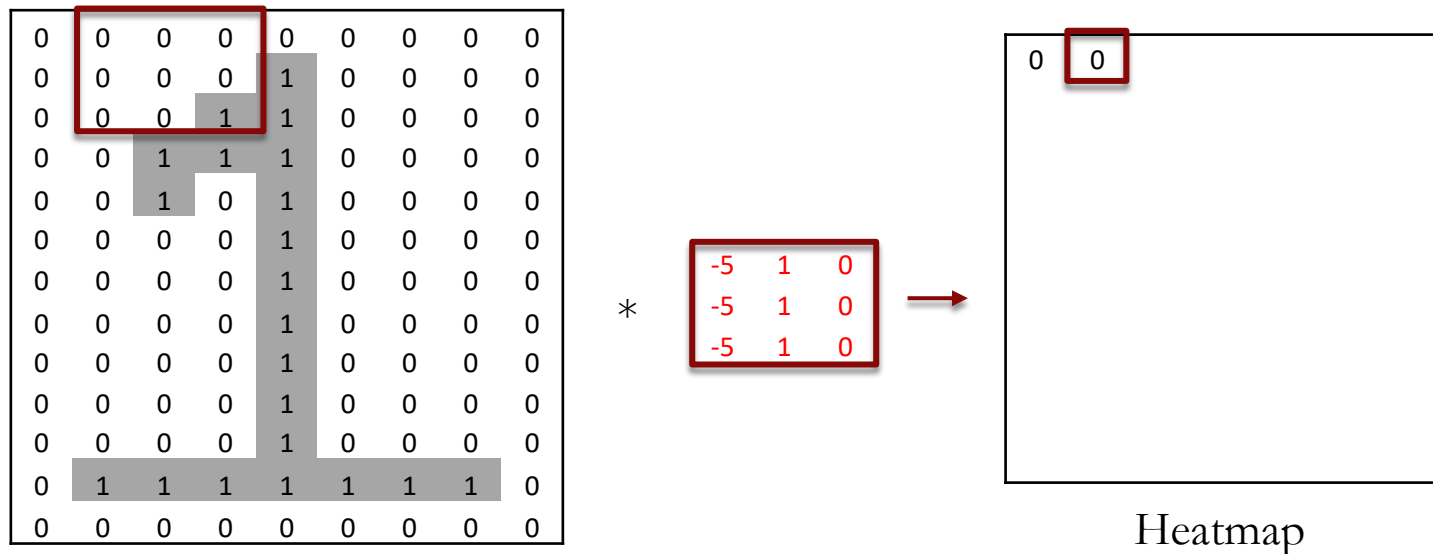
- Slide the filter across the image, taking **sum-products**
 - Apply a ReLU to get rid of any negative values.*
 - Continue row by row, generating another image.



*In deep learning, we always add the ReLU to convolution, but by default convolution does not have ReLU.

Performing a Convolution

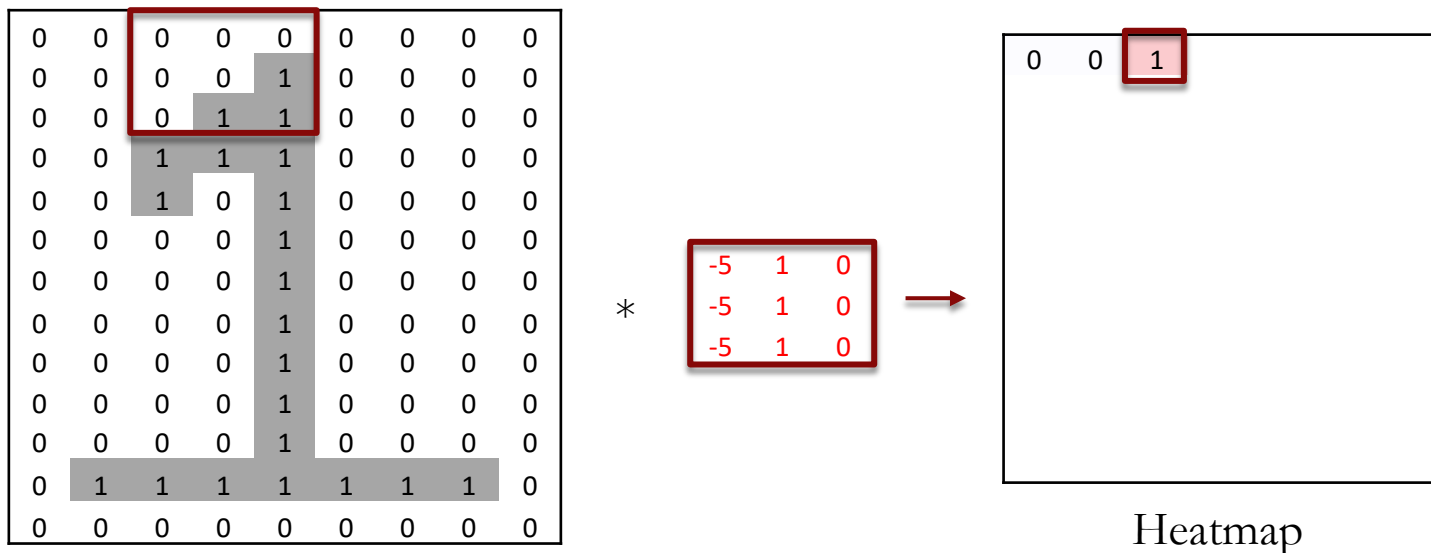
- Slide the filter across the image, taking **sum-products**
 - Apply a ReLU to get rid of any negative values.*
 - Continue row by row, generating another image.



*In deep learning, we always add the ReLU to convolution, but by default convolution does not have ReLU.

Performing a Convolution

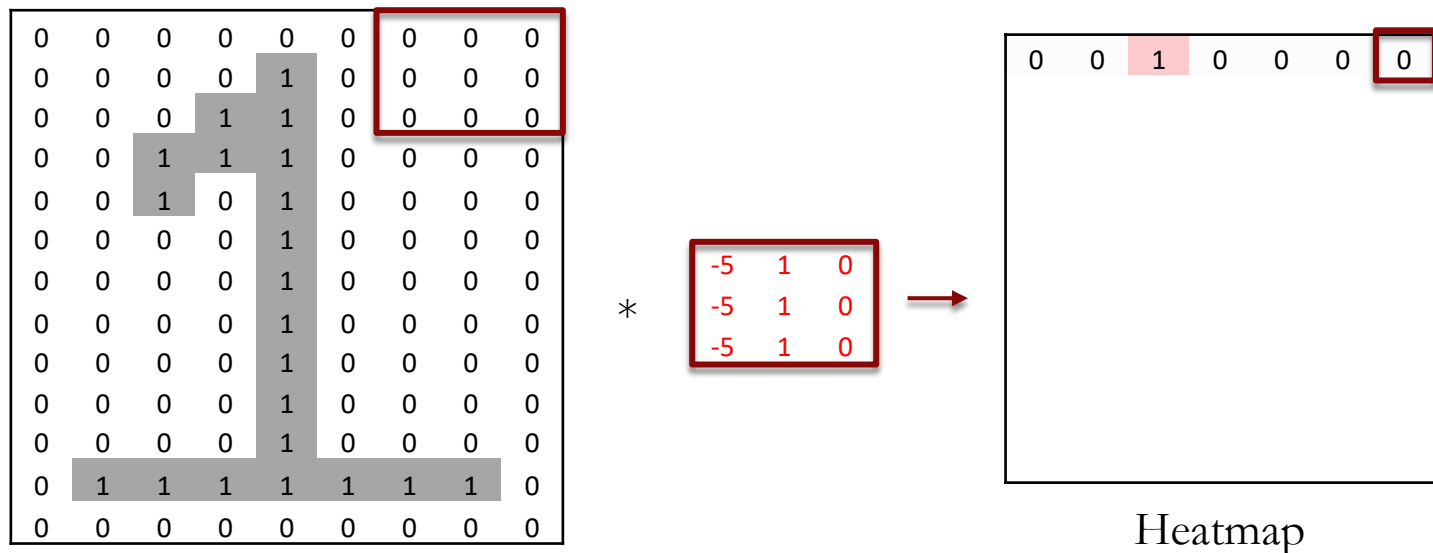
- Slide the filter across the image, taking **sum-products**
 - Apply a ReLU to get rid of any negative values.*
 - Continue row by row, generating another image.



*In deep learning, we always add the ReLU to convolution, but by default convolution does not have ReLU.

Performing a Convolution

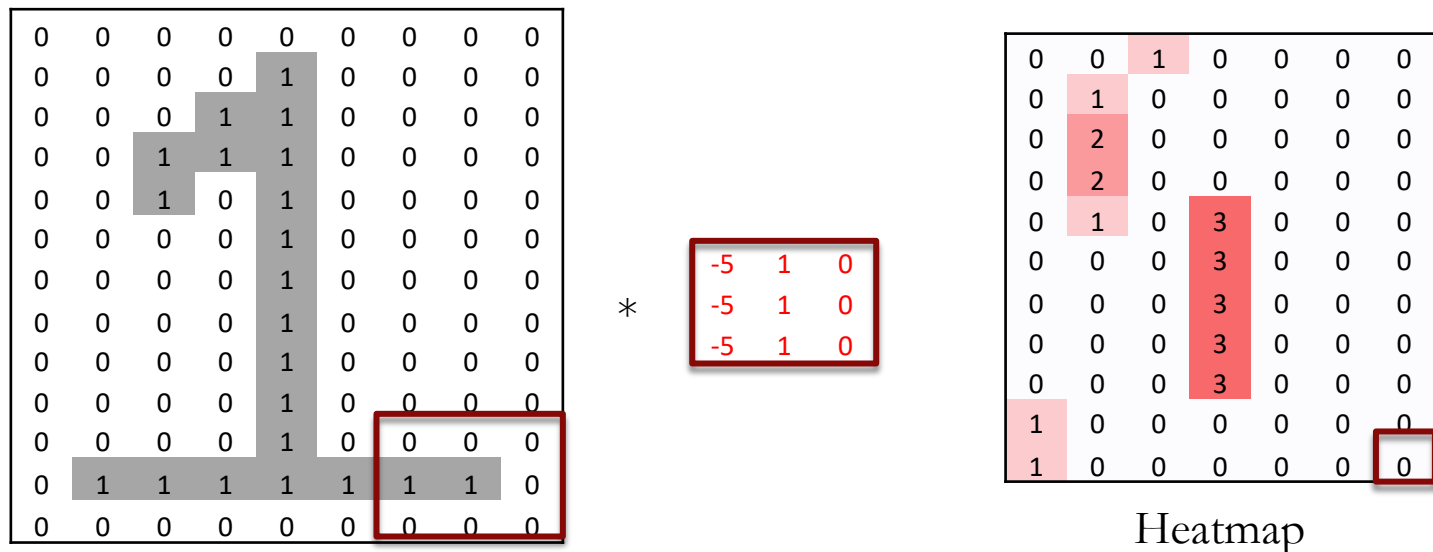
- Slide the filter across the image, taking **sum-products**
 - Apply a ReLU to get rid of any negative values.*
 - Continue row by row, generating another image.



*In deep learning, we always add the ReLU to convolution, but by default convolution does not have ReLU.

Performing a Convolution

- Slide the filter across the image, taking **sum-products**
 - Apply a ReLU to get rid of any negative values.*
 - Continue row by row, generating another image.



*In deep learning, we always add the ReLU to convolution, but by default convolution does not have ReLU.

Different Filters Detect Different Features in the Image

0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	1	1	0	0	0	0
0	0	1	1	1	0	0	0	0
0	0	1	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0

*

-5	1	0
-5	1	0
-5	1	0



0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	2	0	0	0	0	0	0
0	2	0	0	0	0	0	0
0	1	0	3	0	0	0	0
0	0	0	3	0	0	0	0
0	0	0	3	0	0	0	0
0	0	0	3	0	0	0	0
0	0	0	3	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0

Vertical Edges
are Detected

0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	1	1	0	0	0	0
0	0	1	1	1	0	0	0	0
0	0	1	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0

*

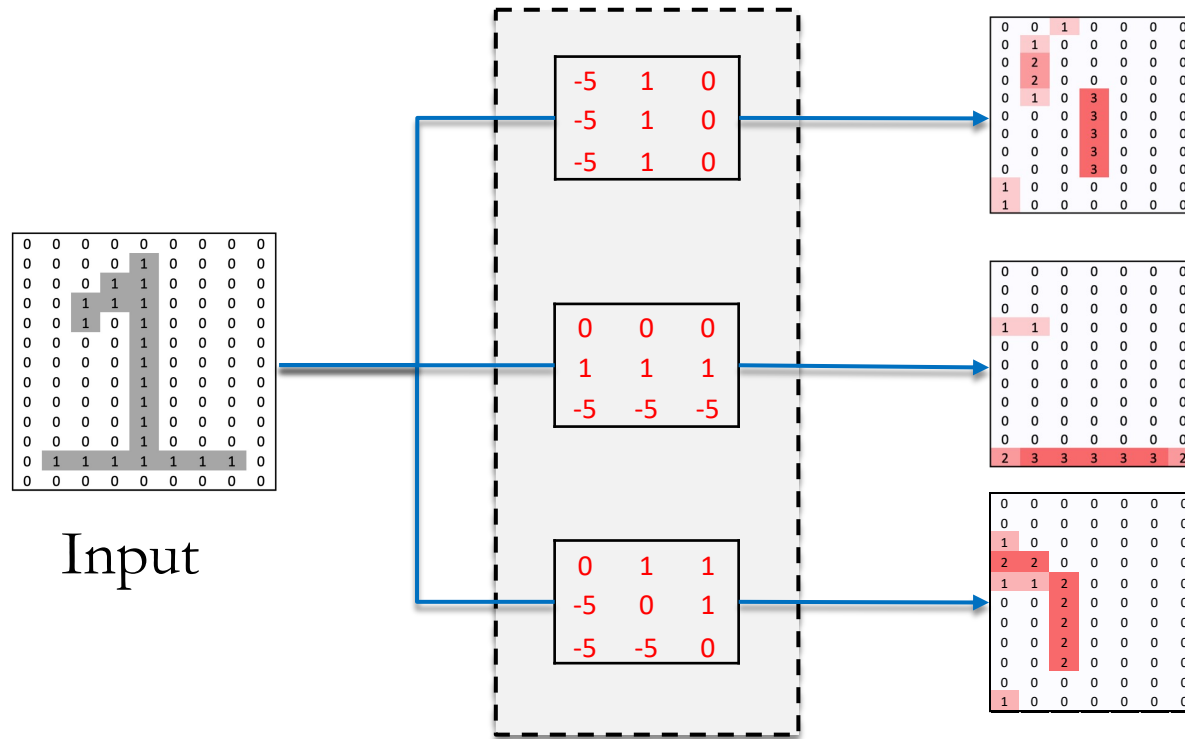
0	0	0
1	1	1
-5	-5	-5



0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
2	3	3	3	3	3	3	2

Horizontal Edges
are Detected

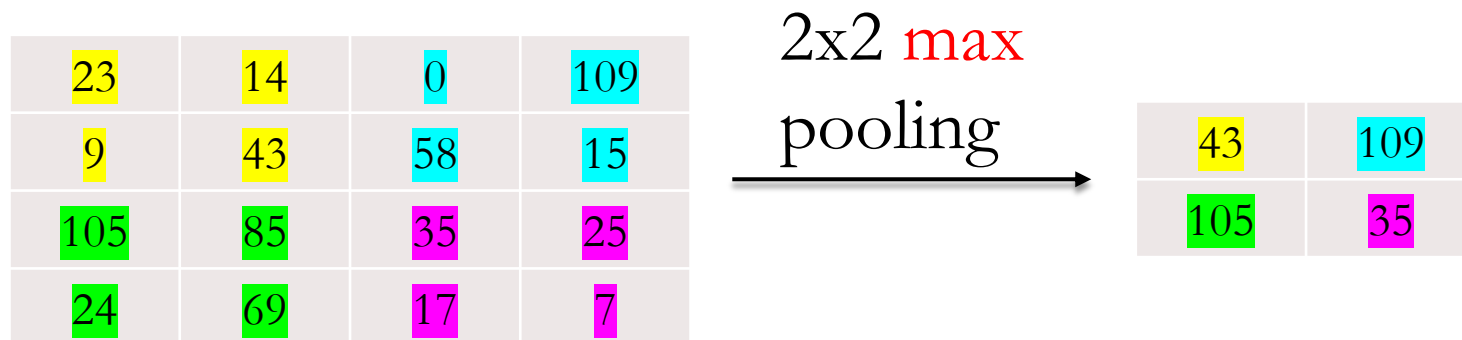
Convolution Layer



A convolution layer consists of **one or more filters**. It takes as input an image and produces as output the convolution with each filter.

Pooling

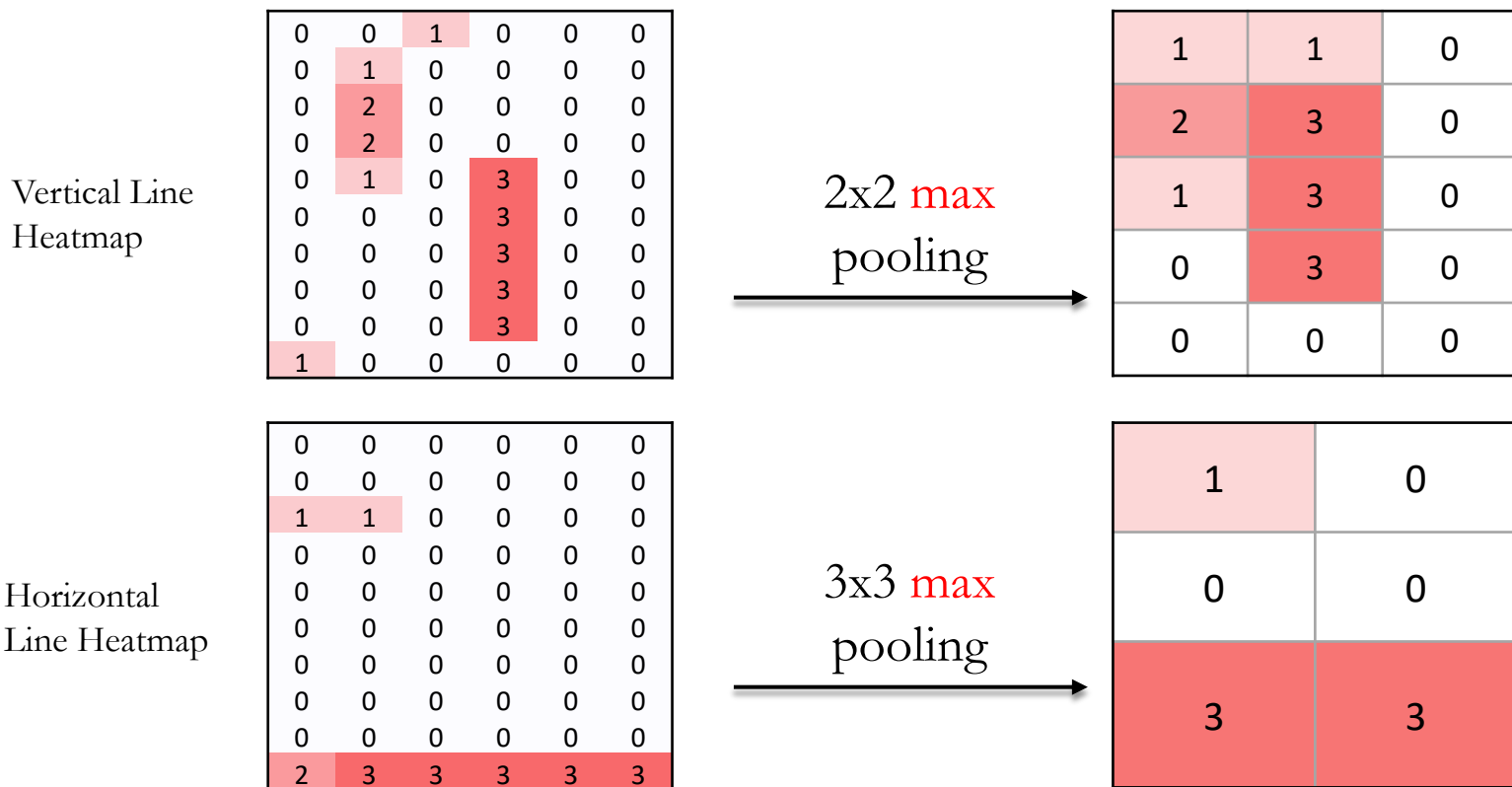
- **Pooling** (also called down-sampling or subsampling layers) reduce the size of each image/heatmap coming out of a convolutional layer



- In 2x2 *max-pooling*, we take the **maximum** value from each 2x2 box. Average pooling is also possible.

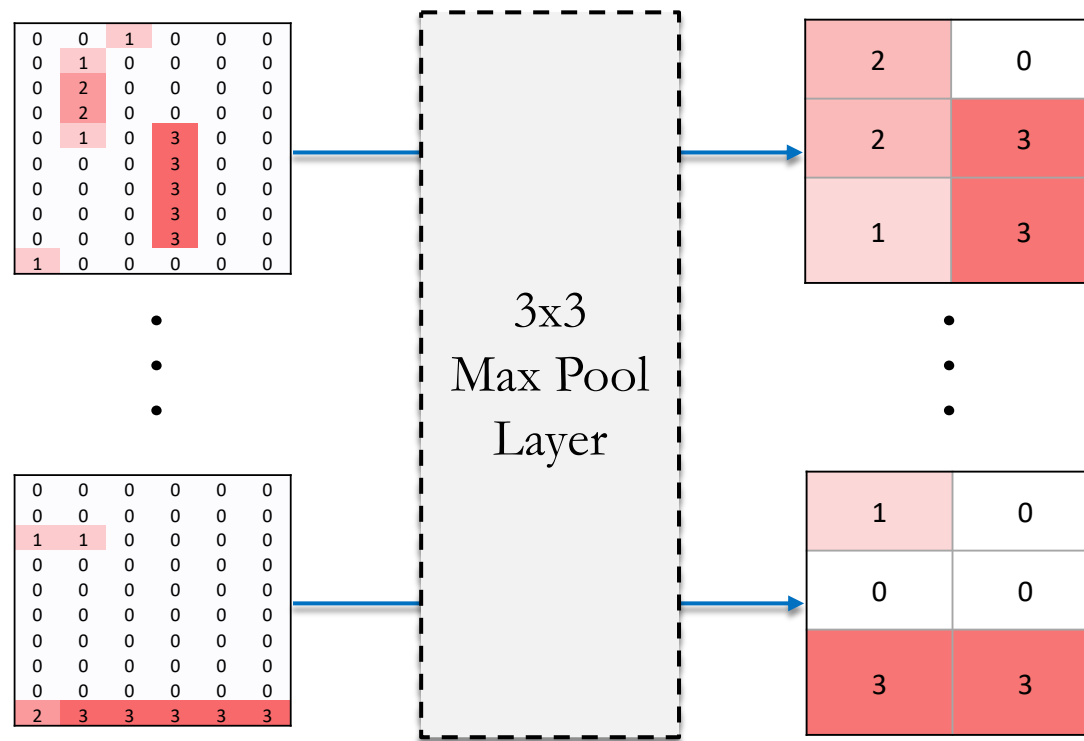
Max Pooling Example

- Condense the information in the new image.



Max Pooling Layer

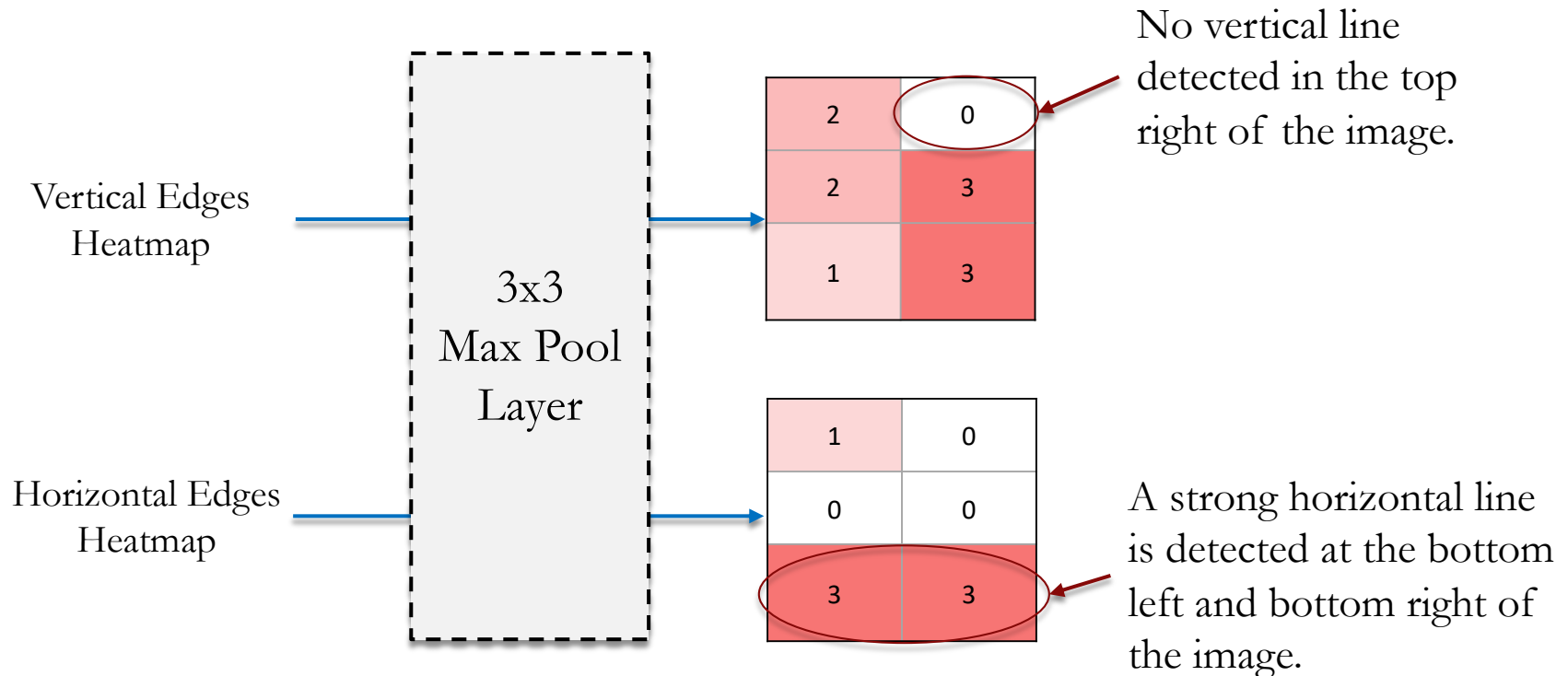
- A **Max Pooling Layer** take as input a set of images and applies max pooling to each one.

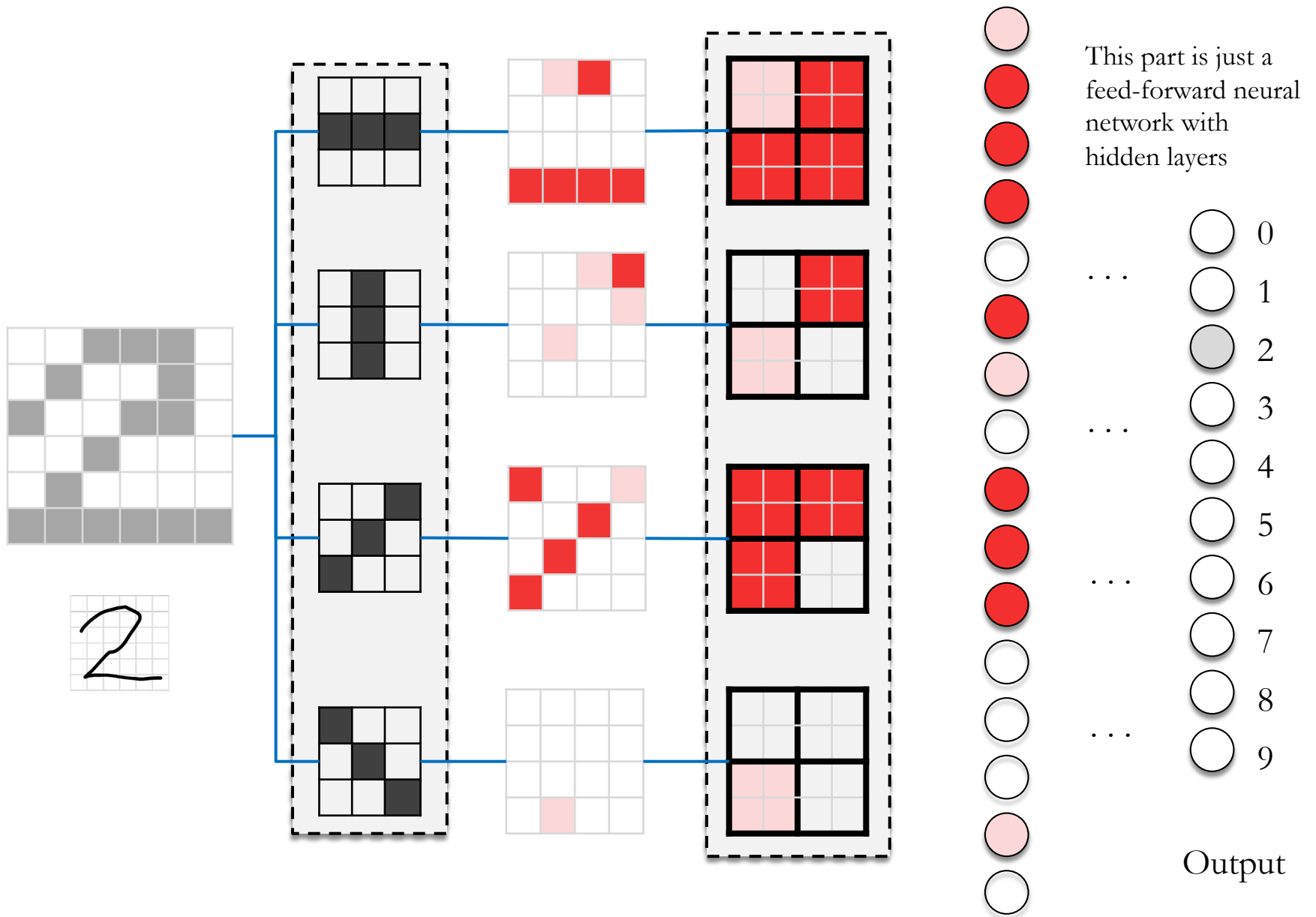


Pooling Extracts Features

0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	1	1	0	0	0	0
0	0	1	1	1	0	0	0	0
0	0	1	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0
0	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0

- We can now treat these numbers as features in a feed-forward neural network.





Input Image

Convolutional Layer

Heatmap

Max Pooling Layer

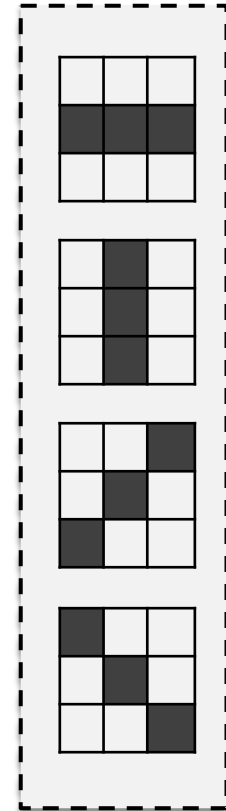
Flatten

Previous Example

- The previous slide showed an example with 4 filters that detect horizontal, vertical, diagonal and anti-diagonal lines respectively.
- The resulting heatmaps show the prevalence of these 4 types of lines in each part of the original image.
- The pooling layer condenses this information down to 4 values (top-right, top-left, bottom-right, bottom-left) for each of the 4 heatmaps.
- These 16 values are flattened into a list of 16 features. Those features can be passed through a feed-forward neural network to get probabilities.

Optimizing the Filters

- How do we find the optimal filters?
- Treat the numbers in each filter like weights and learn them from data, just like we learned the weights of a feed-forward neural network.
- Optimization will simultaneously find:
 - The best filters
 - The best weights/bias for the feed-forward neural network that makes the digit 0-9 prediction.



Analogy from Collaborative Filtering

- We did not specify the “architypes.” Instead, we let the optimization process to determine the architypes and
 - $S_{a,m}$: rating of movie m by archetype a
 - $w_{u,a}$: weight of archetype a in the composition of user u
- Let us compute these values to minimize the sum of squared residuals, namely the difference between observed and predicted user-movie ratings for all user-movie pairs in the observed ratings

$$\underset{w_{u,a}, S_{a,m}}{\text{minimize}} \sum_{\substack{\text{all observed} \\ \text{user-movie} \\ \text{pairs } (u,m)}} (\text{OBSR}_{u,m} - (w_{u,1}S_{1,m} + w_{u,2}S_{2,m} + \cdots + w_{u,9}S_{9,m}))^2$$

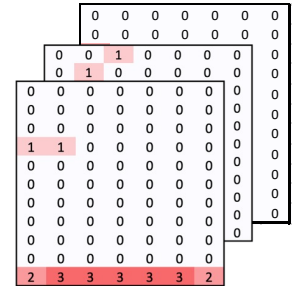
CNN Performance

Prediction Model	Error Rate (%)	Accuracy (%)
CART	12.1	87.9
Random Forest	3.1	96.9
Neural Network with one hidden dense layer	2.1	97.9
Neural Network with two hidden dense layers	2.2	97.8
Neural Network with one Convolutional layer of 32 filters, plus one dense hidden layer	1.4	98.6

- CNN's further reduced the error rate (2.2% to 1.4%)
- Can we do even better?

Why Stop at 1 Convolution?

- Let's treat the heatmap as **another image!**
- This will allow us to detect more complex features.
 - First convolution layer: horizontal, vertical, diagonal lines
 - Second convolution layer: loops, curves
 - Third convolution layer: complex objects like faces (example below)



Chaining Convolution Layers

Aside: Color Images



- A color image has 3 channels.
- You can think of this image with dimensions 1024 x 1024 x 3

[152, 148, 136, 134, 135, 139, 135, 138, 136, 135, 128, 136, 140, 139,	
[146, 143, 139, 140, 140, 136, 140, 140, 140, 134, 142, 131, 135, 138,	
[140, 130, 141, 143, 137, 141, 141, 133, 137, 137, 143, 132, 114, 147,	
[139, 135,	[181, 174, 176, 183, 174, 174, 172, 175, 168, 176, 172, 177, 173, 177, 179,
[149, 147,	[177, 177, 174, 175, 174, 169, 174, 178, 178, 174, 179, 164, 172, 177, 175,
[163, 149,	[180, 175, 176, 170, 174, 181, 180, 176, 182, 174, 173, 162, 147, 177, 192,
[149, 146,	[178, 176, 175, 173, 173, 180, 173, 174, 171, 159, 173, 160, 124, 143, 186,
[110, 143,	[177, 176,
[81, 122,	[195, 181,
[107, 119,	[183, 186,
[50, 76,	[131, 176,
[72, 73,	[104, 137,
[74, 87,	[140, 140,
[67, 86,	[75, 87,
[66, 95,	[82, 75,
[60, 72,	[82, 104,
[38, 37,	[66, 102,
[38, 45,	[67, 100,
	[73, 79,
	[59, 52,
	[48, 45,
	[44, 59,
	[66, 46,
	[241, 248, 245, 241, 233, 242, 250, 246, 251, 232, 245, 239, 240, 239, 241,
	[242, 248, 242, 243, 246, 240, 245, 251, 241, 243, 249, 235, 240, 244, 241,
	[250, 240, 242, 247, 255, 250, 239, 244, 241, 255, 246, 186, 128, 211, 249,
	[255, 242, 244, 243, 244, 249, 235, 239, 240, 206, 245, 162, 55, 136, 220,
	[250, 242, 244, 227, 236, 254, 248, 247, 232, 131, 188, 112, 36, 102, 193,
	[246, 240, 247, 181, 186, 234, 235, 227, 245, 132, 92, 42, 53, 96, 181,
	[211, 238, 229, 122, 111, 158, 137, 141, 225, 158, 63, 65, 122, 109, 183,
	[152, 230, 180, 91, 98, 101, 32, 33, 159, 150, 86, 129, 136, 170, 238,
	[118, 158, 101, 111, 157, 116, 39, 22, 76, 111, 90, 87, 75, 165, 215,
	[111, 97, 71, 83, 59, 29, 48, 45, 46, 36, 36, 55, 98, 147, 101,
	[7, 19, 36, 32, 20, 21, 39, 46, 40, 23, 22, 32, 59, 133, 97,
	[19, 26, 25, 27, 39, 32, 25, 34, 29, 45, 46, 28, 42, 36, 95,
	[35, 24, 35, 40, 27, 35, 38, 27, 34, 36, 46, 33, 58, 98, 97,
	[46, 27, 29, 29, 22, 34, 26, 29, 30, 24, 38, 40, 155, 227, 183,
	[23, 44, 28, 22, 25, 22, 23, 44, 30, 31, 41, 34, 135, 164, 129,
	[29, 28, 26, 24, 18, 25, 27, 30, 53, 85, 50, 76, 74, 81, 57,
	[16, 13, 28, 34, 31, 26, 25, 36, 71, 121, 61, 56, 61, 115, 106,
	[24, 17, 44, 27, 15, 24, 43, 48, 60, 50, 26, 18, 29, 94, 139,
	[28, 39, 86, 50, 26, 38, 64, 87, 41, 16, 25, 29, 36, 38, 53,
	[33, 9, 33, 75, 73, 46, 46, 66, 79, 53, 27, 39, 51, 39, 5,
	[28, 8, 28, 46, 29, 17, 15, 42, 66, 64, 26, 23, 31, 61, 33,
	[19, 20, 33, 31, 22, 27, 27, 18, 38, 59, 35, 17, 35, 68, 57,

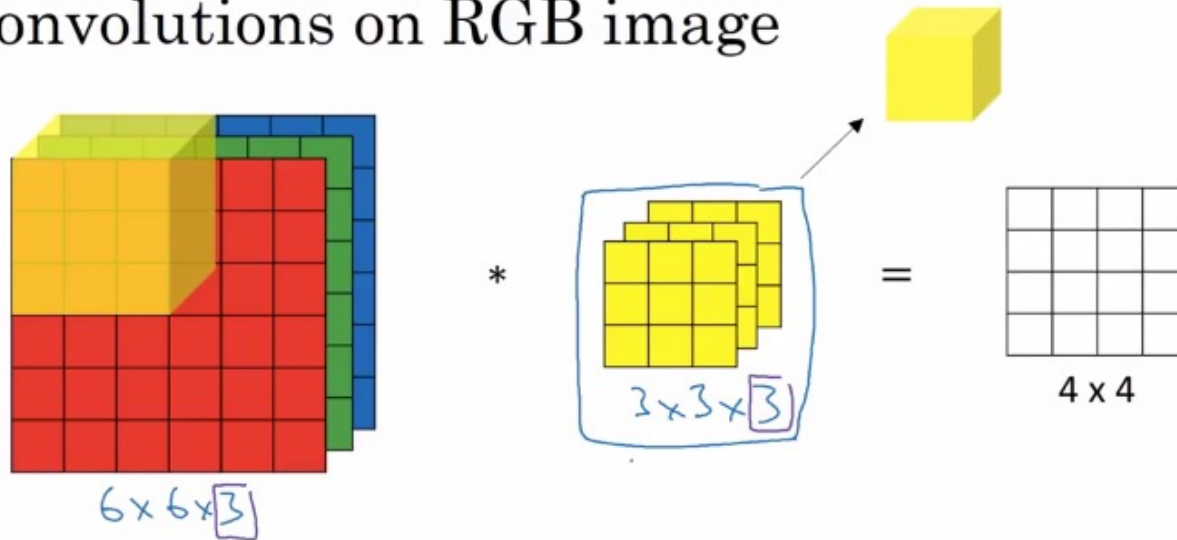
Red

Green

Blue

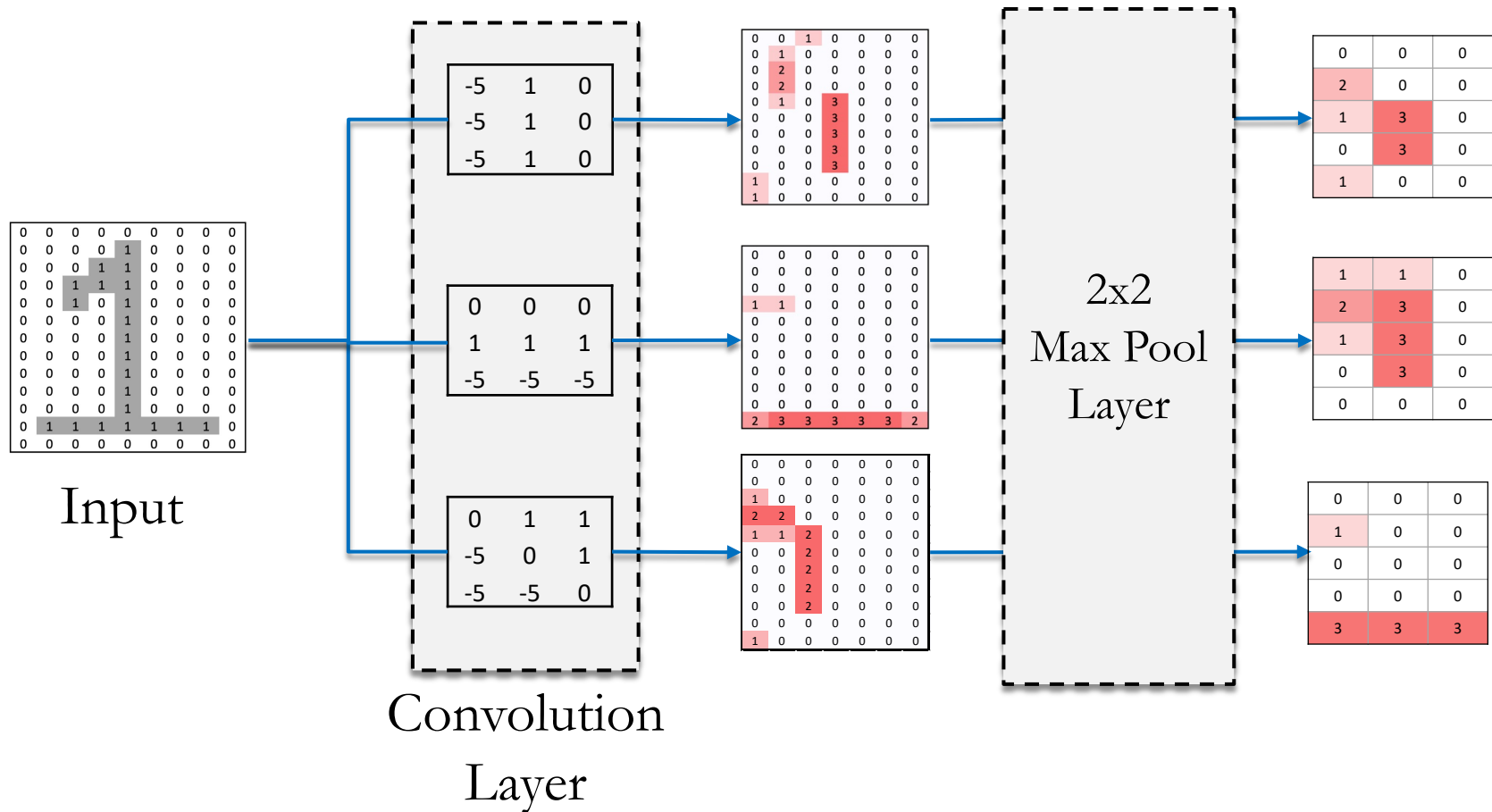
Applying a Convolutional Filter to a Color Image

Convolutions on RGB image

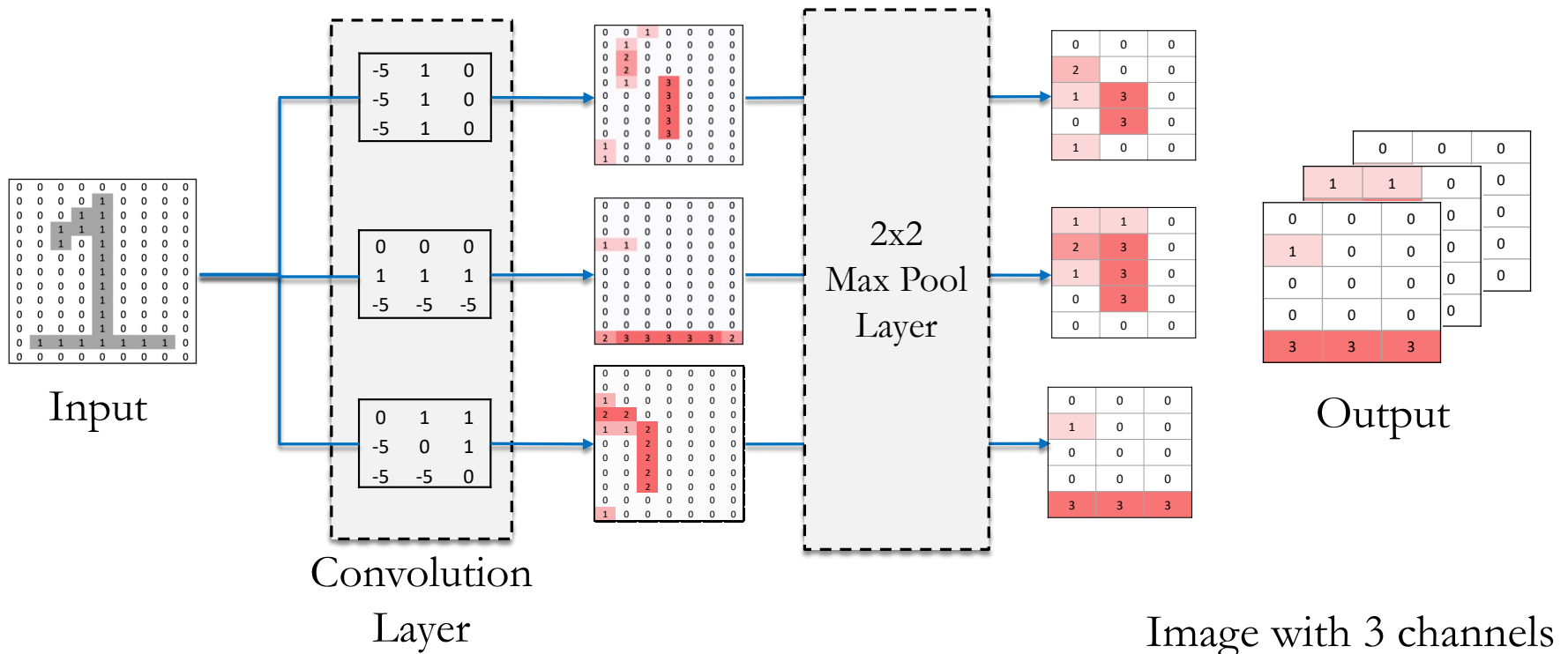


- If our input is a color image, (with 3 channels) then our filters will be a “cube” rather than a square. It will have dimension $k \times k \times 3$.
- The other aspects of the convolution operation are unchanged: sliding window sum-product + ReLU.
- The resulting heatmap still tells us where this cube appears often in the image.

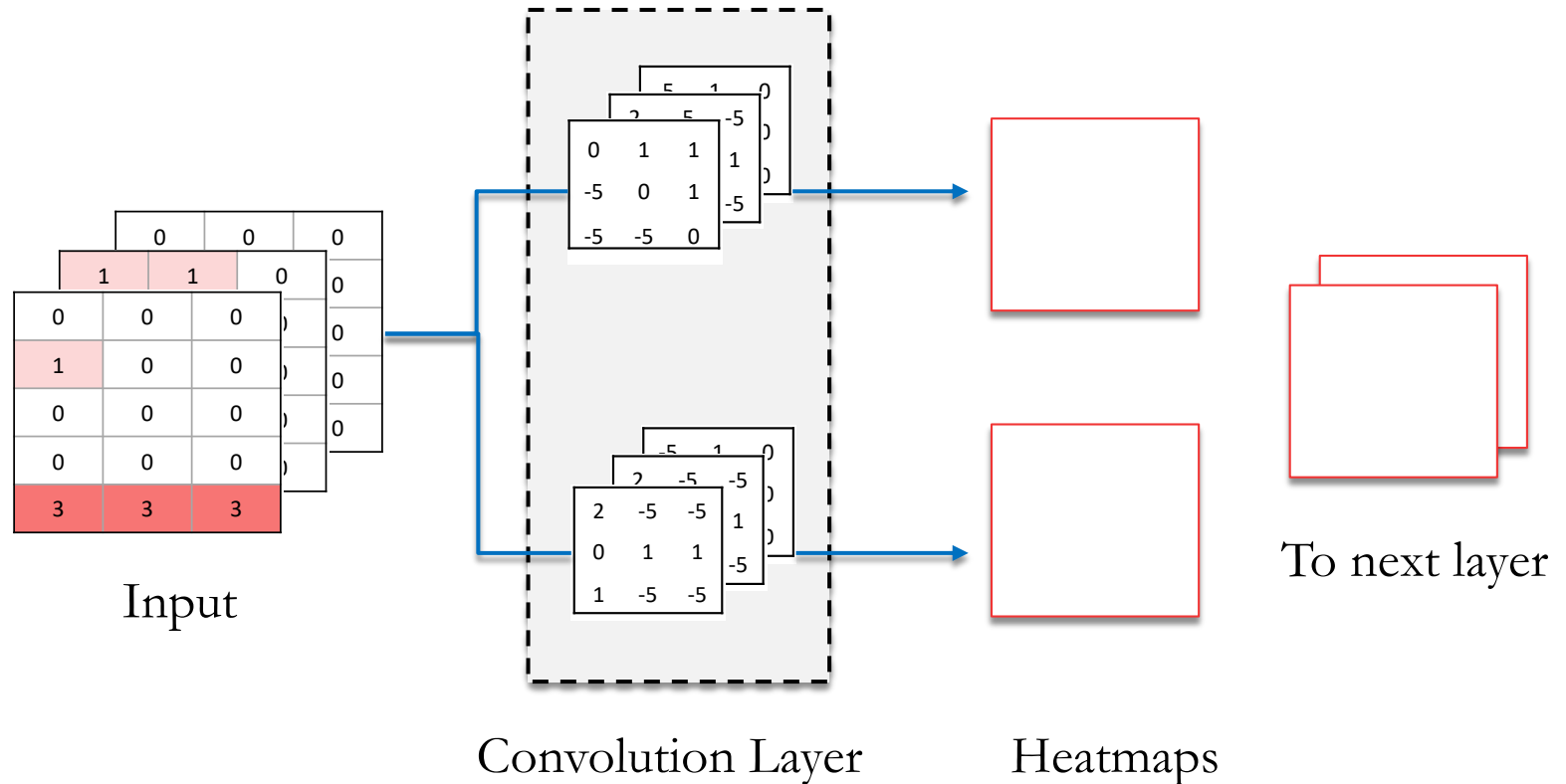
First Convolution + Pooling



First Convolution + Pooling



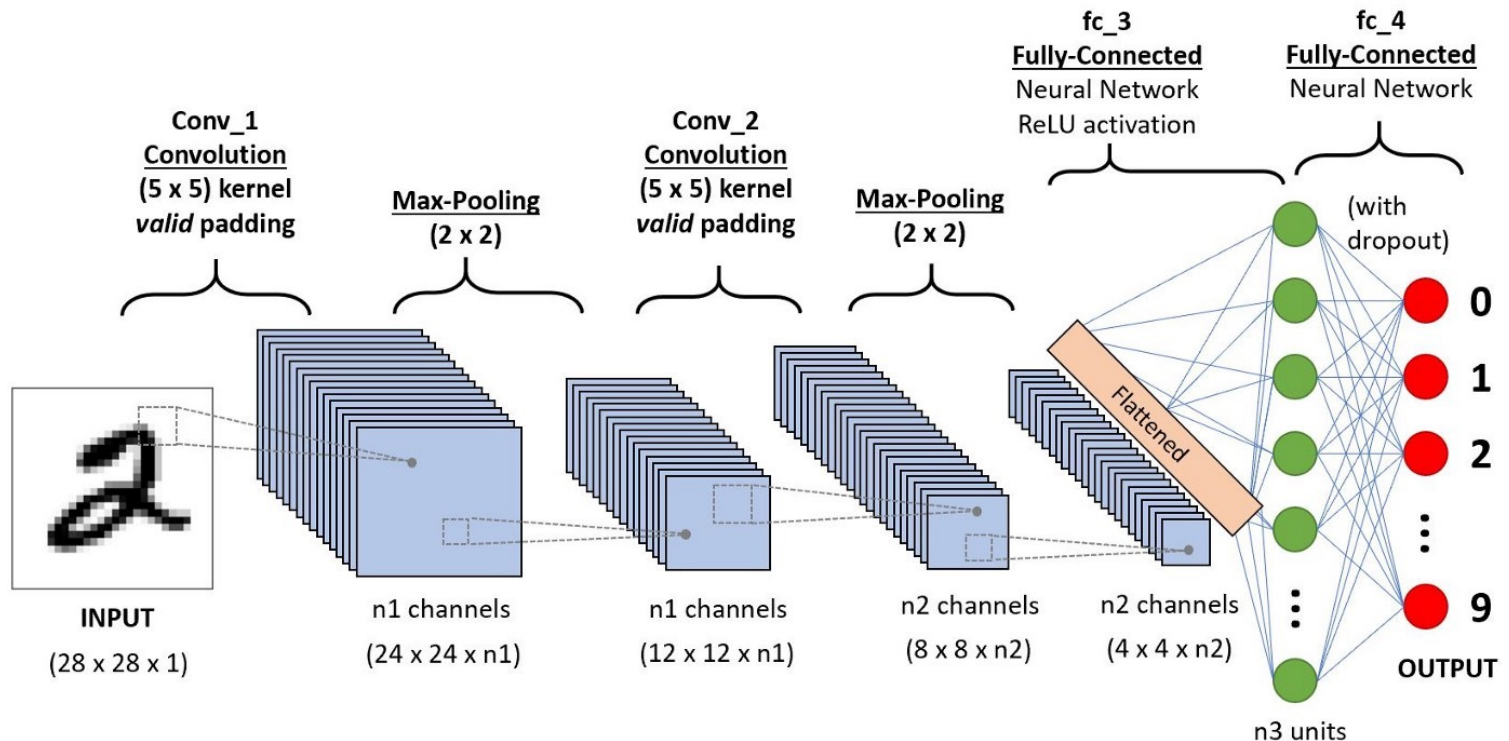
Second Convolution: 2 Filters



Since the input image has 3 channels, each filters also has 3 channels.

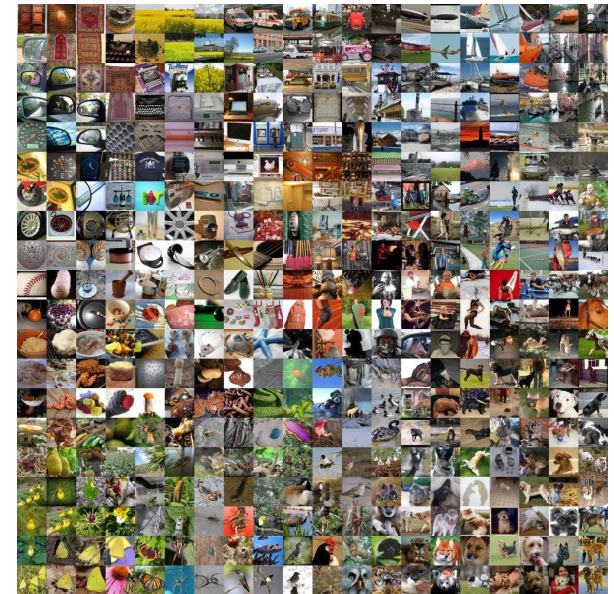
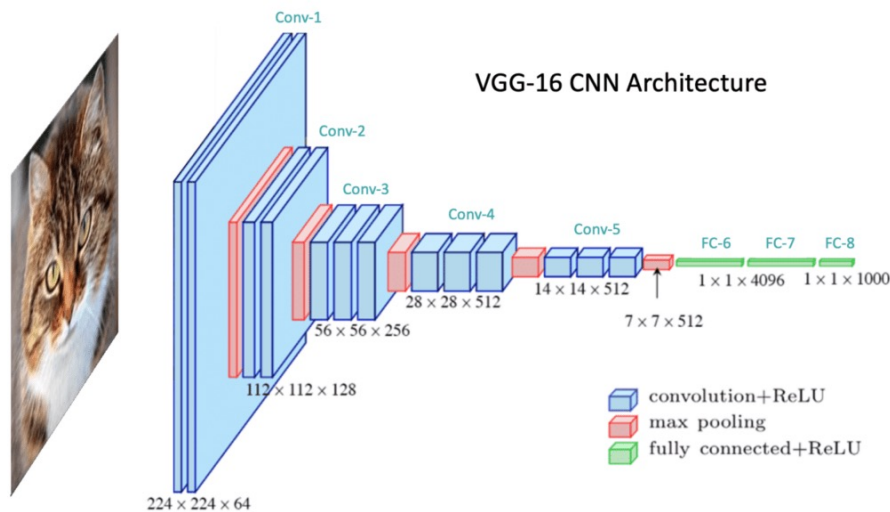
Modern CNNs

2021 – CNNs use many more filters



Source: <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>

Example



VGG-16 is a particular CNN trained by researchers at Oxford. Trained on ImageNet: 14 million images in 20,000 categories (e.g., strawberry, balloon)

CNN with Three Convolutional Layers

Prediction Model	Error Rate (%)	Accuracy (%)
CART	12.1	87.9
Random Forest	3.1	96.9
Neural Network with one hidden dense layer	2.1	97.9
Neural Network with two hidden dense layers	2.2	97.8
Neural Network with one Convolutional layer of 32 filters, plus one dense hidden layer	1.4	98.6
Neural Network with <u>three</u> Convolutional layers, two Pool layers, and one dense layer	0.9	99.1

- The error rate has dropped below 1%

Beyond CNN

- Modern alternatives to VGG-16
 - **ResNet-50 / ResNet-101**: Allowing deeper networks with far fewer parameters (e.g., ResNet-50 has ~ 26 M parameters).
 - **EfficientNet-B0 to B7**: Outperforming older networks while remaining lightweight.
 - **ConvNeXt**: Adopting design choices from Vision Transformers (ViTs).

Text Embeddings

Capturing the meaning of language

Recall: from Collaborative Filtering

Each user is characterized by 9 weights across archetypes

User	Archetype								
	1	2	3	4	5	6	7	8	9
John	0.7	0.1	...	...	...	...	...	...	0.2
Isabel	0.8	1.6	...	...	...	...	...	...	-0.3
Charles	0.2	-0.6	...	...	...	...	...	...	1.1
Paul	...	...	...	...	...	...	...	...	...
Rahul	...	...	...	...	...	...	...	...	...
Jose	0.2	0.3	0.1	0.9	-0.3	-0.4	0.8	-0.8	0.7
...	...	...	...	...	...	...	...	...	...

$W_{u,a}$: the weight of user u for archetype a

For example, in the above table: $W_{Jose,2} = 0.3$ and $W_{Jose,9} = 0.7$

Alternative interpretation

- Each viewer is represented by a 9-dimensional vector
- Or, a point in a 9-dimensional space
- Collaborative filtering is a method to “learn from data” a representation of such a 9-dimensional space, and where each viewer sits in such a space
- “Word Embedding”: representing each word as a point in a high-dimensional space
- Deep learning methods “learn from data” both a representation of a high-dimensional space, and where each word sits in such a space

Motivating Application

Product Search

Q waterproof winter jacket



Product A
winter fleece
gloves



Product B
waterproof
winter boots



Product C
ski sports
jacket



Product D
lightweight
fleece jacket

To a Computer...

Q 冬季防水夹克



Product A
冬季羊毛手套



Product B
冬季防水冬靴



Product C
滑雪运动夹克



Product D
轻质羊毛夹克

Simple Approach: Bag of Words

	waterproof	winter	jacket	fleece	gloves	boots	sports	ski	lightweight
query	1	1	1	0	0	0	0	0	0
A	0	1	0	1	1	0	0	0	0
B	1	1	0	0	0	1	0	0	0
C	0	0	1	0	0	0	1	1	0
D	0	0	1	1	0	0	0	0	1

The query is most similar to product B

Why Bag of Words Can Fail

- Bag of words treats each word as its own entity, like if it were a word in a foreign language you did not know.
- To calculate similarity between two phrases, simply count the number of similar phrases (e.g., waterproof winter jacket vs. winter fleece gloves)
 - Keyword matching: Treating each word as its own entity.
 - Find the product which matches the largest number of words from query.
- This approach fails to capture the meaning of words (synonyms).

Encoding the Meaning of Text

- It would be nice to know how “similar” two words are.

gloves

hoodie

parka

boot

jacket

sweater

fleece

shoe

Encoding the Meaning of Text

- It would be nice to know how “similar” two words are.
- In a two-dimensional space...

A 2D word embedding visualization showing relationships between clothing items. The words are arranged in a 2D space, with 'parka' and 'jacket' on the left, 'hoodie' and 'sweater' in the center, and 'gloves', 'boot', and 'shoe' on the right. 'fleece' is positioned below 'jacket'. The relative positions suggest semantic relationships, such as 'parka' being similar to 'jacket' and 'fleece', and 'gloves' being similar to 'boot' and 'shoe'.

parka jacket fleece hoodie sweater gloves boot shoe

Word and Sentence Embeddings

- Word Embeddings
 - Assigning a vector (list of numbers) to each word. The embedding captures (abstractly) the meaning of the word.
 - Words with similar meaning should have vectors that are close.
 - Words with opposite meanings should have vectors that are far apart.
- Sentence Embeddings
 - Obtain a vector that captures the meaning of a sentence.
 - Often obtained through averaging the word embeddings

Word Embeddings

- For each word in the English language, assign to it a list of numbers.
 - Typically a long list of numbers (300+ numbers)

Winter $\rightarrow$ [0.34, -0.45, ..., 2.74, -0.02]

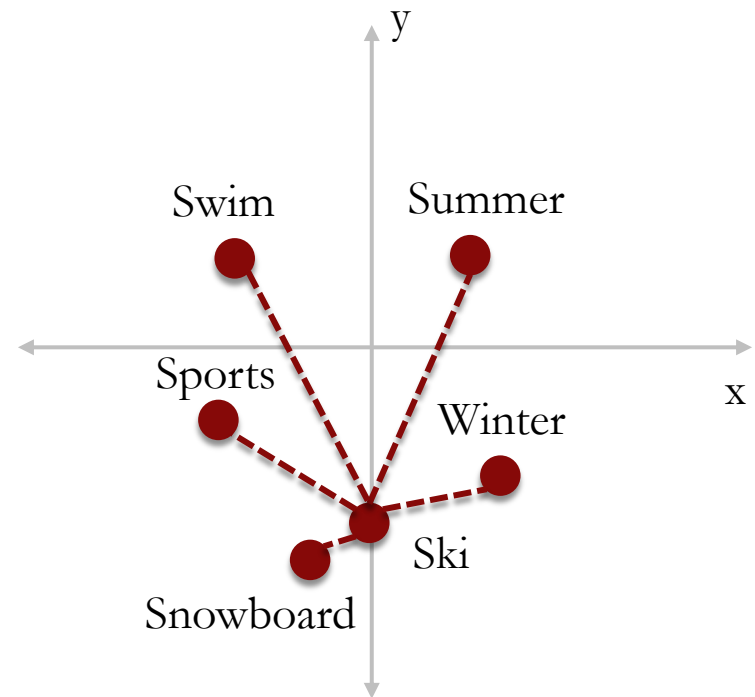
Summer $\rightarrow$ [0.33, 0.27, ..., 2.70, -1.02]

- You can think of each number as:
 - GPS Coordinates: A “location” for each word.
 - Features: (uninterpretable) features for each word.

Word Embeddings in 2-Dimensions

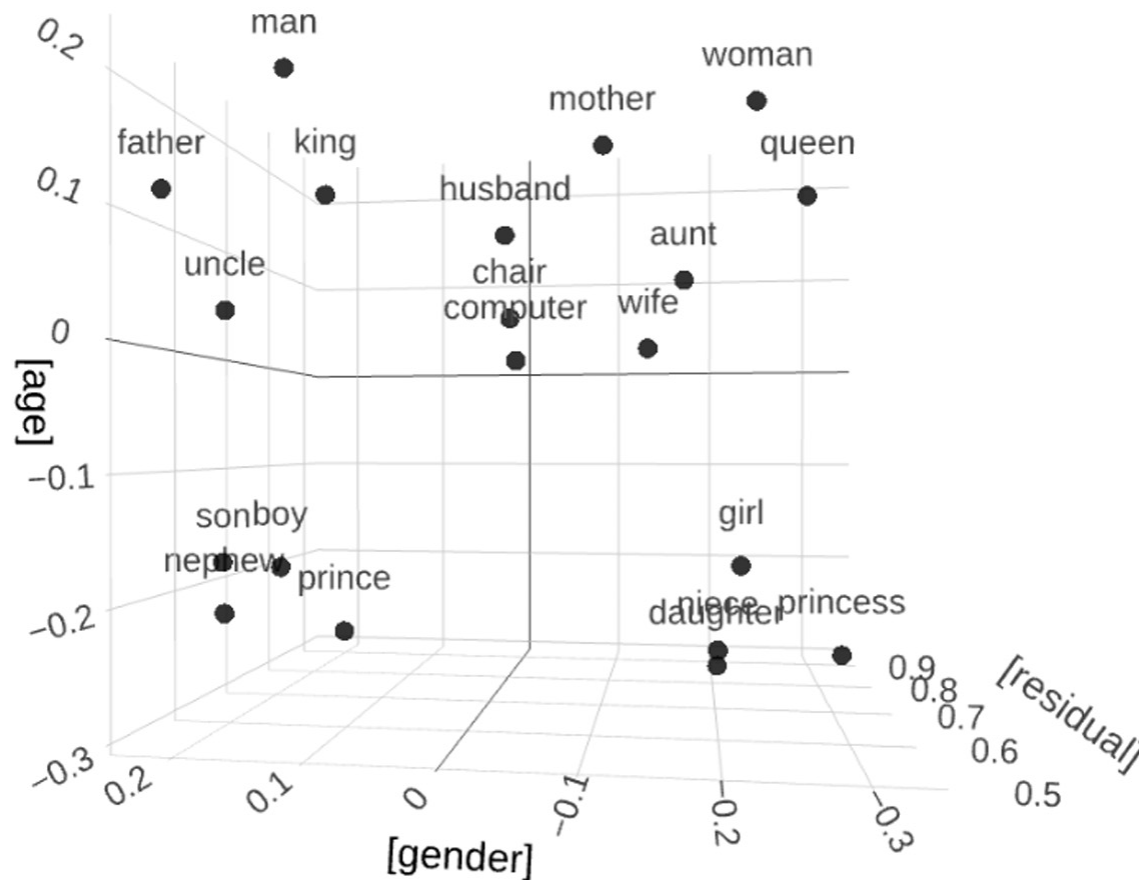
Winter $\rightarrow$ [0.34, -0.45]

Word	x	y
Winter	0.34	-0.45
Summer	0.33	0.27
Ski	-0.02	-0.47
Snowboard	-0.05	-0.51
Sports	-0.55	-0.08
Swim	-0.28	-0.14



----- Distances (closeness in meaning)

Word Embeddings in 3 Dimensions



How can Word Embeddings Help?

Q waterproof winter jacket



Product A
winter fleece
gloves



Product B
waterproof
winter boots



Product C
ski sports
jacket



Product D
lightweight
fleece jacket

“Sentence” Embedding by Averaging the Word Embeddings

Q waterproof winter jacket

→ [-0.19, -0.16]



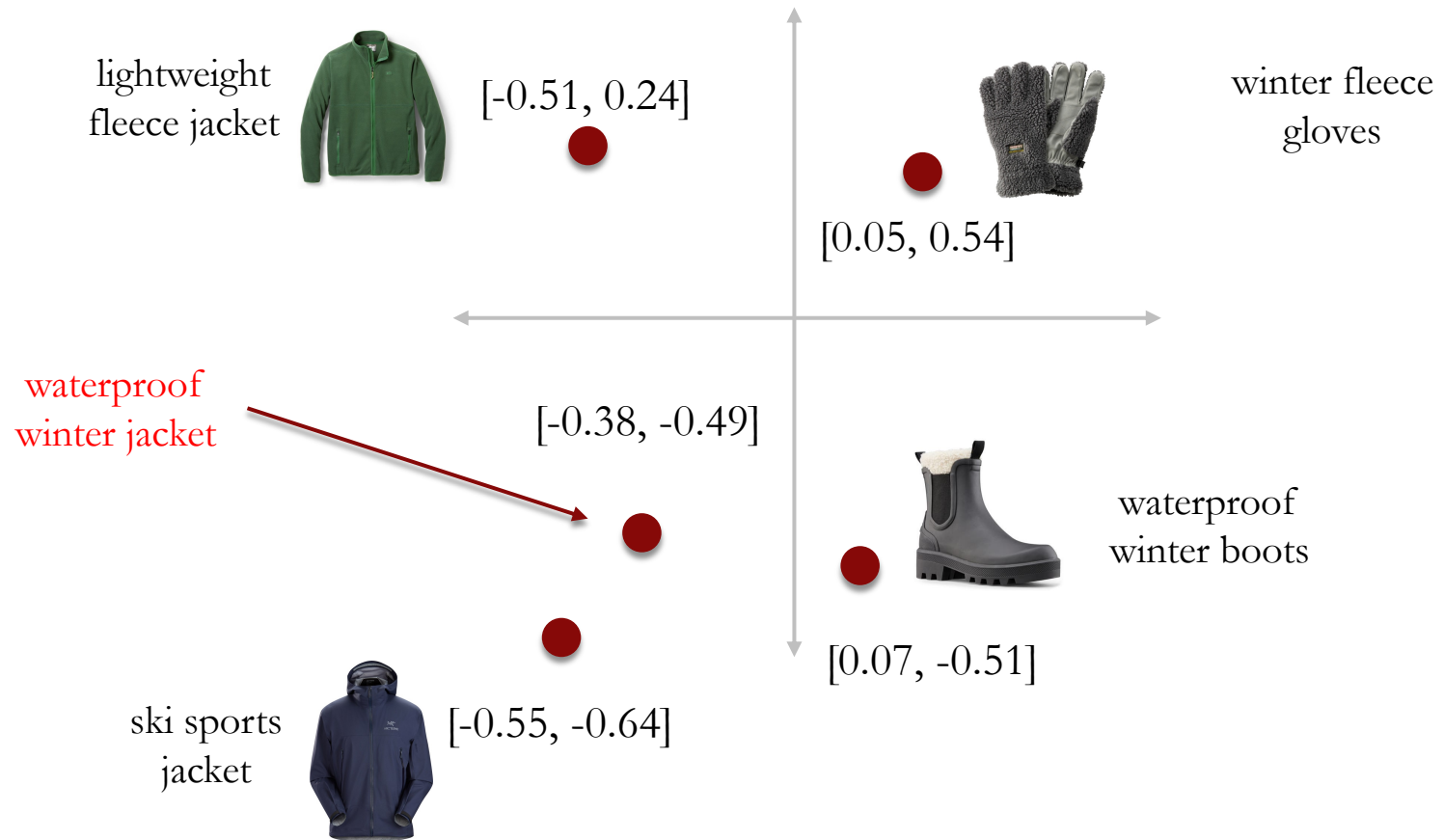
Product C

ski sports
jacket

→ [-0.18, -0.21]

Word	x	y
Waterproof	-0.65	-0.46
Winter	0.09	0.44
Jacket	-0.12	-0.47
Ski	-0.62	-0.17
Sports	0.19	-0.08
...	...	...

Calculate Distance Between Query and Products



How to Obtain Word Embeddings?

- The short answer... you download them from Google, Stanford, Meta...etc.
 - Organizations who have spent hours if not days finding embeddings that best represent the English language by reading lots and lots of text (Wikipedia, news articles, books...etc.)
- Well... how did these organizations build these embeddings?

How to Obtain Word Embeddings

- The key challenge is to **evaluate the quality of some word embeddings**.
 - If we can evaluate, then we can generate using optimization.

Word	x	y
Winter	0.34	-0.45
Summer	0.33	0.27
Ski	-0.02	-0.47
Snowboard	-0.05	-0.51
Sports	-0.55	-0.08
Swim	-0.28	-0.14

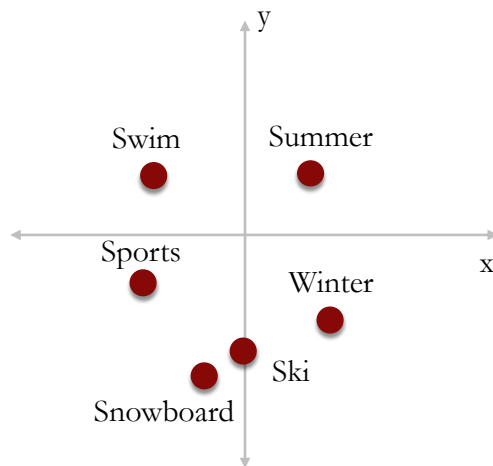
This is Good

Word	x	y
Winter	0.34	-0.15
Summer	0.35	0.24
Ski	-0.22	0.27
Snowboard	-0.45	0.49
Sports	-0.89	1.09
Swim	-0.38	2.34

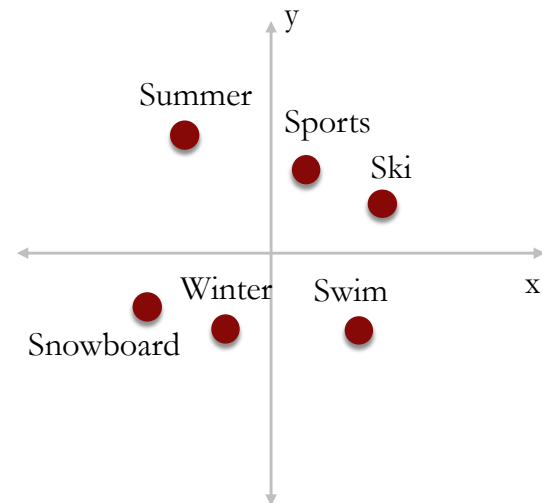
This is Bad

How to Obtain Word Embeddings

- The key challenge is to **evaluate the quality of some word embeddings**.
 - If we can evaluate, then we can generate using optimization.



This is Good



This is Bad

How do we know if two words are related?

- "You can understand a word by the company it keeps."
 - A word's meaning can be inferred from words that it often occurs near.
- For example:
 - Ski, snowboard, winter, sports often occur in the same sentence.
 - Summer and winter often occur in the same sentence
 - Winter and swim probably don't occur so often. Same for ski and summer.
- A good embedding model should assign distances that align to this observation.
- There are numerous ways to define this mathematically.

Pre-Trained Embedding Models

Method	Organization	Training Corpus	# of Words
Word2Vec	Google	News articles	100 Billion
GloVe (Global Vectors)	Stanford	Wikipedia + internet pages (common crawl)	840 Billion
FastText	Meta AI Research	Wikipedia + internet pages (common crawl)	600 Billion

Each word embedding model is based on a different metric that is optimized. The embeddings are pre-trained on a large corpus of documents, something that only a corporation has the resources to do.

Each Embedding Model

- Each embedding model outputs a vector for each word in the English language.

a	→	[-2.34, -0.56, ... 3.19]
aardvark	→	[8.34, -7.46, ... -5.19]
...		
zebra	→	[9.19, -5.55, ... 1.84]
zygote	→	[-3.14, -2.06, ... 0.49]

Facts about Word2Vec Embeddings

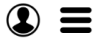
- **Size:** 3 million unique word/phrases
- **Dimensionality:** 300 dimensions per word
- File size: ~3.4GB
- Includes phrases (like "New_York")
- Case-sensitive (e.g., "Apple" and "apple" have different vectors)

Semantle

<https://semantle.com/>

- Semantle is a Wordle-like game where you try to guess a secret word.
- Every time you make a guess, you are told a **similarity score** between your guess and the secret word.
- These scores are determined by Word2Vec.

SEMANTLE 🔍



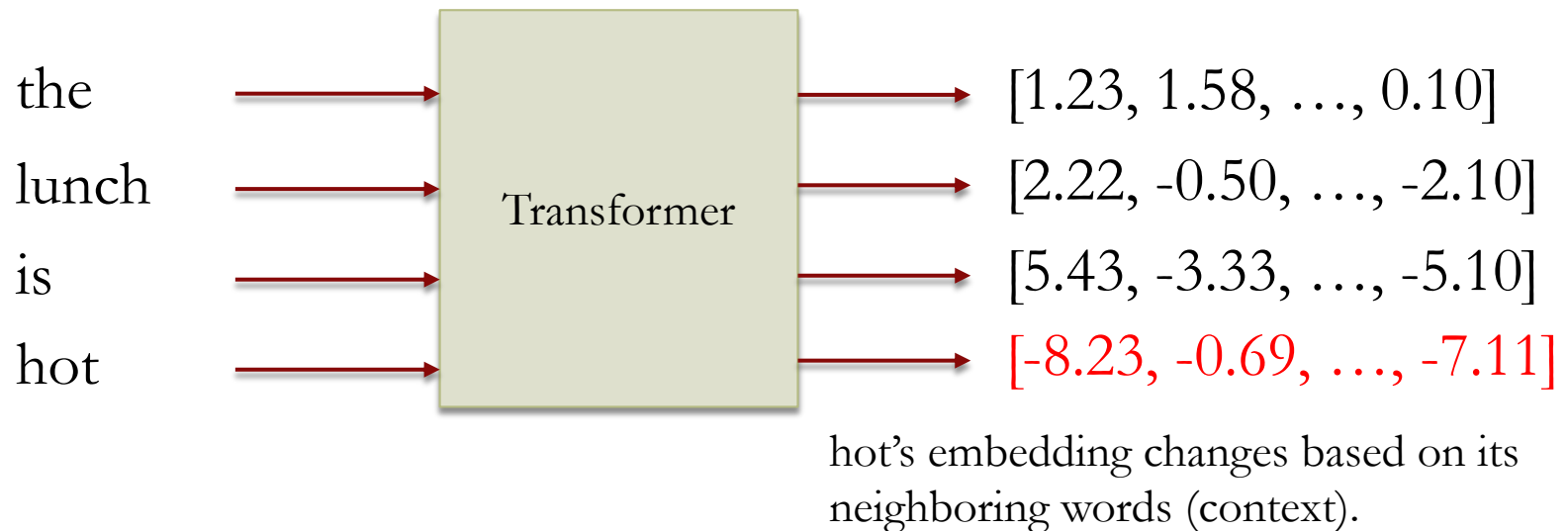
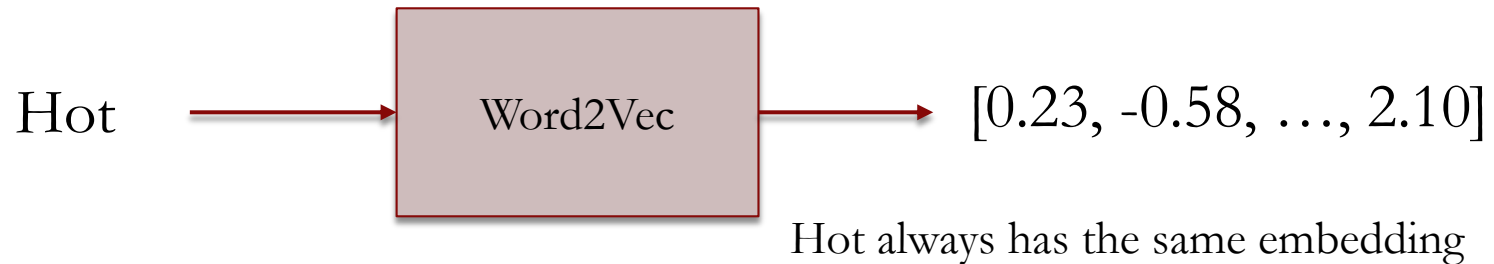
The nearest word has a similarity of **77.22**, the tenth-nearest has a similarity of **60.07** and the thousandth nearest word has a similarity of **40.29**

Game #1039

Guess

#	Guess	Similarity	Proximity
15	ape	55.97	979 / 1000
Sort - Similarity			
14	monkey	49.89	870 / 1000
12	animal	46.31	663 / 1000
13	chimpanzee	39.95	(tepid)
8	human	39.65	(tepid)
4	person	36.19	(tepid)
9	humanity	32.21	(tepid)
3	adjective	22.83	(tepid)
11	mother	22.18	(tepid)
1	noun	22.03	(tepid)
6	scientist	21.94	(tepid)
2	verb	18.47	(cold)
5	washington	7.28	(cold)
10	agent	5.90	(cold)
7	engineer	2.50	(cold)

Contextual Embeddings



IMDB Review Sentiment Analysis

- 50,000 reviews in total (25,000 positive, 25,000 negative)
- Each review is a free-form English text string.
- The target of interest is whether the review expresses a **positive** or **negative** sentiment towards the movie.



"A delightful surprise! The humor was spot-on, and the emotional depth caught me off guard. I'll be rewatching this one for sure."

→ Positive

Cleaning + Bag of Words Vectorization

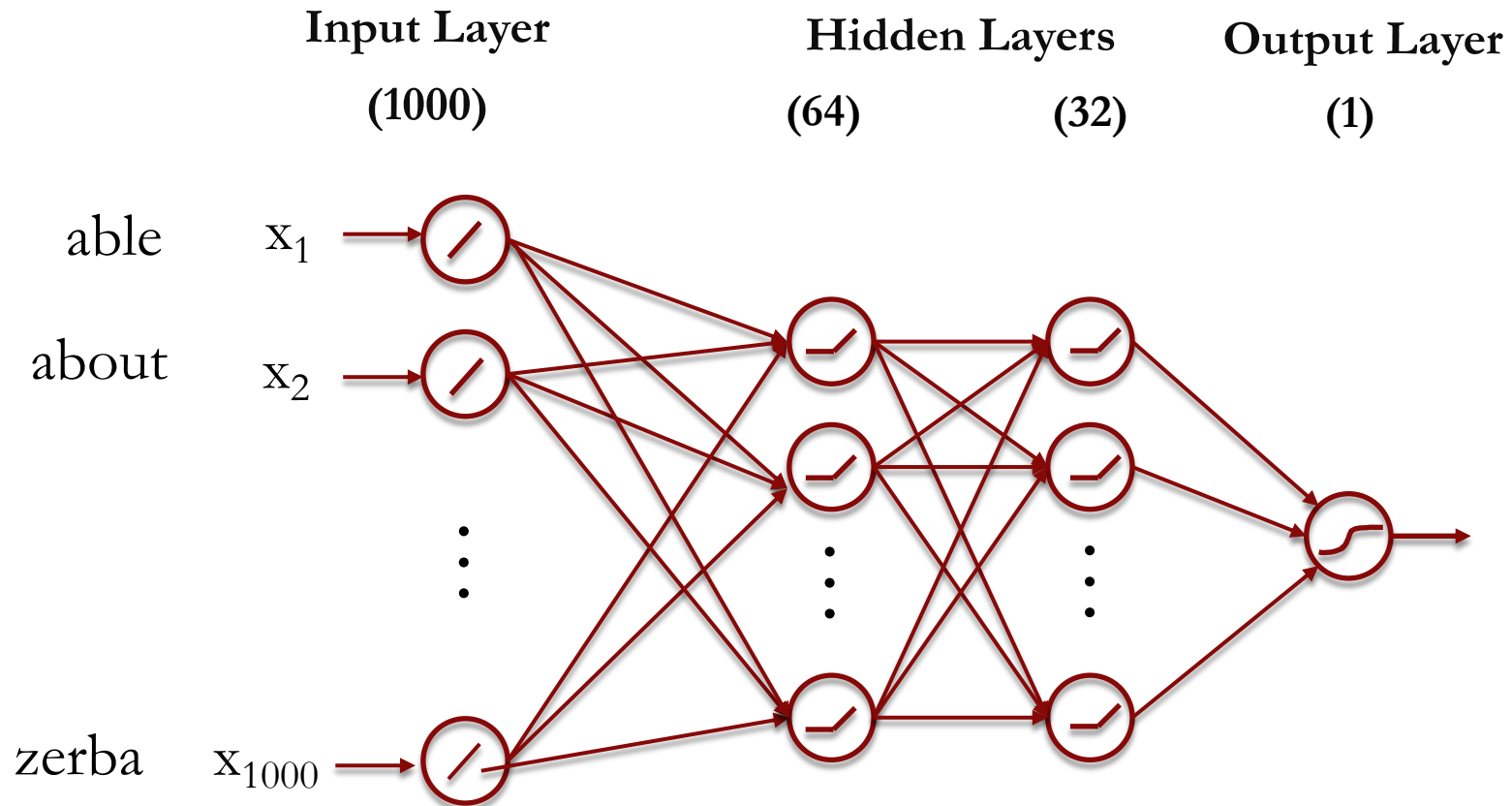
able	about	above	absolutely	across	act	acted	acting	...	year	years	yes	yet	york	you	young	your	yourself	zombie	is_positive
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	2	0	0	4	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0
0	1	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	1	0	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	2	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	2	1	0	0	1	2	0	1	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

Each row is a movie review (5000 in total)

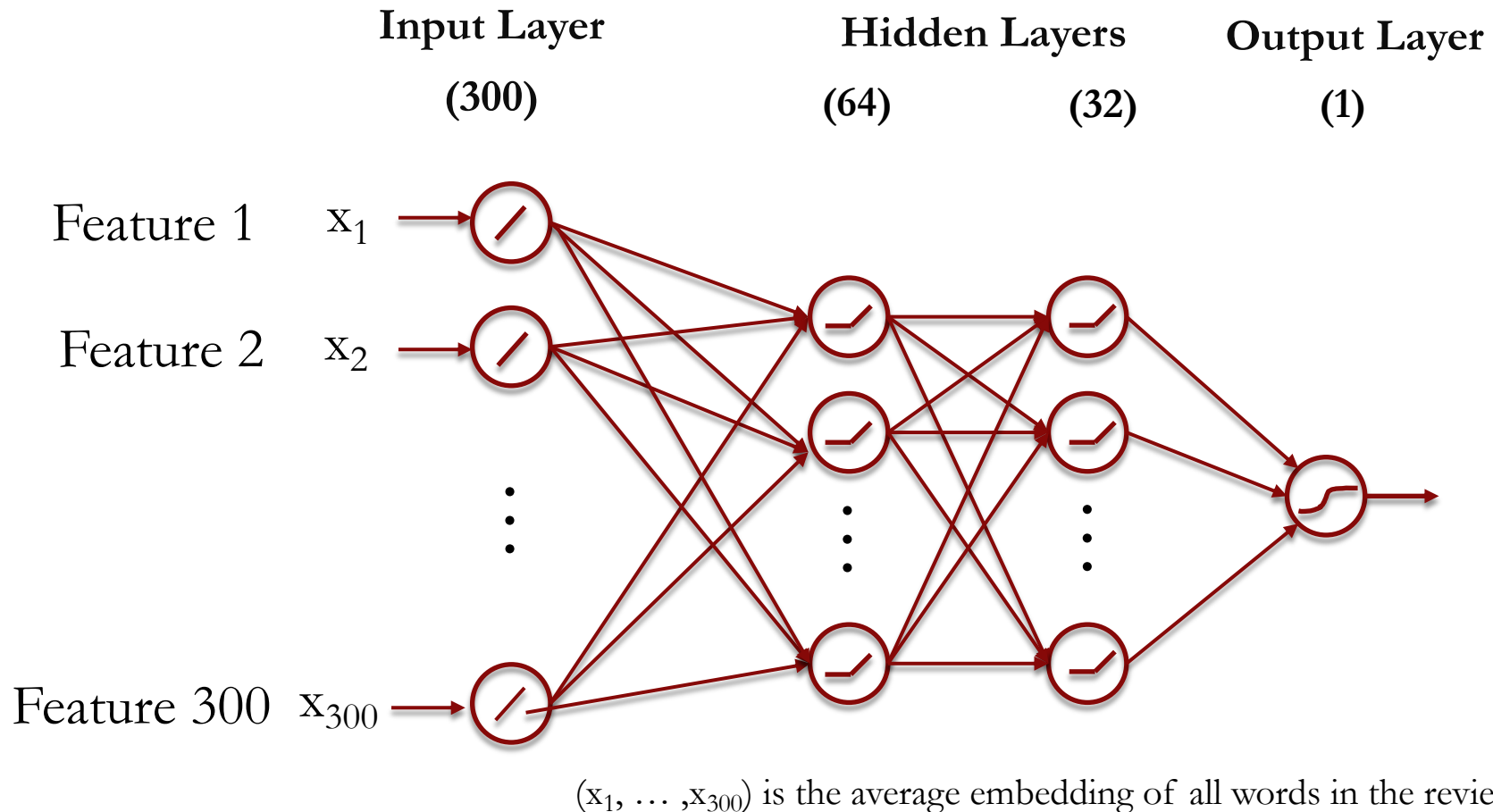
Each column is a word (only the top 1000 words)

The values are how many times the word appears in the review.

Bag of Words NN



Word2Vec/Transformer Embedding NN



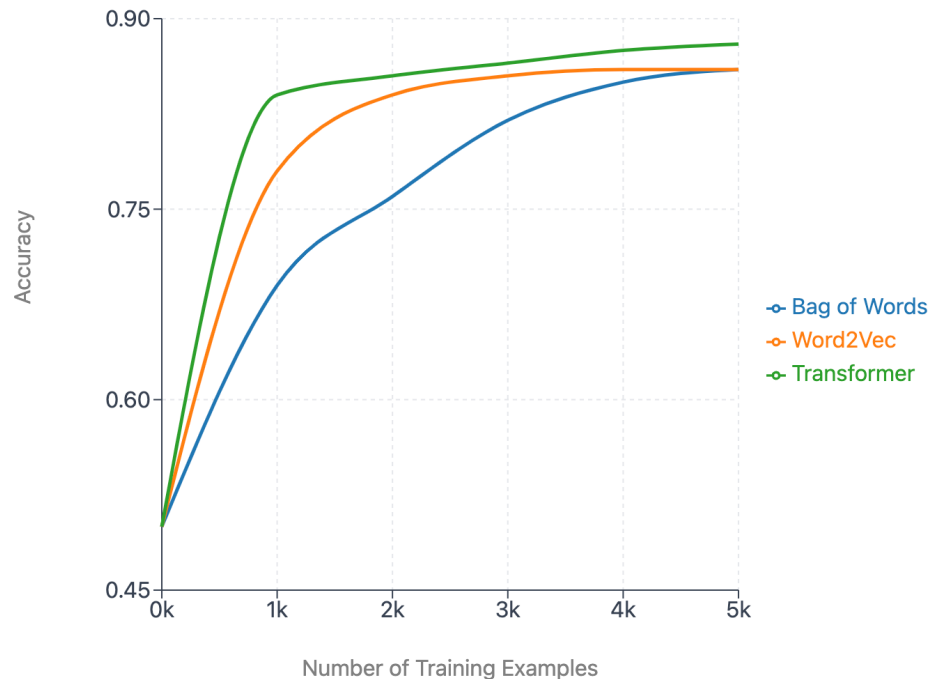
Sentiment Classification Results

Prediction Model	Error Rate (%)	Accuracy (%)
Logistic Regression / CART	28.4%	71.6%
Random Forest	18.2%	81.8%
Neural Network with one hidden layer	16.6%	83.4%
Neural Network with two hidden layers	14.1%	85.9%
NN using Word2Vec embedding	14.1%	85.9%
NN using BERT embedding	12.3%	87.7%

- Word2Vec does not do better overall, but can achieve this performance with fewer training examples (next slide).
- BERT does better because it takes into context (e.g., sarcasm).

Number of Examples Needed

Sentiment Analysis Model Comparison

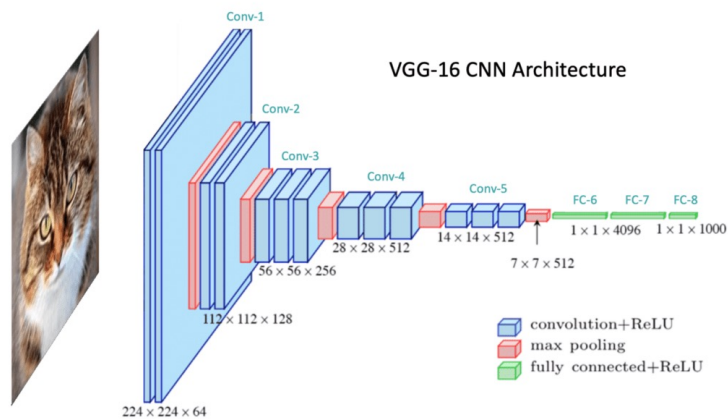


- Although Word2Vec and BERT do not improve the overall performance much, they work well with much fewer examples.

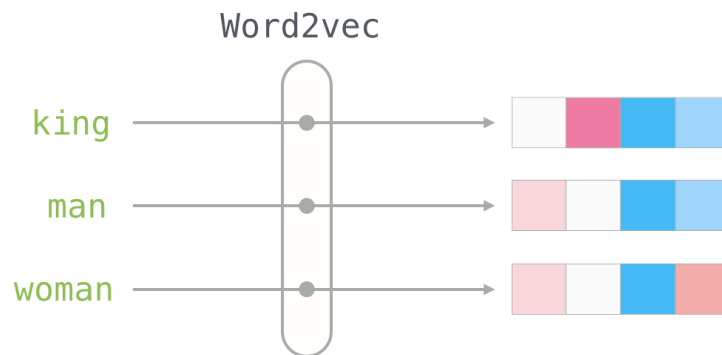
Trade-Offs

Aspect	Neural Networks	Traditional Models (e.g., Logistic Regression)
Performance on Unstructured Data	Excels at learning representations, extracting useful features	Struggles to detect complex patterns
Interpretability	Often considered "black box" models	Typically easier to interpret, coefficients indicate feature importance
Data Requirements	Generally requires large datasets	Can work well with smaller datasets
Computational Resources	Typically require more computational power including GPU's	Less computationally intensive
Training Time	Training can take days/weeks	Generally faster to train

Pre-Trained Models



ImageNet



Wikipedia

Transfer Learning and Fine-Tuning

- Pre-trained models learn general knowledge from a massive dataset (like Word2Vec learning English from Wikipedia, VGG from ImageNet)
- The model is great at general-purpose tasks like embedding text or classifying images.
- This model has an easier time learning specific tasks:
 - Classifying images that were not in ImageNet
 - Text that is not publicly available on the internet
- Taking an existing pre-trained model as a starting point. **Fine-Tune** it to your specific application.
 - Requires much less data than training from scratch
 - Preserves general knowledge while learning task-specific patterns

Conclusion

- Convolutional neural networks and word embeddings are two deep learning architectures designed specifically for image and text data. They typically outperform using “generic” approaches like feed forward neural networks.
- CNN’s are based on the convolution operator, which generates a heatmap of where a small patch of pixels (the filter) occurs in the image.
- Word embeddings capture the meaning of words and sentences by using high-dimensional vectors, where distances between vectors encode similarity in meaning.